

Control por Computador

Indice

- ❑ Introducción al control de procesos por computador
- ❑ Conceptos básicos para programación de control
- ❑ Interfaz del computador con el exterior
- ❑ El control secuencial
- ❑ Las matemáticas del control
- ❑ Programación del lazo de control
- ❑ Programación en lenguaje C
- ❑ Implantación del control en el computador

Indice

- ❑ **Introducción al control de procesos por computador**
- ❑ Conceptos básicos para programación de control
- ❑ Interfaz del computador con el exterior
- ❑ El control secuencial
- ❑ Las matemáticas del control
- ❑ Programación del lazo de control
- ❑ Programación en lenguaje C
- ❑ Implantación del control en el computador

El Control de Procesos por Computador

□ Proceso o sistema

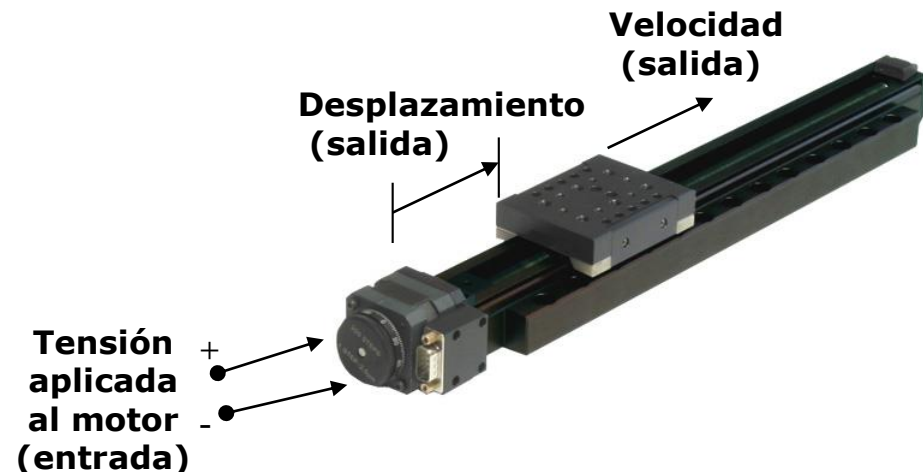
- Conjunto de elementos físicos que cumplen un cometido común
- **Salidas del sistema:** valores medibles cuya variación en el tiempo se desea controlar
- **Entradas al sistema:** acciones que se pueden realizar para modificar los valores de las salidas



Ejemplo 1: sistema térmico
(calentador de fluido por gas)



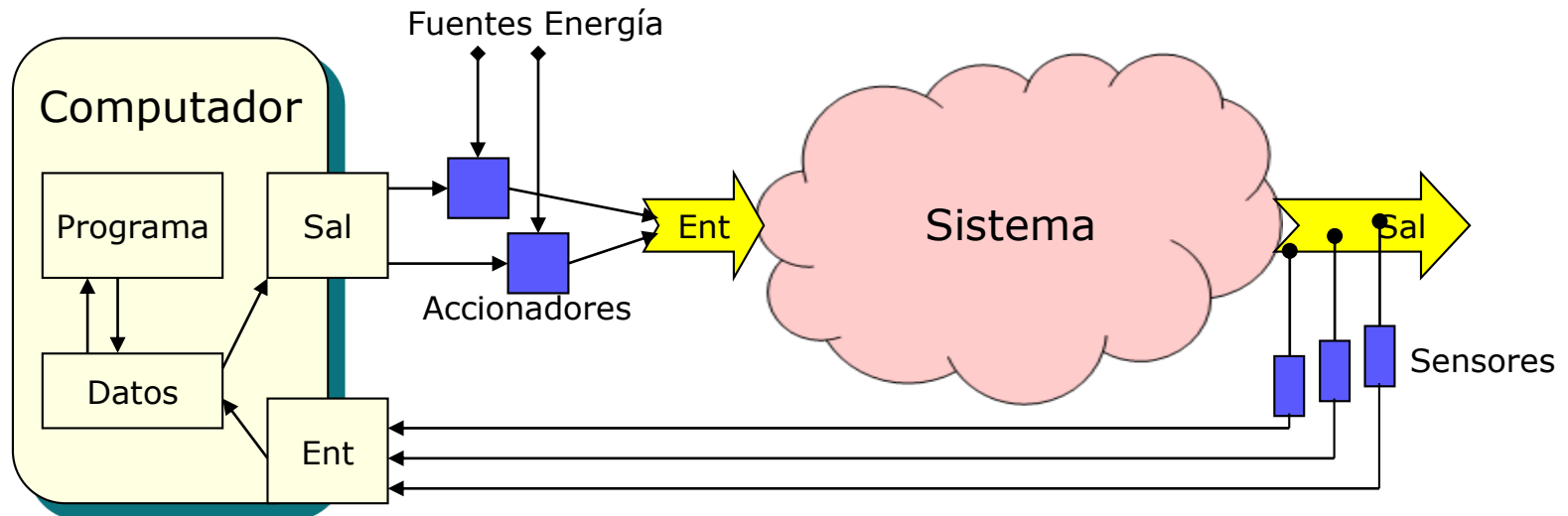
Ejemplo 2: sistema mecánico
(motor CC + guía lineal)



El Control de Procesos por Computador

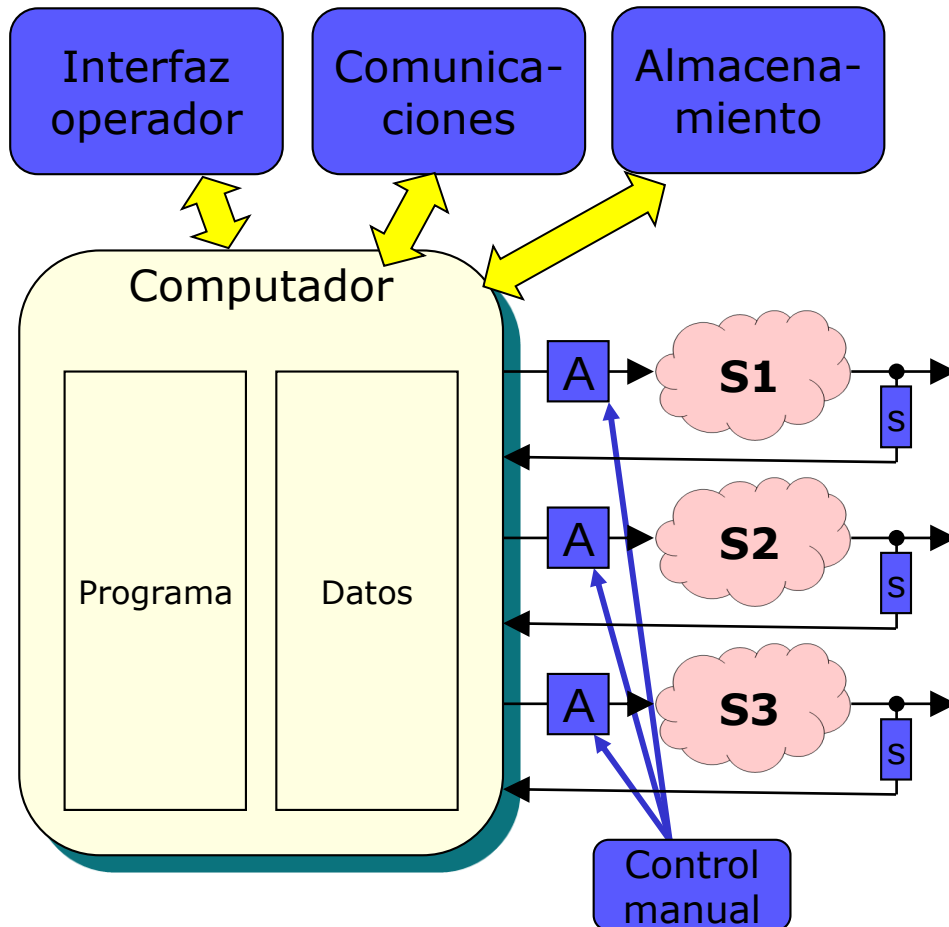
- Control de un proceso o sistema
 - Variables principales involucradas:
 - **Referencia o consigna:** valor deseado para la salida en cada momento
 - **Error:** Valor referencia – Valor salida
 - **Acción de control:** valor a aplicar en la entrada para conseguir (idealmente) error cero
 - Todas las variables evolucionan en el tiempo

- Control de proceso por computador
 - Un **Computador** lee las Salidas del sistema, realiza cálculos de control, y modifica las Entradas del sistema de acuerdo con estos cálculos
 - Operación en **Tiempo Real**: es importante que la acción de control sea correcta y realizada en tiempo



El Control de Procesos por Computador

❑ Elementos de un equipo de control



▪ Computador:

- El “cerebro” del control
- Programa y datos alojados en memoria

▪ Sensores:

- Dan al computador información del estado de los sistemas, midiendo sus salidas.

▪ Accionadores:

- Permiten al computador generar cambios sobre los sistemas, modulando la energía entregada por fuentes externas

▪ Interfaz operador:

- Panel(es) que permite(n) al humano intervenir en el control, generando órdenes y recibiendo informaciones

▪ Comunicaciones:

- Interconexión con otros computadores para intercambio de informaciones y comandos

▪ Almacenamiento:

- De configuraciones y resultados

▪ Control manual:

- Reemplaza las salidas del computador en caso de error o alarma

Ejemplos de sistemas



Control de temperatura en horno-túnel



Teléfono móvil



Robot de encolado automático



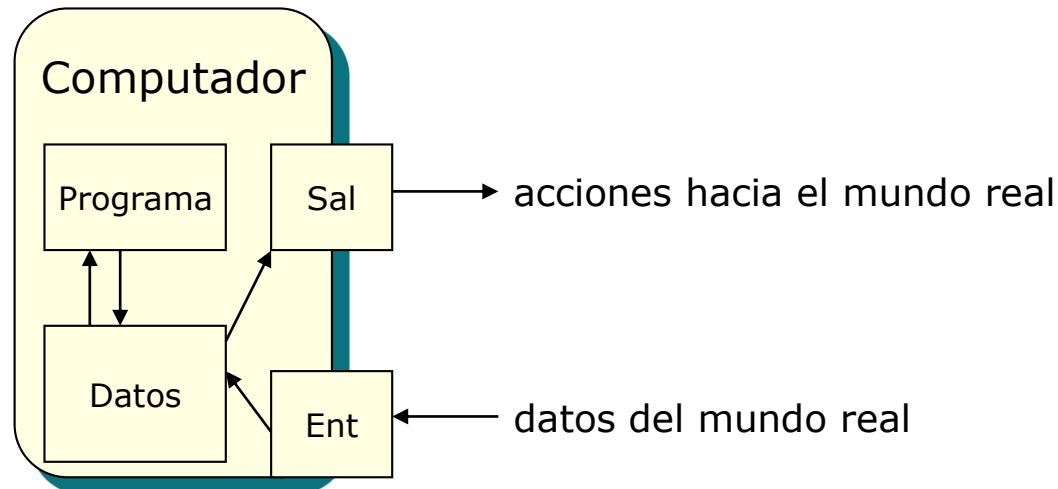
Control de vuelo de un avión



Línea de fabricación de botes 7

Computadores para control

- ❑ El computador es capaz de:
 - Aceptar información digital procedente del exterior a través de sus entradas.
 - Almacenar la información (datos) en su memoria (variables).
 - Procesar los datos de acuerdo a un programa (secuencia de instrucciones sencillas) instalado en su memoria.
 - Generar comandos hacia el exterior a través de sus salidas.



Computadores para control

- ❑ Microcontrolador



- ❑ Procesador Digital de Señal (DSP)



- ❑ Dispositivos Electrónicos Programables (FPGA, PLD)

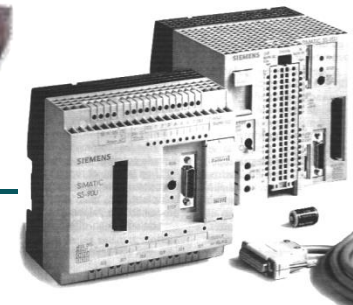
- ❑ Computador embebido



- ❑ Ordenador Industrial

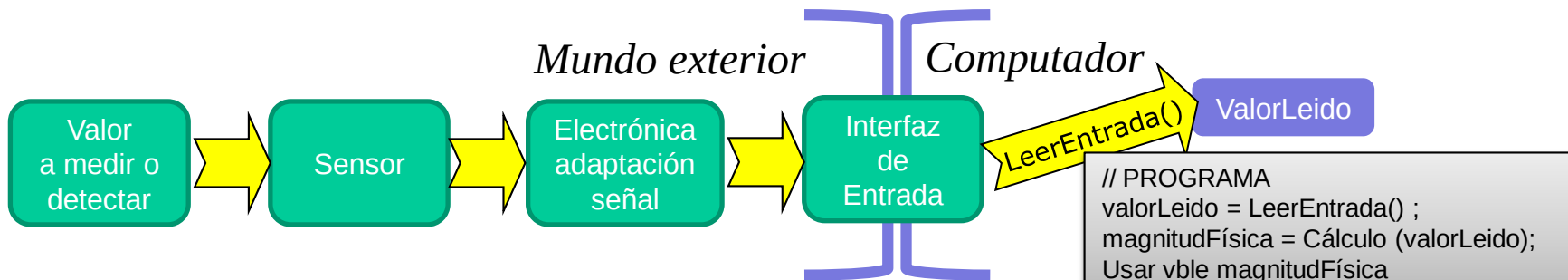


- ❑ Autómata Programable (PLC)



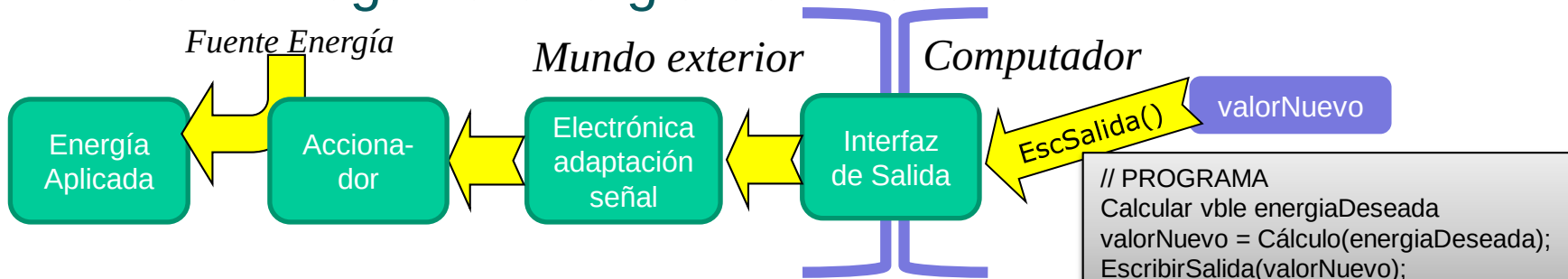
Sensores

- ❑ Convierten magnitudes físicas en valores de pequeña tensión/corriente.
- ❑ Clasificaciones:
 - Según la magnitud medida:
 - De posición (lineal/angular)
 - De velocidad (lineal/angular)
 - De temperatura
 - De presión
 - De presencia
 - ...
 - Según el formato del valor entregado:
 - Analógicos
 - Digitales todo/nada
 - De tren de pulsos
 - Numéricos
- ❑ Para obtener el valor medido:

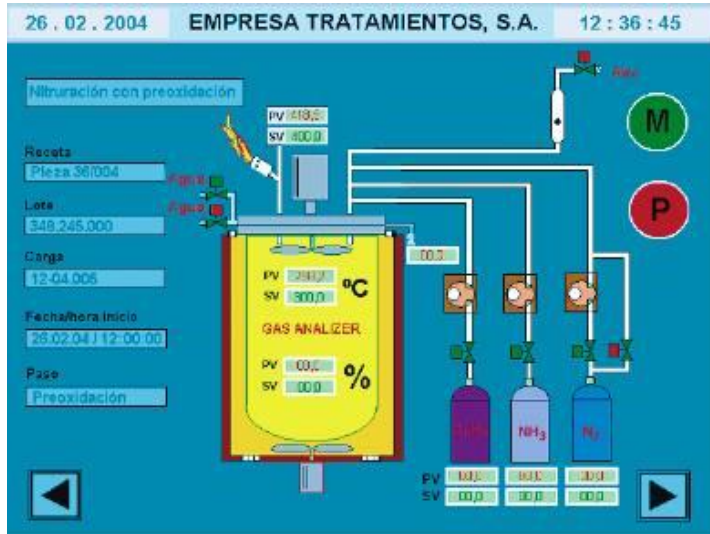


Accionadores

- ❑ Permiten el comando del sistema mediante la energía procedente de una fuente externa, modulada por un valor de pequeña tensión/ corriente.
- ❑ Clasificaciones:
 - Según la energía:
 - Eléctricos
 - Neumáticos
 - Hidráulicos
 - ...
 - Según la acción:
 - Desplazamiento
 - Giro
 - Calentamiento
 - Compresión
 - ...
 - Según la modulación:
 - Analógicos
 - Digitales todo/nada
 - PWM
 - Numéricos
- ❑ Para entregar la energía deseada:



Interfaz con el operador



Interfaz tipo SCADA



Pupitre / botonera



Display gráfico personal

■ Dispositivos salida interfaz humana:

- Indicador luminoso (LED)
- Display (7 segmentos, LCD,...)
- Pantalla (texto/gráficos)
- Impresora
- ...

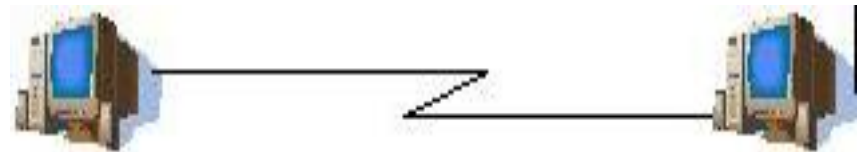
■ Dispositivos entrada interfaz humana:

- Botón / pulsador
- Selector rotativo
- Teclado
- Ratón / Puntero / Joystick
- ...

Comunicaciones

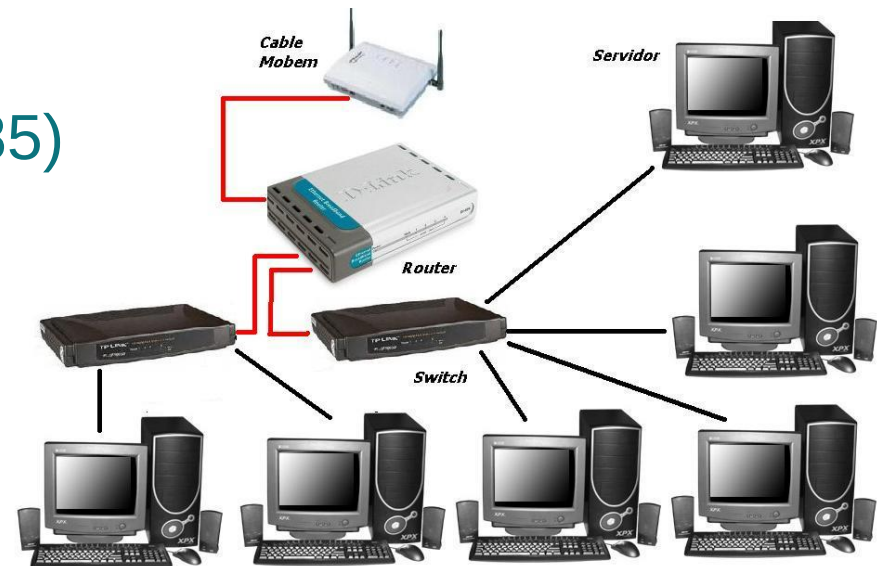
□ Punto a punto:

- Serie (RS-232,USB)
- Paralelo
- Inalámbrica (Bluetooth)



□ Multipunto:

- Serie (I2C, CAN, RS-485)
- Red (Ethernet)
- Inalámbrica (Wifi)



Niveles de control

□ Control secuencial:

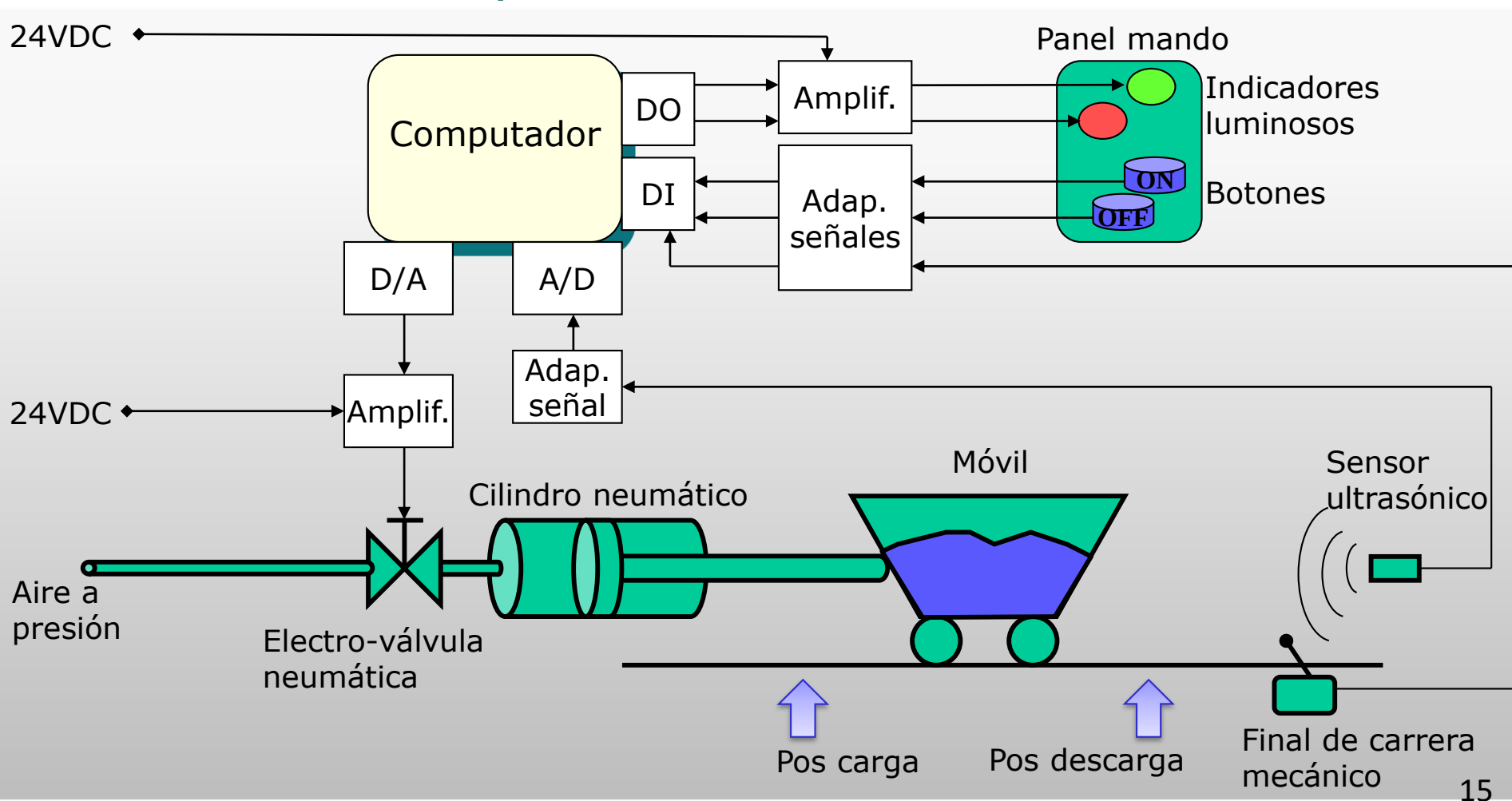
- Genera la secuencia de acontecimientos deseada
- Determina los valores deseados (consignas) de las variables a controlar
- Diagrama de estados / transiciones
- Cambio de estados por:
 - Tiempo máximo
 - Combinación de valores de entradas u otras variables
 - Acciones del operador o de otros computadores

□ Lazo de control:

- Asegura que las variables a controlar alcancen el valor de sus consignas con precisión y velocidad
 - Obtiene medida de la variable a controlar
 - Calcula **error: consigna - variable a controlar**,
 - Modifica la **acción de control** para que el **error** tienda a cero

Ejemplo de sistema de control

Control del posición de una carretilla



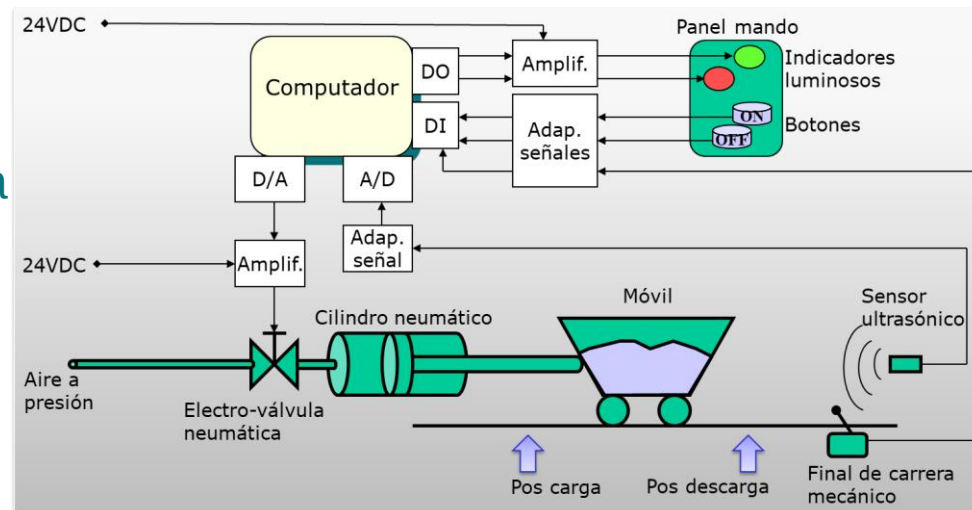
Niveles de control

□ Ejemplo: Control secuencial

- Cuando se pulsa el botón ON, la carretilla debe repetir el ciclo siguiente:

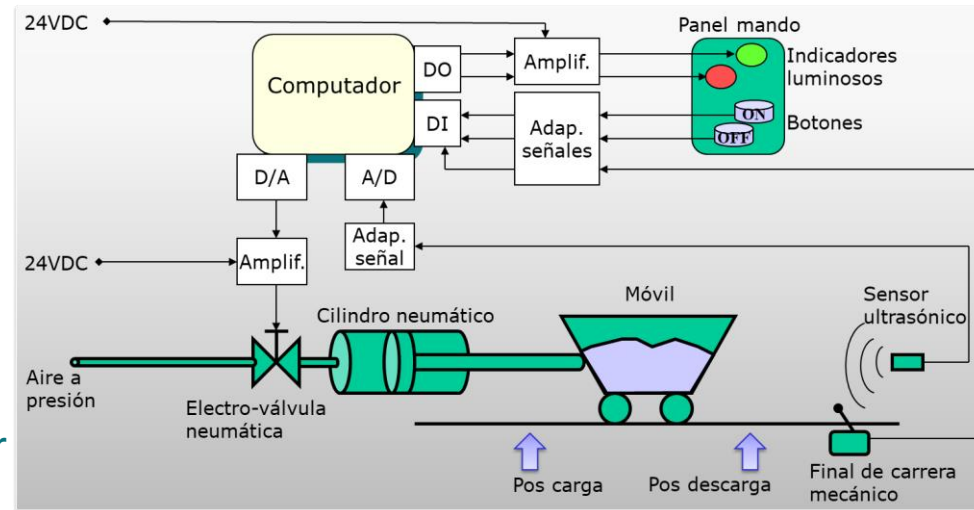
- Moverse a la posición de carga
- Permanecer 5 seg
- Moverse a la posición de descarga
- Permanecer 2 seg

- Si se alcanza el final de carrera mecánico, activar alarma y detener ciclo
- Si el operador pulsa el botón OFF, terminar último ciclo y no repetir



Niveles de control

- Ejemplo: Lazo de control
 - Cuando ordena el movimiento de la carretilla a una posición, se debe alcanzar la misma con la precisión y velocidad requeridas
 - Repetir hasta llegar al destino (consigna):
 - Leer posición actual del sensor ultrasónico
 - $\text{error} = \text{consigna} - \text{posición actual}$
 - Calcular aire a inyectar para reducir error a cero
 - Abrir electro-válvula para inyectar el aire deseado

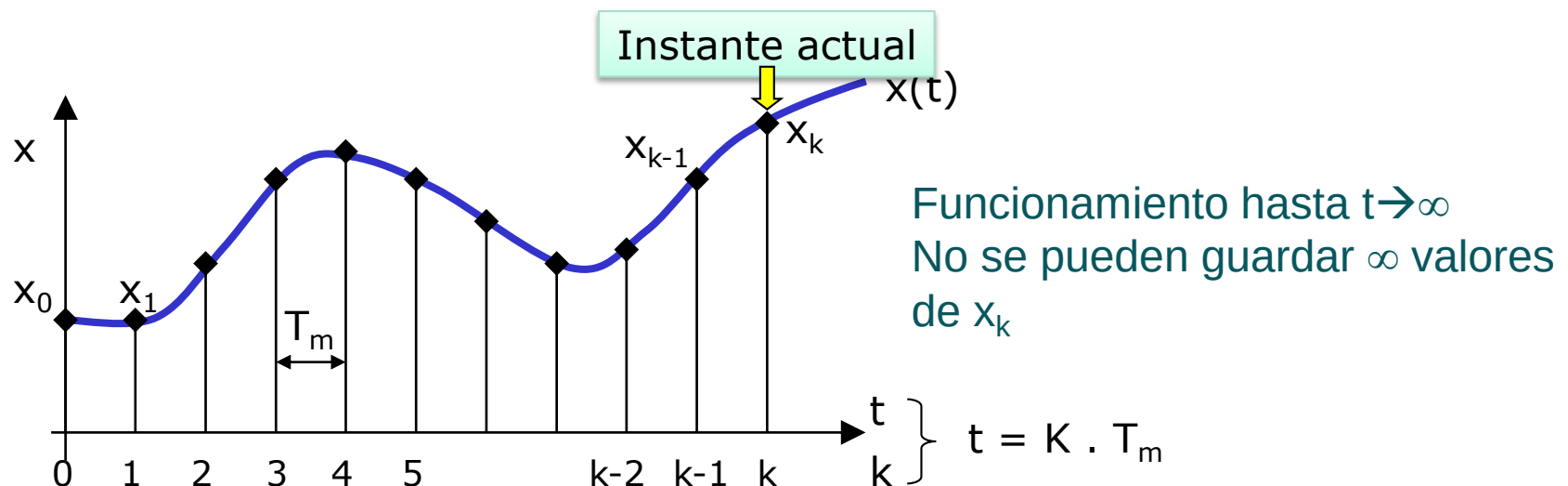


Indice

- ❑ Introducción al control de procesos por computador
- ❑ **Conceptos básicos para programación de control**
- ❑ Interfaz del computador con el exterior
- ❑ El control secuencial
- ❑ Las matemáticas del control
- ❑ Programación del lazo de control
- ❑ Programación en lenguaje C
- ❑ Implantación del control en el computador

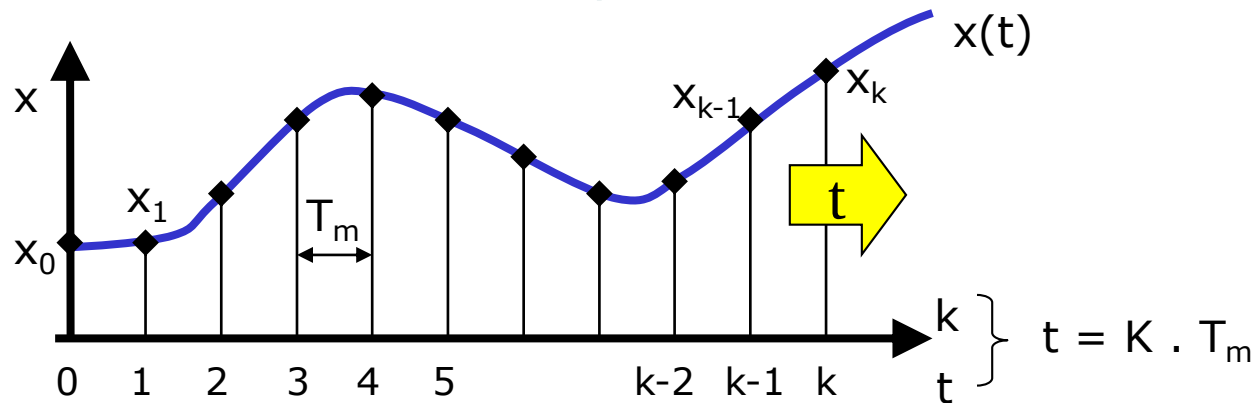
Variables continuas y discretas

- ❑ En el mundo real, las variables son continuas y analógicas.
- ❑ En el computador, las variables son discretas y digitales.
- ❑ Se representa una señal analógica como $x(t)$
- ❑ Se representa una secuencia de valores discretos como $\{x_k\}$
- ❑ En el computador sólo se dispone de valores discretos, obtenidos en instantes separados T_m (periodo de muestreo).

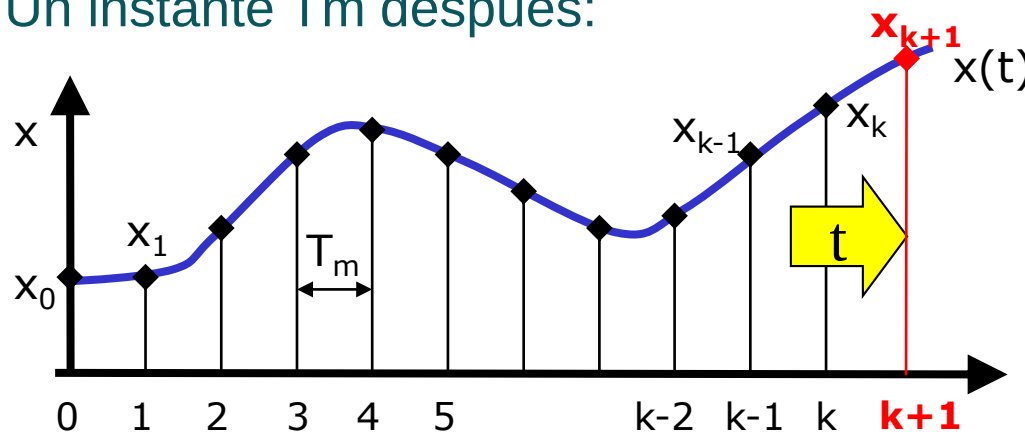


Variables temporales discretas

- En el mundo real, el tiempo avanza...



- Un instante T_m después:

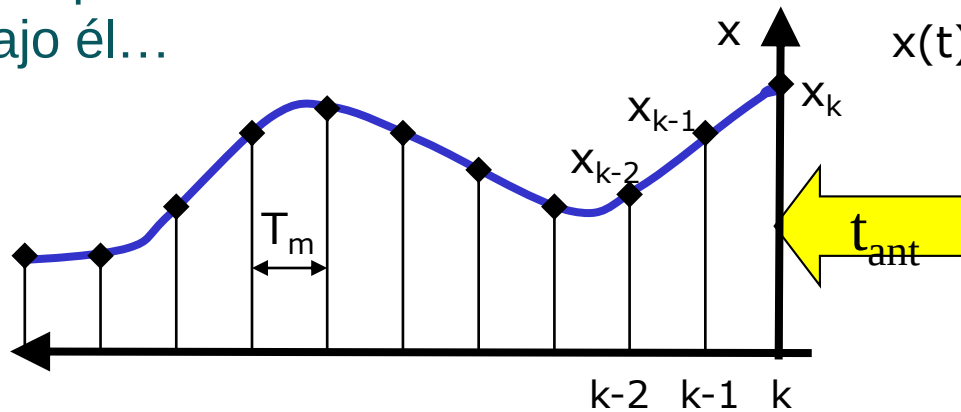


El nuevo valor de este instante es x_{k+1}

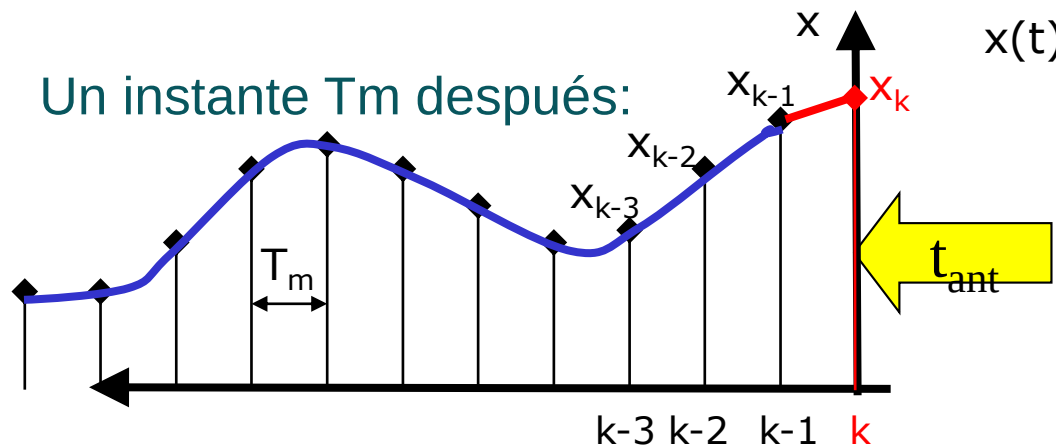
Los valores anteriores se mantienen

Variables temporales discretas

- Desde el punto de vista del programa de control, consideramos siempre el momento actual como el instante k , el tiempo pasa bajo él...



- Un instante T_m después:



El nuevo valor de este instante es ahora x_k

El anterior x_k ahora es x_{k-1} (se ha retrasado un periodo T_m respecto al instante actual)

El anterior x_{k-1} ahora es x_{k-2}
...

Programa básico de control

- Un programa básico de control tendrá el aspecto siguiente:

```
#include ...

int main()
{
    // Declaración de variables necesarias, al menos:
    float ref_k;    // Valor de la consigna actual (instante k)
    float y_k;      // Valor de la salida actual
    float u_k;      // Valor de la acción de control a aplicar

    // Inicializaciones necesarias
    ...

    // Bucle de control
    while (1)
    {
        ref_k=ValorDeLaReferencia(); // De operador, cálculo, medida, etc.
        y_k=ValorDeLaSalida(); // Medida mediante sensor
        u_k=CalculoAccionDeControl(y_k,ref_k); // Según algoritmo deseado
        AplicarAccionDeControl(u_k); // Enviar a accionador
        Sleep(Tm); // No ejecutar código hasta el siguiente Tm
    }
}
```

Programa básico de control

- Si el programa necesita manejar valores actuales y anteriores de una señal (ejemplo, acción de control depende de crecimiento de y_k : necesita y_k e y_{k-1})

```
#include ...

int main()
{
    float ref_k;
    float y_k, y_k_menos_1;    // Valor de la salida actual y la anterior
    float u_k;

    // Inicializaciones necesarias
    . . .
    // Bucle de control
    while (1)
    {
        y_k_menos_1=yk;  // La última  $y_k$  (del instante anterior) es ahora  $y_{k-1}$ 
        ref_k=ValorDeLaReferencia();
        y_k=ValorDeLaSalida();
        u_k=CalculoAccionDeControl(y_k, y_k_menos_1, ref_k);
        AplicarAccionDeControl(u_k);
        Sleep(Tm);
    }
}
```

Programa básico de control

- Generalizando: el programa necesita el valor actual y 'm' anteriores de la señal v ($v_k, v_{k-1}, \dots, v_{k-m}$)

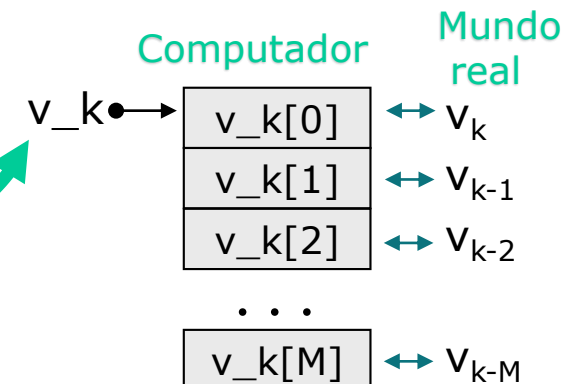
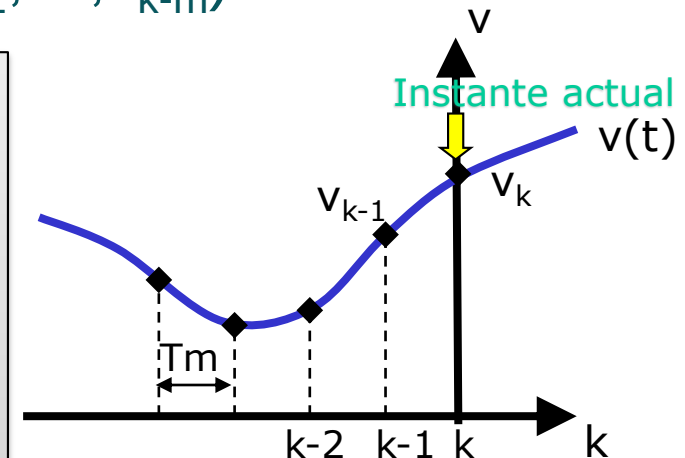
```
#include ...
#define M 6

int main()
{
    float v_k[M+1]; // Desde 0 a m incluido

    // Inicializaciones
    ...
    // Bucle de control
    while (1)
    {
        v_k[M]=v_k[M-1];
        v_k[M-1]=v_k[M-2];
        ...
        v_k[2]=v_k[1];
        v_k[1]=v_k[0];
        v_k[0]=nuevo_valor();
        // Usar tabla v_k con la interpretación

        Sleep(Tm);
    }
}
```

*Mejor función:
Desplaza($v_k, M+1$);*



Programa básico de control

- ❑ Un ejemplo de la vida cotidiana: regar una planta.
Cada día comprobamos si tenemos que regar nuestra planta, utilizando la información de lo que ha llovido en la semana previa:
 - Periodo de muestreo: 1 día
 - Número de datos a "recordar": 1 semana
 - Cada día:
 - Toda la información acumulada es 1 día más vieja
 - Se añade la información del día actual (lluvia medida)
 - Se toma una decision sobre el riego del día actual con la información disponible (día actual y 6 anteriores)
 - Se espera 1 día para repetir la operación
- ❑ Más detalles en

http://isa.uniovi.es/~ialvarez/Curso/descargas/series_temporales_regando.pdf

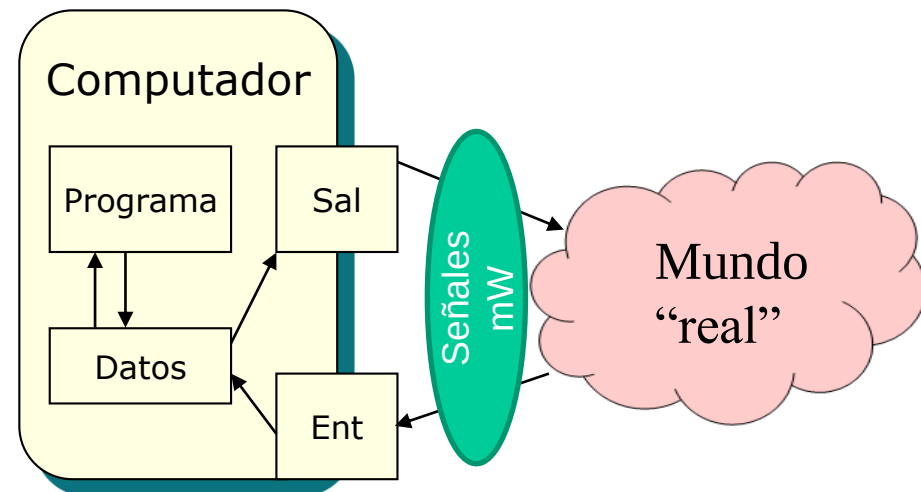


Indice

- ❑ Introducción al control de procesos por computador
- ❑ Conceptos básicos para programación de control
- ❑ **Interfaz del computador con el exterior**
- ❑ El control secuencial
- ❑ Las matemáticas del control
- ❑ Programación del lazo de control
- ❑ Programación en lenguaje C
- ❑ Implantación del control en el computador

Interfaz del computador con el exterior

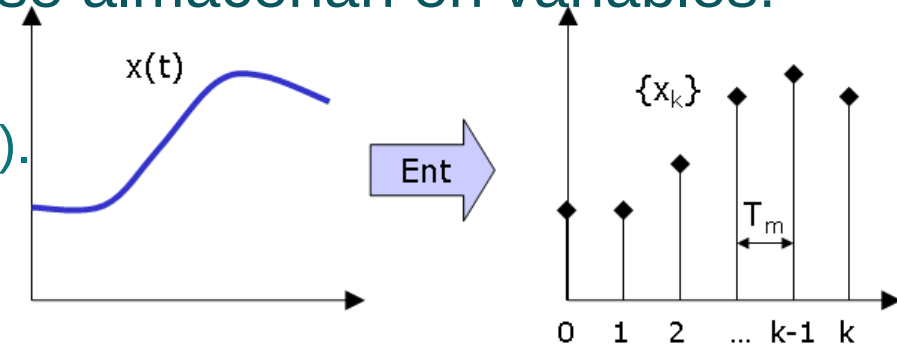
- ❑ El programa del computador maneja datos que son variables alojadas en su memoria
- ❑ Los datos manejados por el programa de control proceden o tienen como destino el mundo “real”
- ❑ Los computadores disponen de subsistemas electrónicos llamados de Entrada y Salida (E/S ó I/O) para hacer el intercambio de información entre mundo exterior y variables internas.
- ❑ El interfaz se realiza mediante pequeñas tensiones y corrientes (potencia de mW)



Interfaz con el “mundo real”

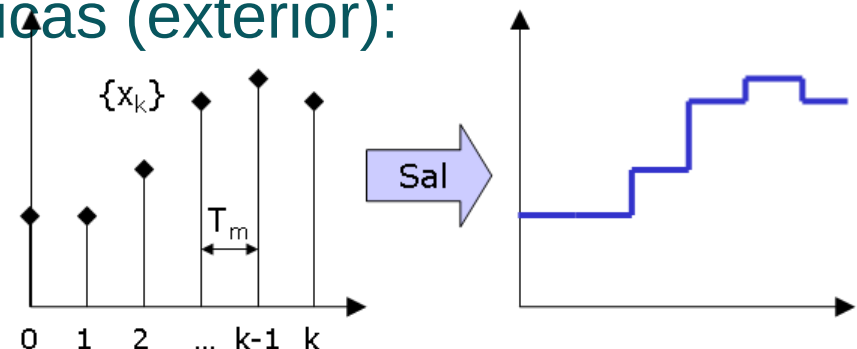
- ❑ **Entradas del computador:** convierten valores continuos y analógicos (exterior) en valores discretos y digitales (computador) que se almacenan en variables:

- Conversores A/D
- Entradas digitales (todo/nada).
- Contadores de pulsos
- Entradas numéricas



- ❑ **Salidas del computador:** convierten valores discretos y digitales almacenados en variables (computador) en señales continuas y analógicas (exterior):

- Conversores D/A
- Salidas digitales (todo/nada)
- Generadores PWM
- Salidas numéricas



Interfaz con el “mundo real”

- ❑ Entradas del computador: diversos dispositivos electrónicos en función de la naturaleza del sensor y la información a extraer de él:
 - Conversor A/D (ADC) :
 - Para sensores que generan una tensión analógica
 - Proporcionan al programa una variable de tipo entero proporcional a la tensión a su entrada
 - Entrada digital (DI):
 - Para sensores que generan una tensión con únicamente 2 estados (0/1)
 - Proporcionan al programa un bit de una variable de tipo entero
 - Contador de pulsos (counter):
 - Para sensores que generan una tensión con únicamente 2 estados (0/1), de la que no interesa el valor instantáneo sino el número de transiciones 0→1 y/o 1→0
 - Proporcionan al programa una variable de tipo entero con el valor de la cuenta
 - Entrada numérica:
 - Sensores con electrónica adicional que entregan valores numéricos

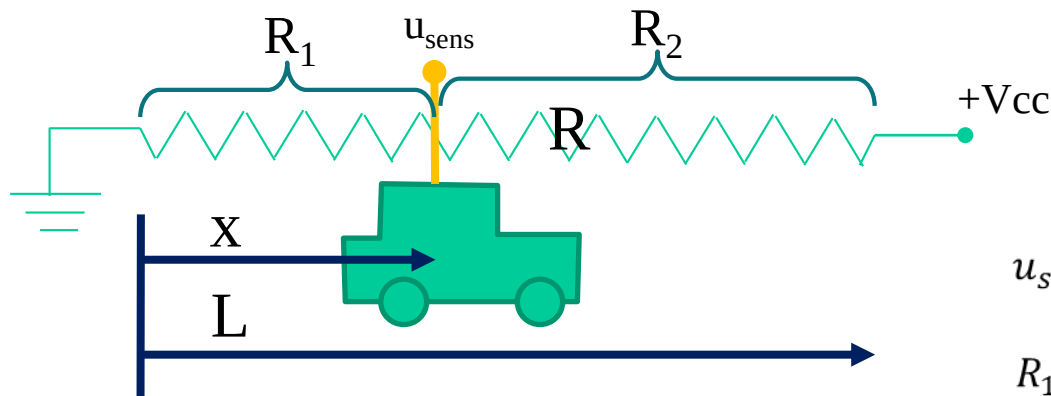
Obtención de valores analógicos

□ Obtención de valores analógicos: conversores A/D

- Sensor: genera una pequeña tensión o corriente directamente relacionada con una magnitud física



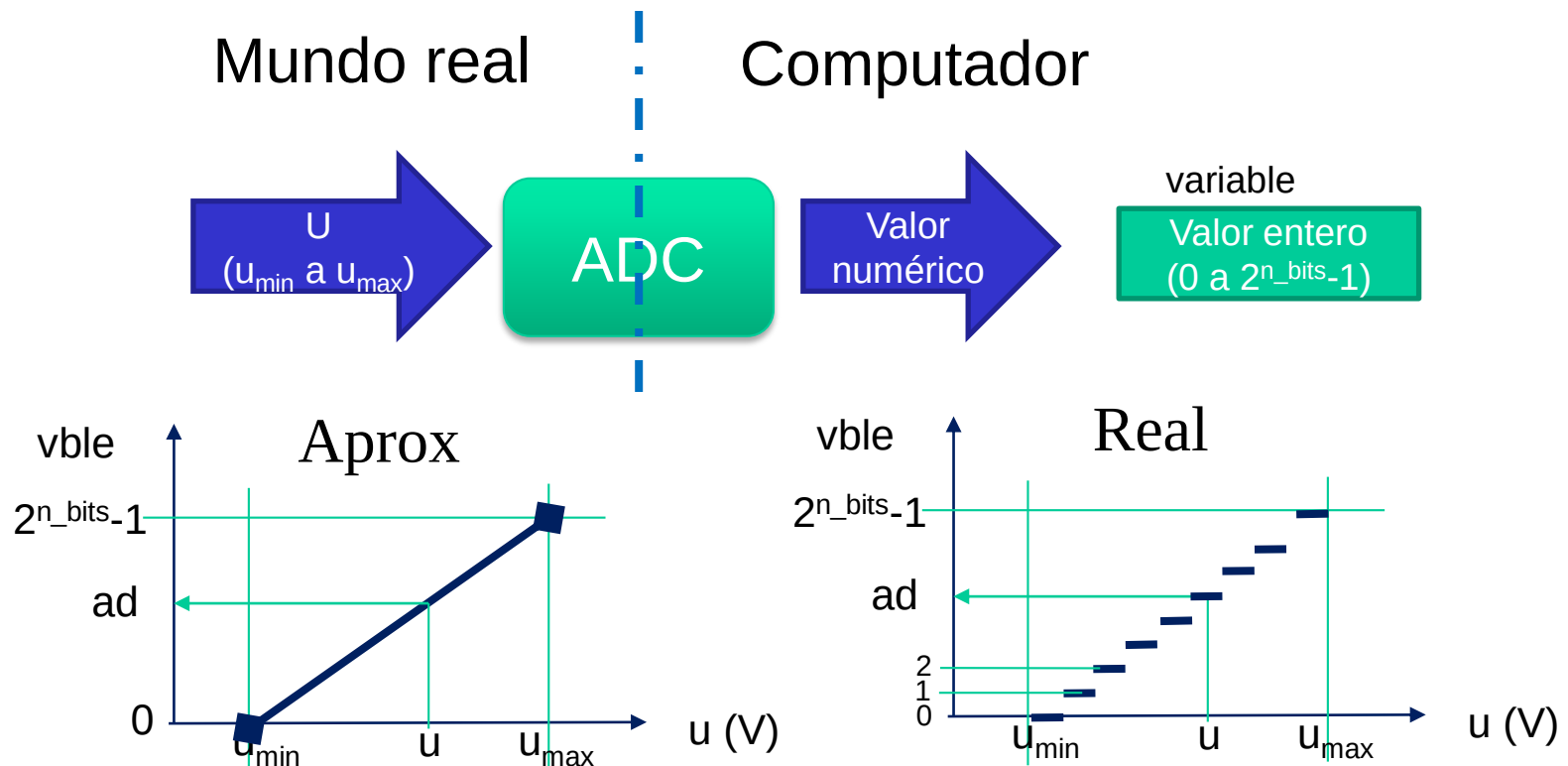
- Ejemplo: potenciómetro para medida de desplazamiento



$$\left. \begin{aligned} u_{sens} &= v_{cc} \frac{R_1}{R} \\ R_1 &= R \frac{x}{L} \end{aligned} \right\} \rightarrow u_{sens} = v_{cc} \frac{x}{L}$$

Obtención de valores analógicos

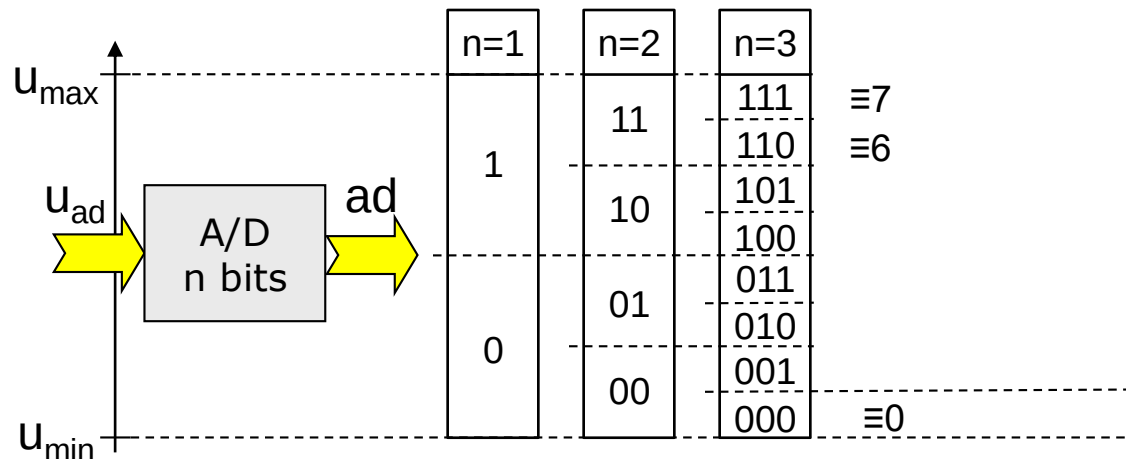
- ❑ Obtención de valores analógicos: conversor A/D
 - Conversor A/D: genera un valor numérico (variable de tipo **entero**) relacionada con la tensión a su entrada
 - Características: $u_{\min}$, $u_{\max}$, n_bits



Obtención de valores analógicos

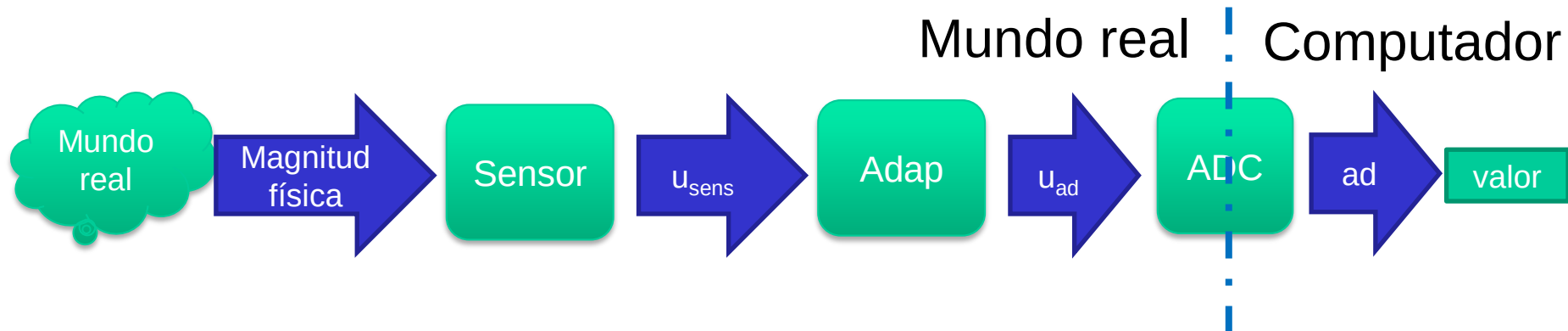
❑ Conversor A/D

- Funcionamiento del conversor A/D de n bits:
 - Divide un rango de tensión de entrada en 2^n intervalos, asociando a cada uno de ellos un valor numérico ($0 \dots 2^n - 1$).
 - Se realiza con electrónica mediante comparaciones sucesivas
 - El resultado de la conversión es un valor entero utilizable por el programa.
 - Valores típicos de n: 8, 10, 12, 16, 20, 24, 32.
 - Tipos: flash, aproximaciones sucesivas, etc
- (<http://www.onmyphd.com/?p=analog.digital.converter>)



Obtención de valores analógicos

- ❑ Si el rango de tensiones generado por el sensor no concuerda con el rango de tensiones del conversor:
 - Se precisa una electrónica de adaptación intermedia (Amplificadores Operacionales)

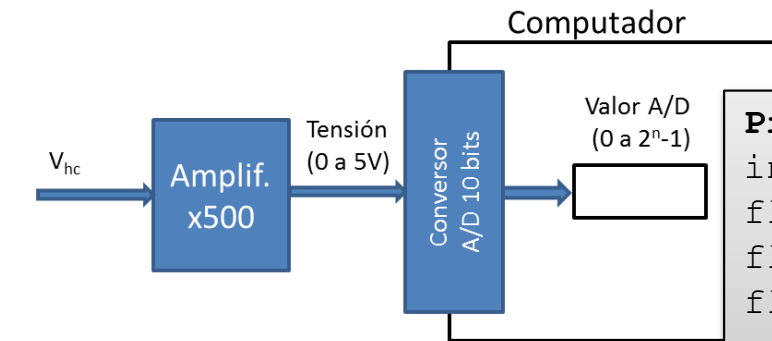
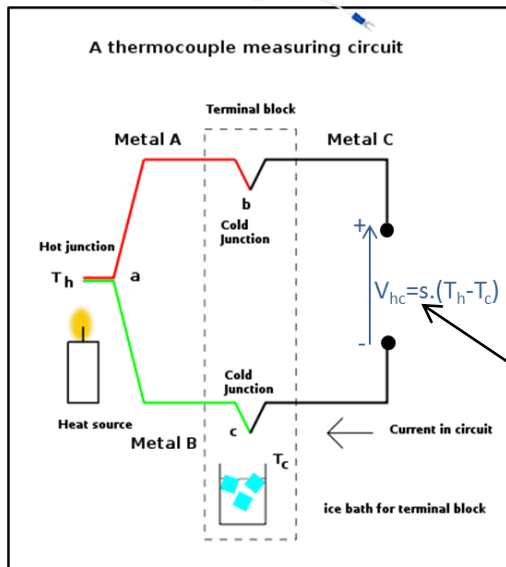


- ❑ El programa del computador sólo puede leer el valor del conversor, que está relacionado (**pero no es**) con la magnitud medida
 - El programa debe calcular la magnitud a partir de las especificaciones de la cadena de medida (sensor, adap, ADC)

Obtención de valores analógicos

- Ejemplo: adquisición de T^a medida por termopar con conversor A/D de 10 bits y límites de tensión 0-5V

Termopar:



$s = 43 \mu\text{V}/^\circ\text{C}$ (termopar tipo T)

¡¡ Usar #define para todas las constantes !!

Programa:

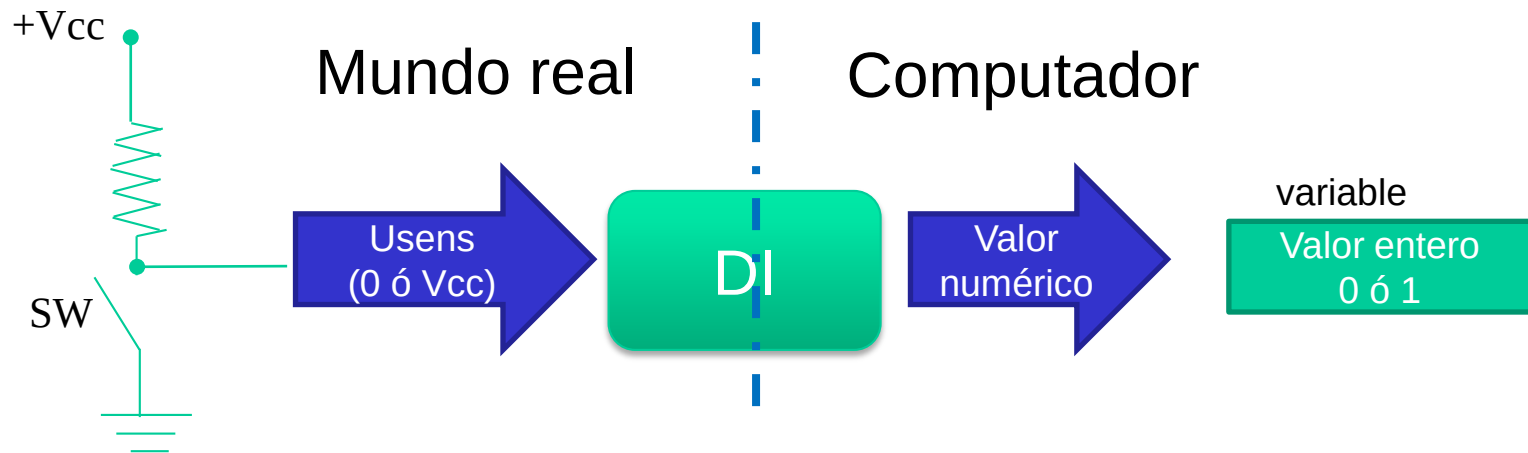
```
int adk;
float Vad_k, Vhc_k, Th_k;
float s=43e-6f; // V/°C
float Tc=0;

adk=LeerAD();
Vad_k=adk*5.0f/1023;
Vhc_k=Vad_k/500.0f;
Th_k=Vhc_k/s-Tc;
```

Mejor en una función: float LeerTempDeg();

Entradas digitales

- ❑ Algunos sensores generan información en forma de todo/nada (true/false , 1/0), que serán capturadas mediante entradas digitales (DI) del computador
 - Ejemplo: interruptor (switch)

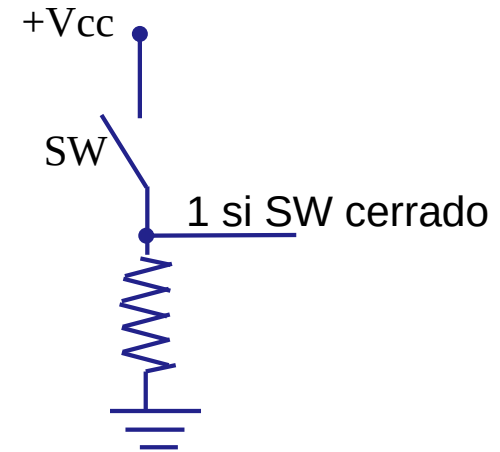
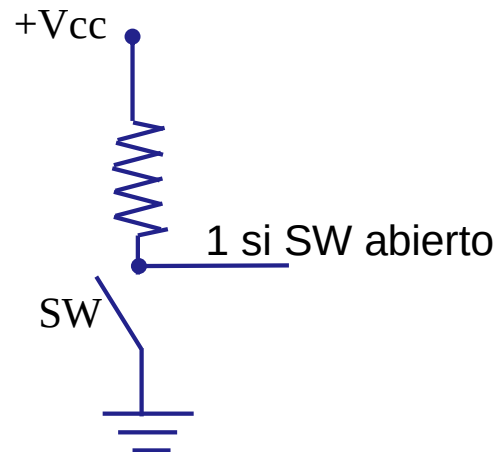


- ❑ El hardware electrónico es de la máxima sencillez
- ❑ En algunos equipos la conexión a +Vcc y la resistencia están ya integrados en la entrada digital (input pull-up)

Entradas digitales

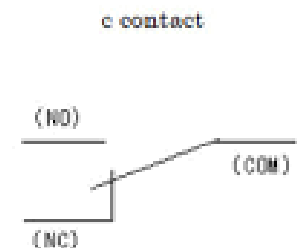
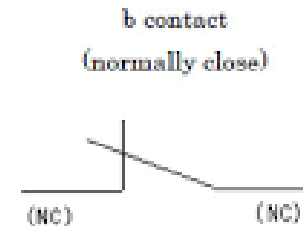
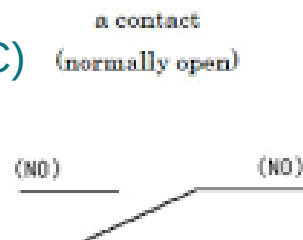
- La interpretación de 0/1 en una DI por parte del programa depende de:

- El conexionado:



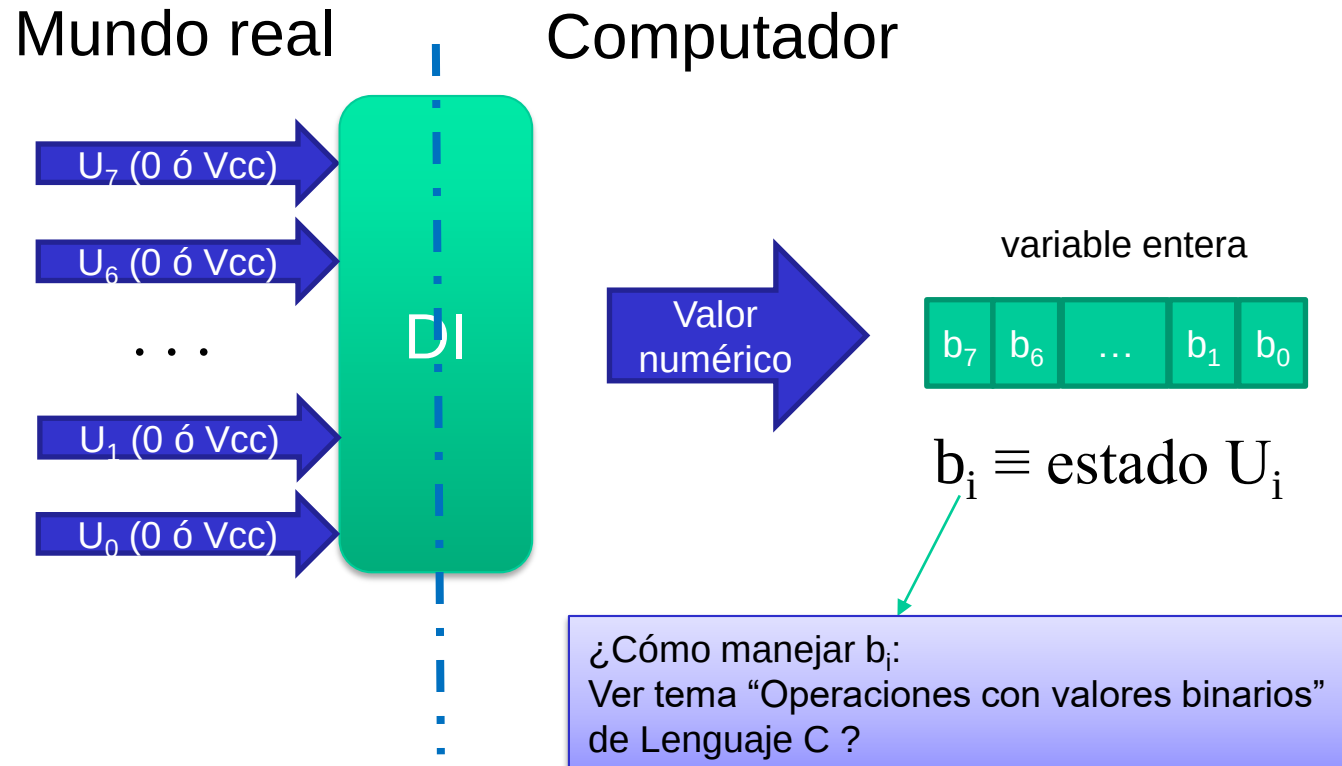
- Y del tipo de interruptor:

- Normally Open (NO)
- Normally Closed (NC)



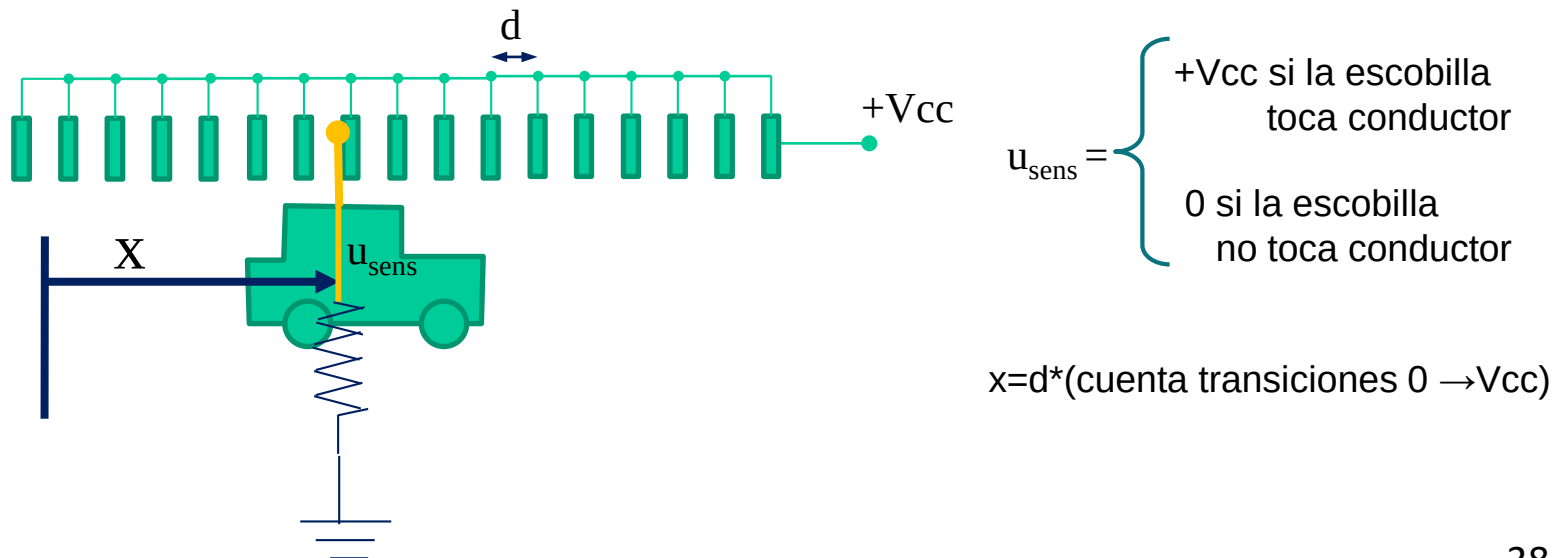
Entradas digitales

- Es habitual que varias entradas digitales se agrupen en bits diferentes de una sola variable
 - Ejemplo: interruptor (switch)



Entradas tipo contador

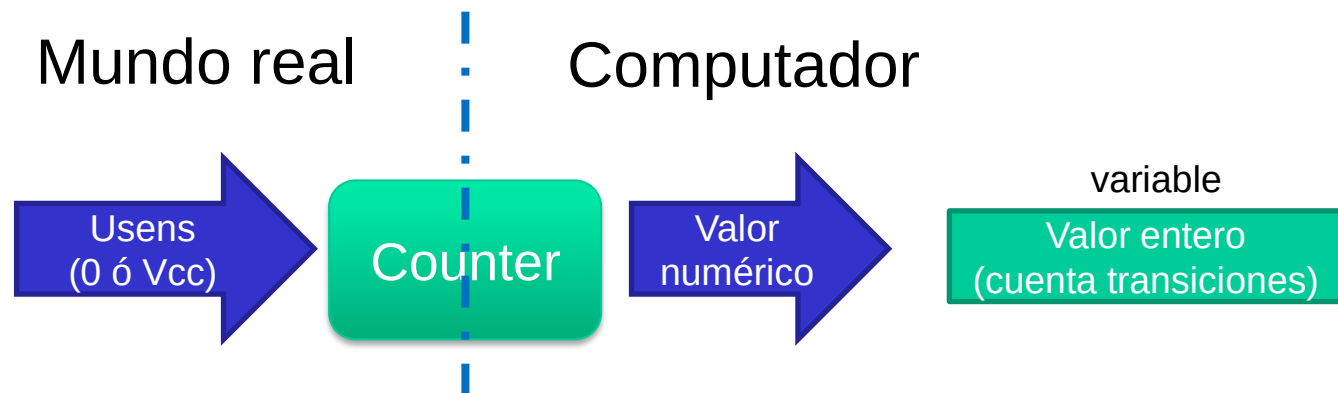
- ❑ Algunos sensores generan información en forma de todo/nada (true/false , 1/0), de la que no interesa el estado (1/0) sino la cuenta de transiciones ($1 \rightarrow 0$ y/o $0 \rightarrow 1$)
- ❑ Estos sensores serán capturados mediante entradas tipo contador
 - Ejemplo: encoder para medida de desplazamiento



Entradas tipo contador

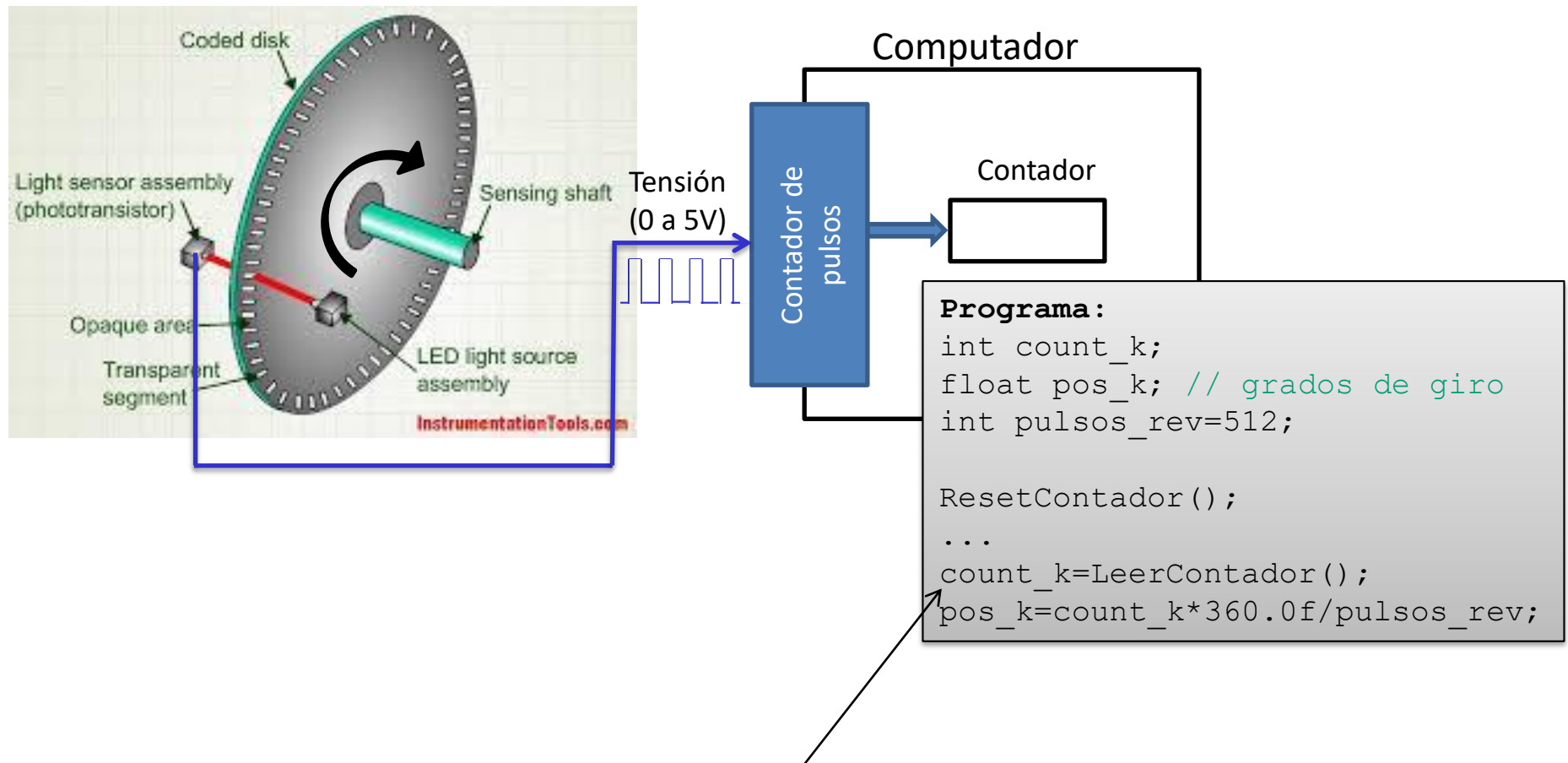
□ Entrada contador:

- Incrementa un contador (entero de n bits) cada vez que se produce un pulso (de subida, de bajada, ambos) en la señal de entrada (todo/nada).
- El valor del contador es un valor entero utilizable por el programa.
- Se puede reiniciar el contador a 0 por programa.
- Se necesita una entrada digital auxiliar para definir el 0 absoluto.
- ¡¡ Ojo con desbordamientos del contador !!



Entradas tipo contador

- Ejemplo: entrada de pulsos de un encóder óptico:



Mejor en una función: float LeerPosDeg();

Entradas tipo contador

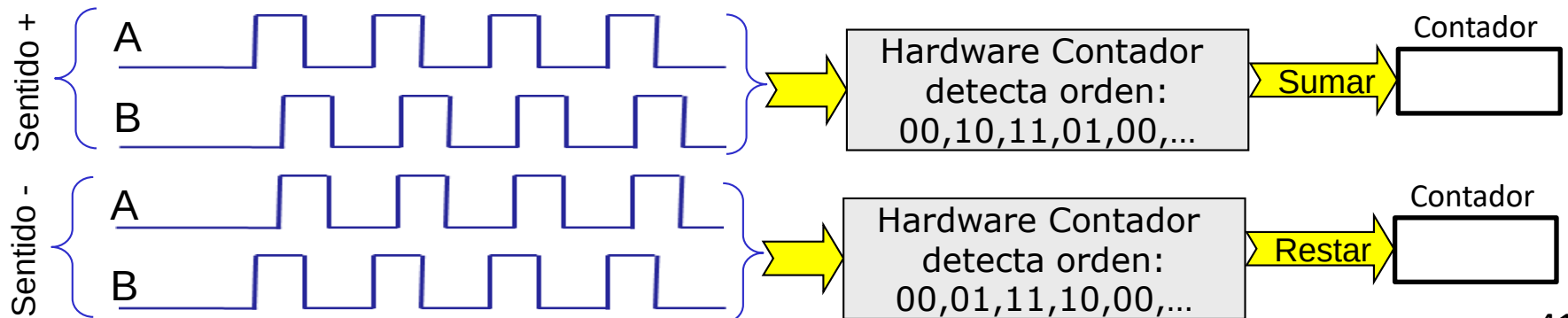
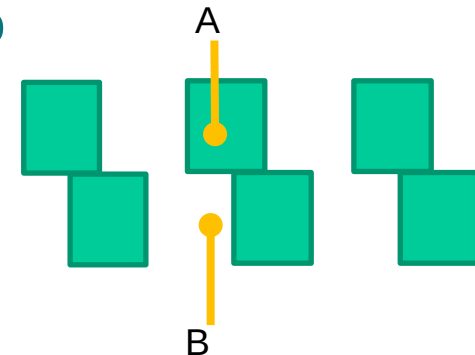
□ Entrada de pulsos: algunas consideraciones

■ Necesidad de referencia “home” para medidas absolutas

- El sensor sólo mide variación respecto a la posición original. Se necesita una referencia absoluta (mediante entrada digital), y mover posición a esa referencia al comienzo

■ Señal en cuadratura para detección de sentido

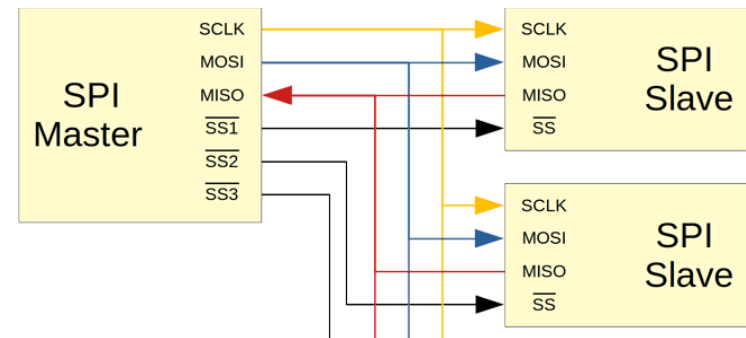
- Con una sola señal no se puede distinguir el sentido del movimiento
- Con dos señales en cuadratura se puede distinguir (habitualmente se integra en el hardware del contador)



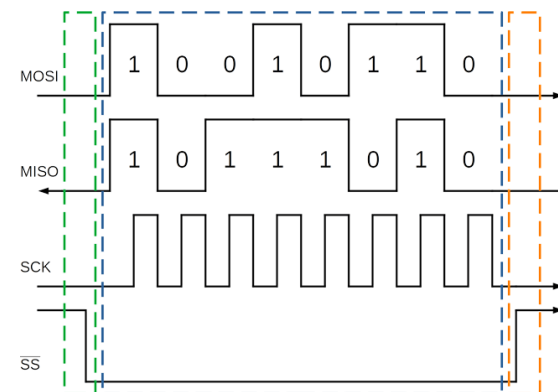
Entradas numéricas

- ❑ Sensores con electrónica adicional que sirven datos directamente utilizables por el computador, a través de buses y protocolos de comunicación:

- SPI
- I2C
- CAN
- RS-232,485
- USB
- ...



- ❑ El programa debe:
 - Configurar el dispositivo
 - Recibir los datos del dispositivo
 - Interpretar los datos



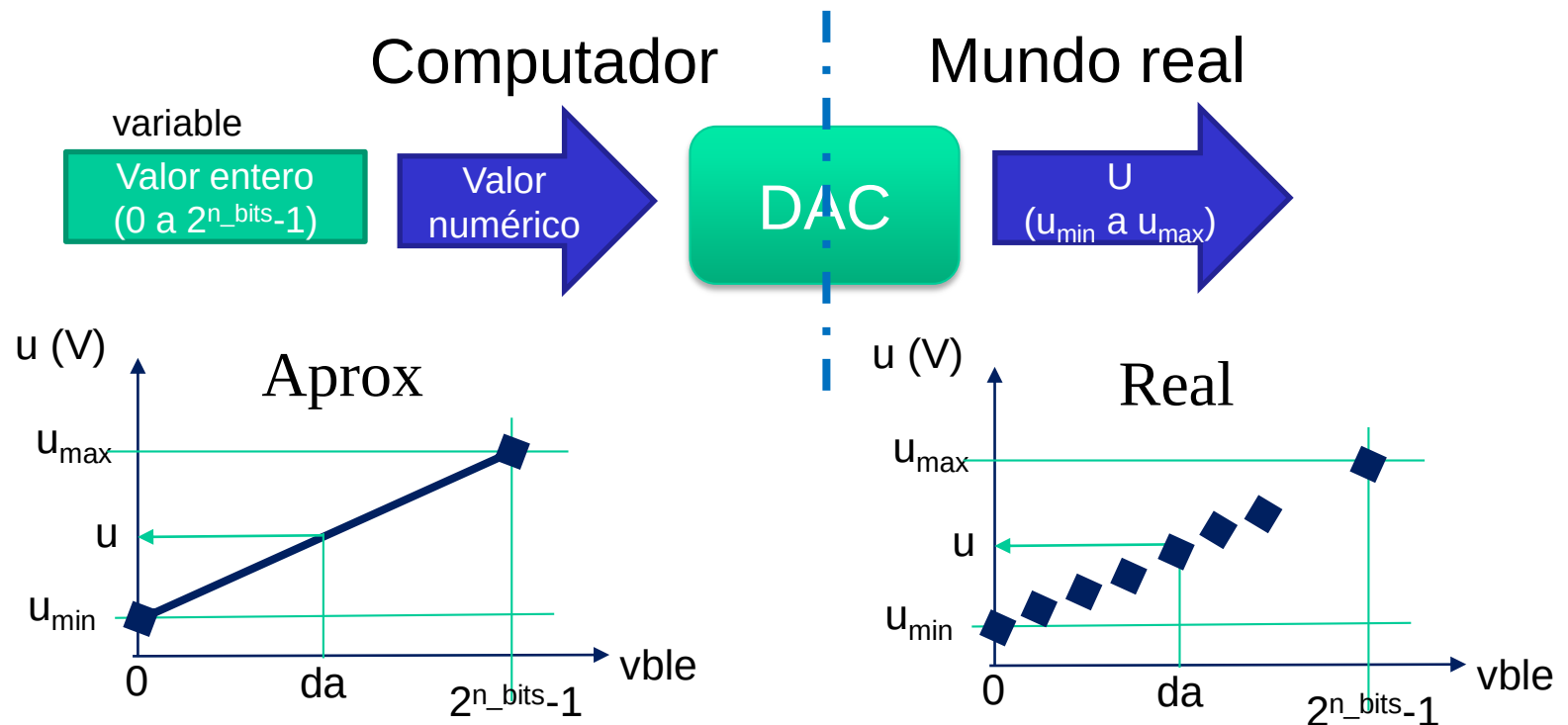
— Selección de esclavo
— Transferencia y recepción
— Desactivación de esclavo

Interfaz con el “mundo real”

- ❑ Salidas del computador: diversos dispositivos electrónicos en función de la naturaleza de la acción a generar:
 - Conversor D/A (DAC) :
 - ✦ Generan una tensión analógica proporcional a un valor entero calculado por el programa
 - Salida digital (DO):
 - ✦ Generan una tensión con únicamente 2 estados ($0/1 \equiv u_{\min}/u_{\max}$) a partir de un bit de una variable de tipo entero proporcionado por el programa
 - Salida modulada en ancho de pulso (PWM):
 - ✦ Generan una tensión en forma de tren de pulsos 0/1 cuyo valor promedio es proporcional a un valor entero calculado por el programa
 - Salida numérica:
 - ✦ Accionadores con electrónica adicional que reciben valores numéricos
- ❑ Para todos ellos, la salida es de poca potencia (mW):
 - Es necesario un accionador que obtenga la energía de una fuente externa al computador
 - El accionador es modulado por la salida del computador

Generación de valores analógicos

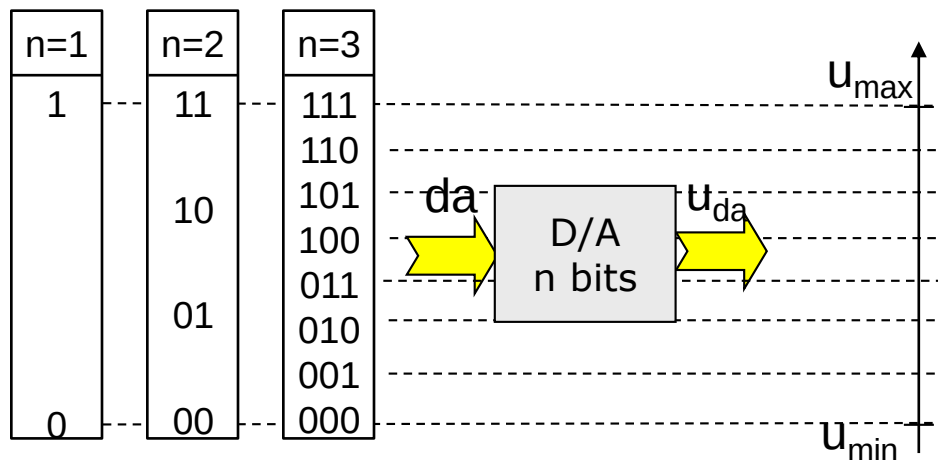
- Generación de valores analógicos: conversor D/A (DAC)
 - Conversor D/A: genera una tensión a su salida relacionada con el valor numérico (variable de tipo **entero**) que se aplica a su entrada
 - Características: $u_{\min}$, $u_{\max}$, n_bits



Generación de valores analógicos

❑ Conversor D/A

- Funcionamiento del conversor D/A de n bits:
 - Genera una tensión de salida en función de un valor numérico ($0 \dots 2^n - 1$) generado por el programa.
 - Se realiza con electrónica mediante adiciones sucesivas
- Valores típicos de n : 8, 10, 12, 16, 20, 24, 32.
- Tipos: oversampling, ladder, ...
(<http://www.onmyphd.com/?p=digital.analog.converter>)

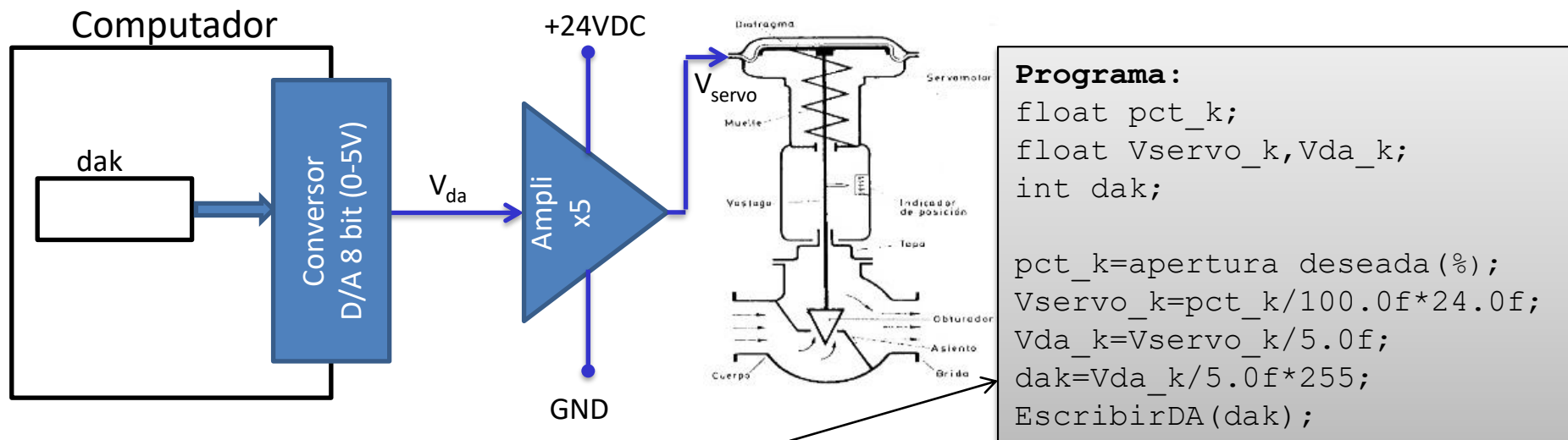


Generación de valores analógicos

❑ Conversor D/A

- La salida es una pequeña tensión con baja potencia (mW) que puede servir para modular energía externa de un accionador analógico

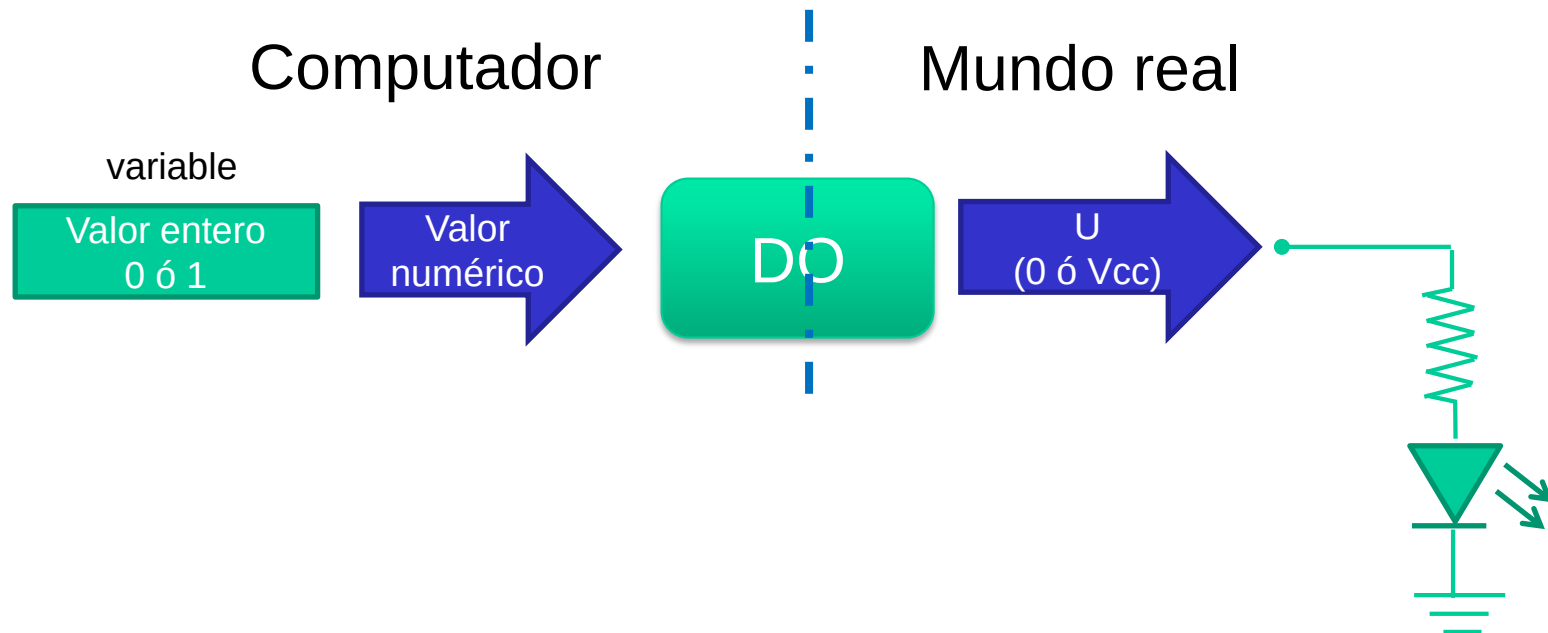
❑ Ejemplo: accionamiento de una electroválvula proporcional de 24VDC, mediante conversor DA de 8 bits y límites de tensión 0-5V



Mejor en una función: void ActivarValvulaPct(float pct);

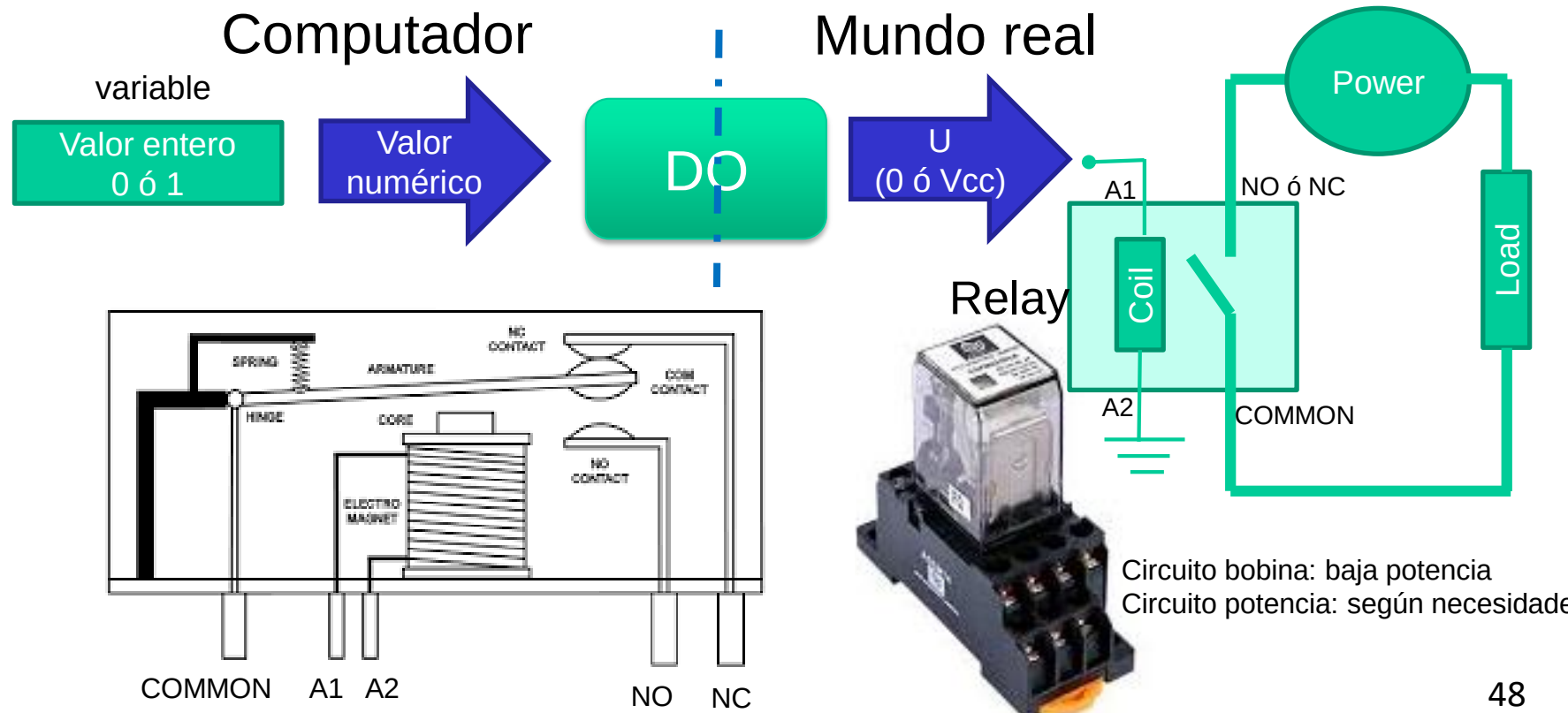
Salidas digitales

- ❑ Algunos accionadores sólo tienen dos acciones posibles: todo/nada (true/false , 1/0)
- ❑ Para ellos se utilizan salidas digitales (DO)
 - Ejemplo: lámpara LED (mW)



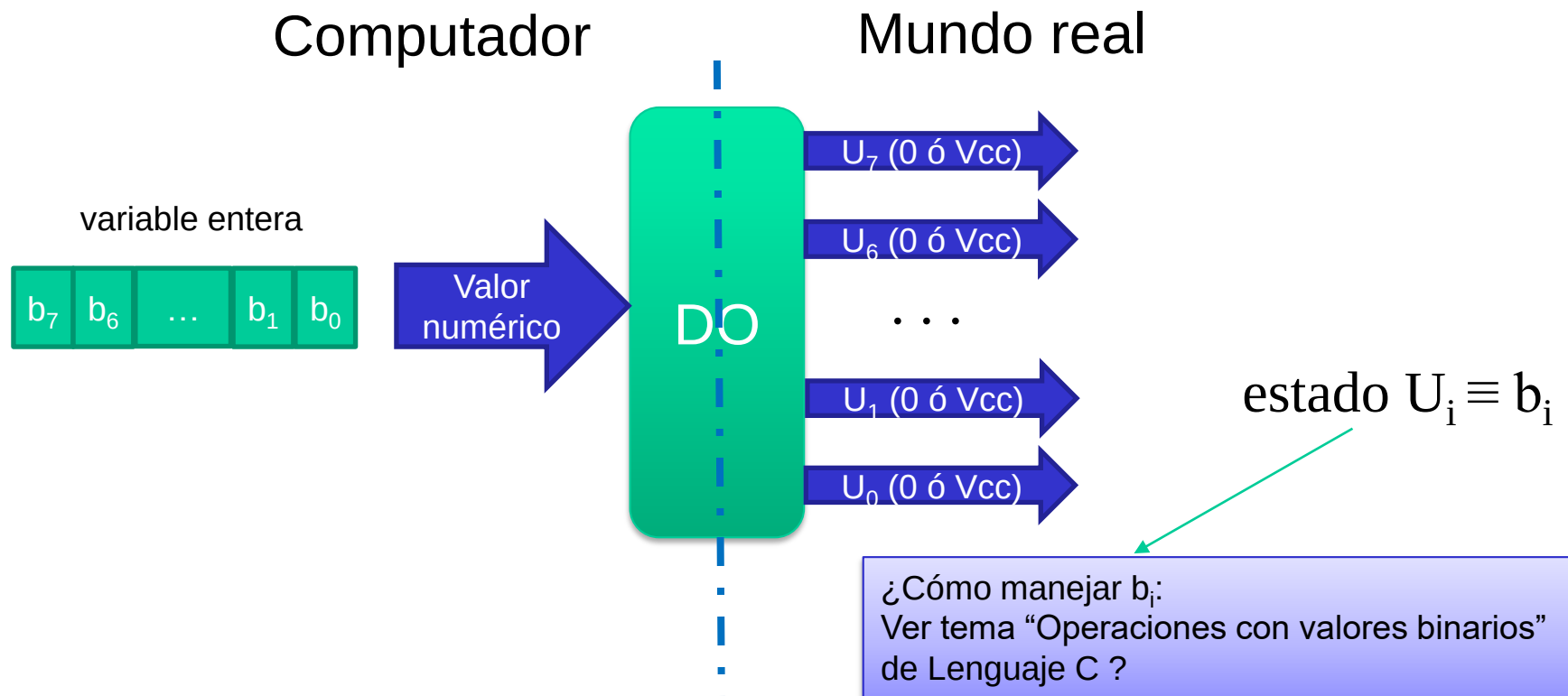
Salidas digitales

- ❑ Si es necesaria mayor potencia, lo usual es utilizar un relé (relay):
 - Electromagnético (bobina+interruptor)
 - De estado sólido (amplificador electrónico)



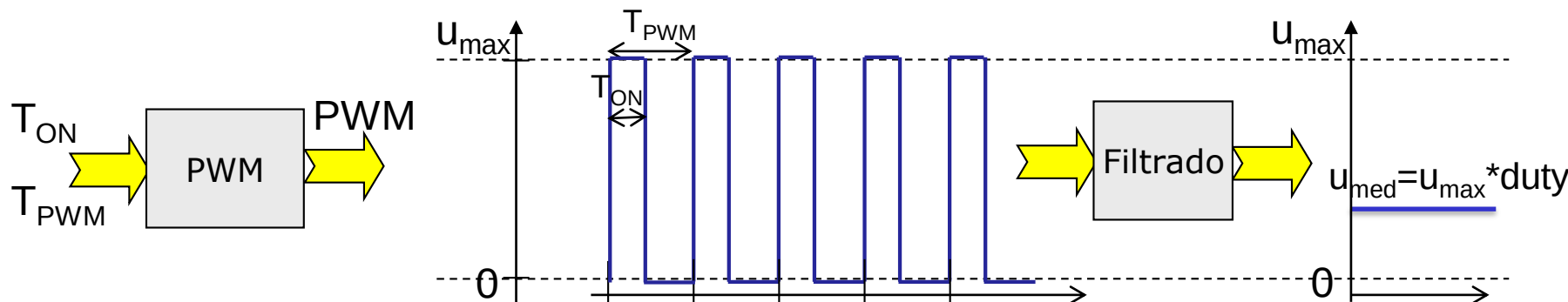
Salidas digitales

- Es habitual que varias salidas digitales se agrupen en bits diferentes de una sola variable



Salidas PWM

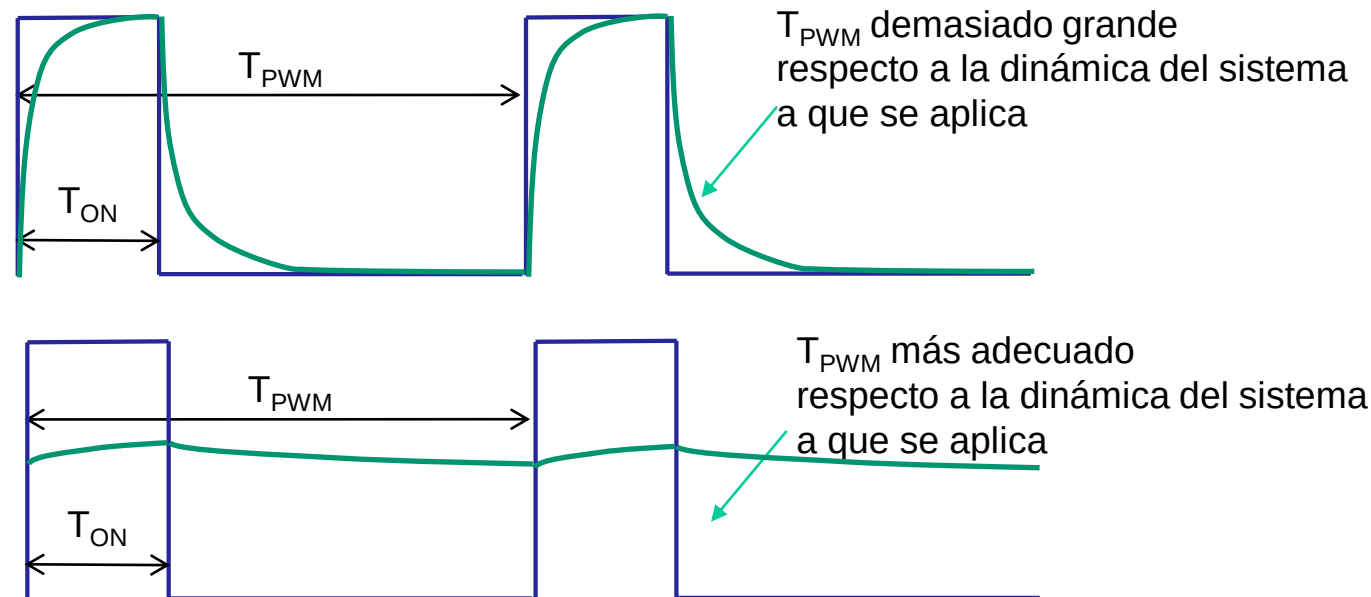
- ❑ PWM: Pulse Width Modulated
- ❑ Funcionamiento del generador PWM:
 - Genera una onda tensión de salida alternativamente ON/OFF, siendo programable el tiempo de onda (T_{PWM}) y la duración de encendido (T_{ON}).
 - La relación T_{ON} / T_{PWM} se conoce como ciclo duty.
 - El resultado, una vez filtrado, es un valor analógico constante proporcional al valor del ciclo duty.
 - En muchas ocasiones actúa como filtro 'natural' la propia dinámica del sistema sobre el que se aplica la onda
 - T_{PWM} suele ser fijo y función del filtro/accionamiento/sistema.



Salidas PWM

□ PWM: Pulse Width Modulated

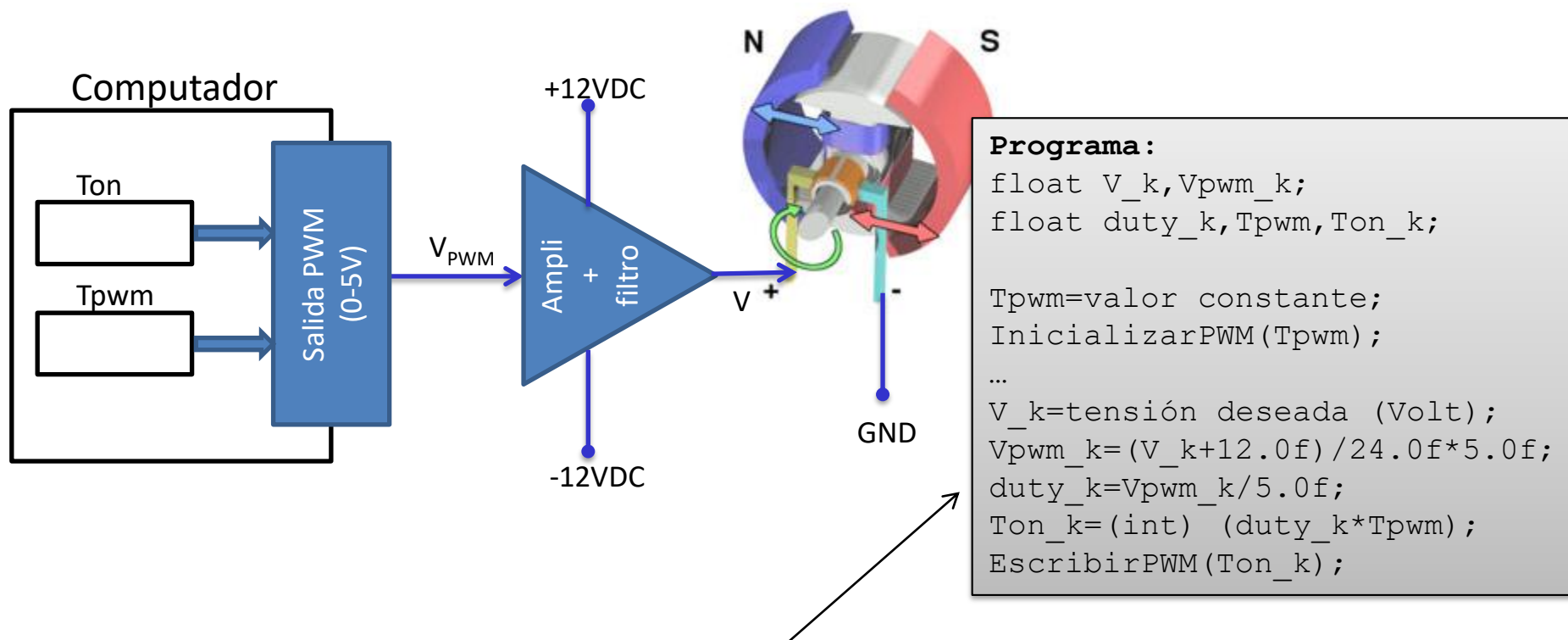
- Dos parámetros para controlar la onda: T_{pwm} y T_{on}
- T_{pwm} = valor **fijo** dependiente del sistema



- T_{on} = valor **de control** que se modifica para cambiar la acción
- $duty = 0 \text{ a } 1 \text{ (0 a 100\%)} = T_{on}/T_{pwm}$

Salidas PWM

- Ejemplo: accionamiento de un motor DC de 12V mediante un generador PWM de 5V y un amplificador.

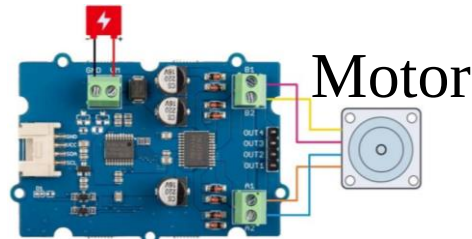


Mejor en una función: void ActivarMotor(float Vmotor);

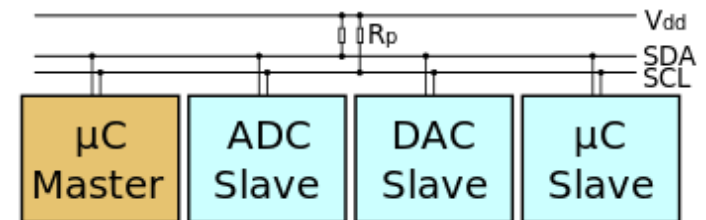
Salidas numéricas

- ❑ Accionadores con electrónica adicional que reciben datos del computador, a través de buses y protocolos de comunicación:

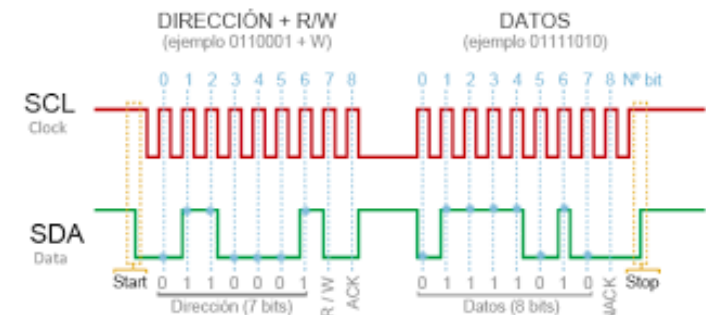
- SPI
- I2C
- CAN
- RS-232,485
- USB
- ...



Motor



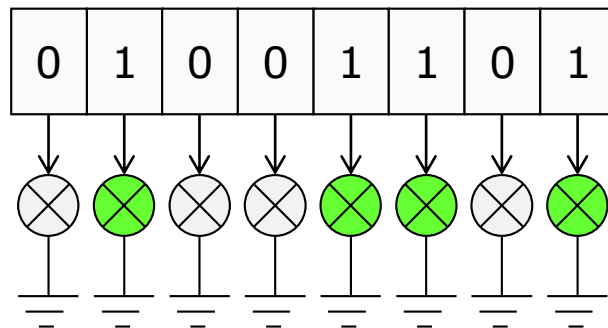
- ❑ El programa debe:
 - Configurar el dispositivo
 - Enviar comandos al dispositivo



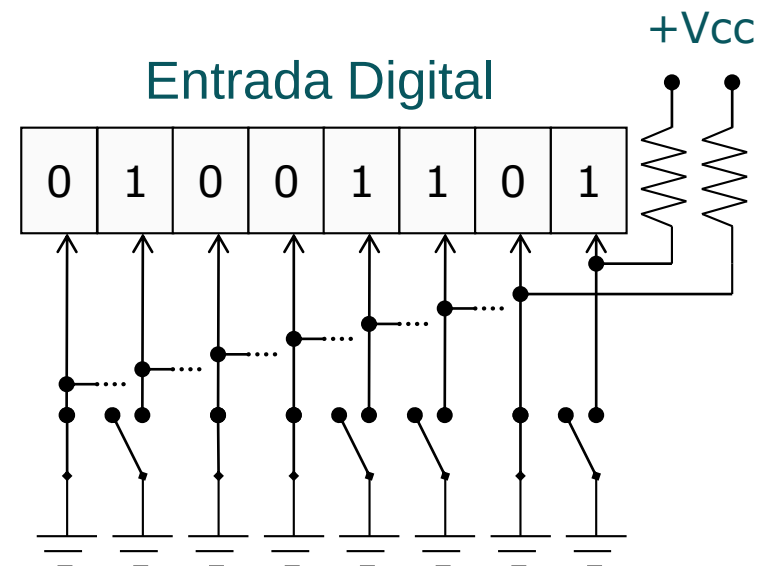
Programación de E/S digital

- ❑ El interfaz con muchos dispositivos se realiza en formato todo/nada (1/0).
- ❑ Con estos dispositivos, se utilizan E/S digitales.
- ❑ Las E/S digitales están formadas por valores enteros, de los cuales cada bit está conectado a un dispositivo externo.

Salida Digital



Entrada Digital



Operaciones con variables

- ❑ Operaciones con el bit de peso P de la variable entera X:
 - En la mayoría de ocasiones, se debe operar con un solo bit (E ó S) sin alterar el resto.

```
int x,p;  
int mascara;  
...Dar valores a p y x...  
mascara=1<<p;
```

- ¿Está activo el bit de peso P de la variable X?

```
if (x & mascara)  
...  
...
```

- Poner a 1 el bit de peso P de la variable X:

```
x=x | mascara;
```

- Poner a 0 el bit de peso P de la variable X:

```
x=x & ~mascara;
```

- Cambiar el valor del bit de peso P de la variable X:

```
x=x ^ mascara;
```

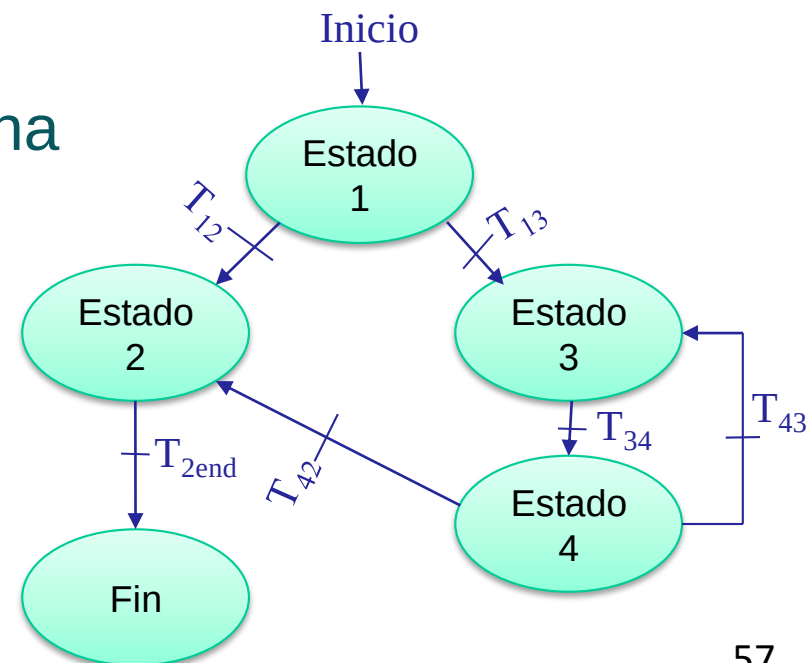


Indice

- ❑ Introducción control de procesos por computador
- ❑ Conceptos básicos para programación de control
- ❑ Interfaz del computador con el exterior
- ❑ **El control secuencial**
- ❑ Las matemáticas del control
- ❑ Programación del lazo de control
- ❑ Programación en lenguaje C
- ❑ Implantación del control en el computador

Control secuencial

- ❑ El control secuencial "fija" las consignas para los lazos de control, de acuerdo con:
 - El estado actual (y anteriores)
 - Los datos disponibles (actual y anteriores) de:
 - Sensores
 - Variables internas
 - Informaciones del usuario u otros computadores
 - Tiempo transcurrido
 - etc.
- ❑ Se suele implementar como una máquina de estados

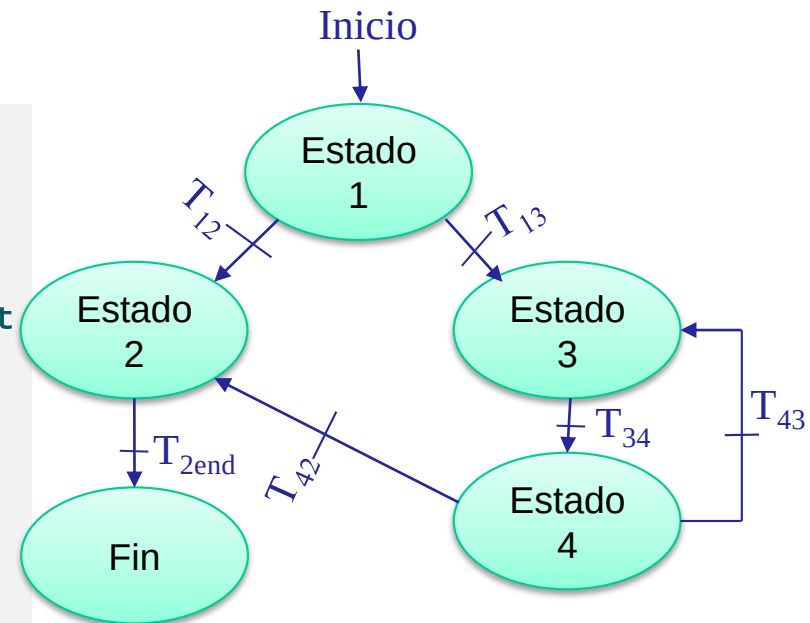


Control secuencial

- ❑ Máquina de estados en C: valor entero para el estado

```
#define ESTADO_1    1  // Comentario sobre el estado
#define ESTADO_2    2  // Comentario sobre el estado
...
int main() {
    int estado[2]; // estado[0]=actual, estado[1]=ant
    estado[0]=estado[1]=ESTADO_1;
    AccionesEntradaEstado(estado[0]);

    while (estado[0]!=ESTADO_END) {
        estado[1]=estado[0];
        estado[0]=CompruebaCambioEstado(estado[1]);
        if (estado[0]!=estado[1]) {
            AccionesSalidaEstado(estado[1]);
            AccionesEntradaEstado(estado[0]);
        }
        AccionesDentroEstado(estado[0]);
        delay(Tm);
    }
}
```



Control secuencial

- Máquina de estados en C: valor entero para el estado
 - `void AccionesEntradaEstado(int estado);`
Acciones a realizar solamente cuando se entra en el estado: inicializar variables (tiempo, contadores), actualizar salidas, guardar log, etc.
 - `void AccionesSalidaEstado(int estado);`
Acciones a realizar solamente cuando se sale del estado: actualizar salidas, guardar log, etc.
 - `void AccionesDentroEstado(int estado);`
Acciones a realizar para cada periodo de muestreo cuando nos encontramos en este estado: lazo de control, activación de salidas, etc.
 - `int CompruebaCambioEstado(int estado);`
Chequea las condiciones de las diferentes transiciones de salida del estado; si alguna se cumple, devuelve el nuevo estado que se activa.

Control secuencial

- ❑ Máquina de estados en C: valor enum para el estado
 - Un tipo de datos enum es equivalente a un entero, pero se especifica en su declaración los valores "únicos" (constantes) que pueden tener las variables de este tipo
 - Si no se da valor a las constantes, se les asigna uno por defecto (comenzando en 0, incrementando en 1)
 - enum simplemente sirve para clarificar el significado de una variable

```
enum estadosEnum
{ ESTADO_1 , // Comentario sobre el estado, ESTADO_1 ≡ 0
  ESTADO_2 , // Comentario sobre el estado, ESTADO_2 ≡ 1
  ...
  ESTADO_X = 37 , // Asigna valor, ESTADO_X ≡ 37
  ESTADO_Y , // Comentario sobre el estado, ESTADO_Y ≡ 38
  ...
} ; // !!!! OJO PUNTO Y COMA AL FINAL !!!!

...
int main() {
    enum estadosEnum estado[2];
    estado[0]=estado[1]=ESTADO_1;

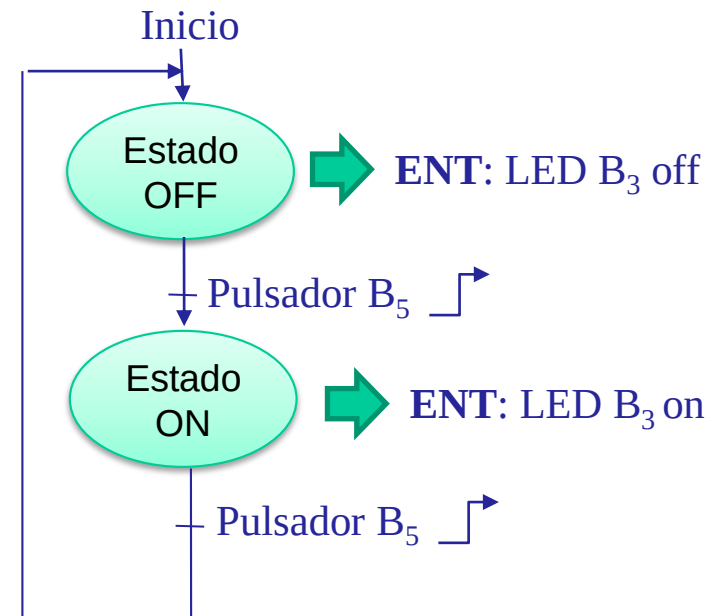
    ... Resto igual ...
}
```

Control secuencial

- Un ejemplo sencillo: pulsador que cambia estado ON/OFF de un LED

```
enum estadosEnum { ESTADO_OFF, ESTADO_ON };
...
int main() {
    enum estadosEnum[]={ESTADO_OFF, ESTADO_OFF};
    int di[2]={0,0}
    Desactivar Bit 3 de salidas digitales
    while (1) {
        di[1]=di[0];
        di[0]=Leer entradas digitales
        if ((Bit Peso 5 de di[0] está activo) Y TAMBIÉN
            (Bit Peso 5 de di[1] NO está activo) {
            estado[1]=estado[0];
            switch (estado[1]) {
            case ESTADO_OFF:
                estado[0]=ESTADO_ON;
                Activar Bit 3 de salidas digitales
                break;
            case ESTADO_ON:
                estado[0]=ESTADO_OFF;
                Desactivar Bit 3 de salidas digitales
                break;
            } // Fin switch

        } // Fin if
        delay(Tm);
    }
}
```



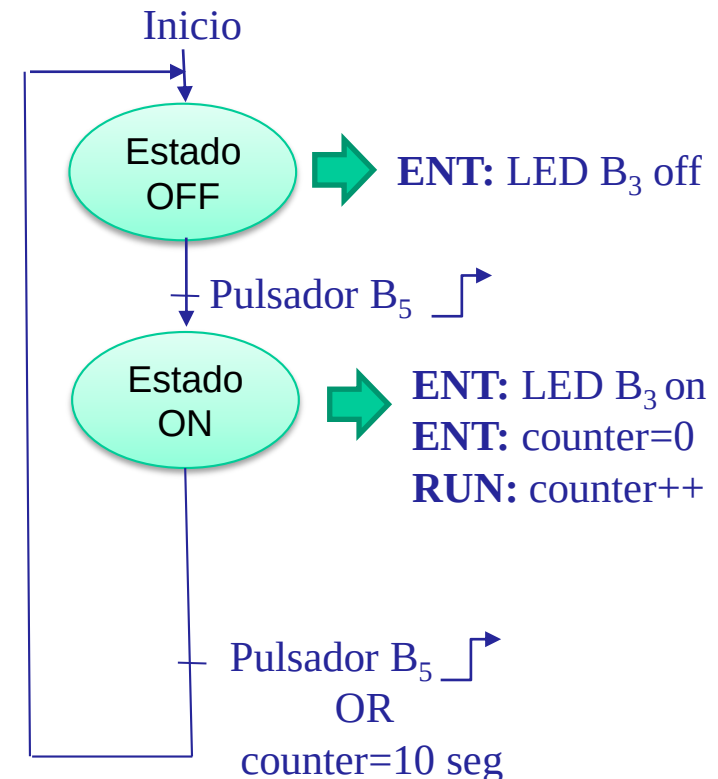
Control secuencial

- Un ejemplo sencillo: pulsador que cambia estado ON/OFF de un LED, mantiene encendido 10 seg máx

```
enum estadosEnum { ESTADO_OFF, ESTADO_ON };

...
int main() {
    int counter_ms=0;
    ...
    while (1) {
        di[1]=di[0];
        ...
        case ESTADO_OFF:
            estado[0]=ESTADO_ON;
            Desactivar Bit 3 de salidas digitales
            counter_ms=0;
            break;
        } // Fin switch
    } // Fin if

    if (estado[0]==ESTADO_ON) {
        counter_ms+=Tm_ms;
        if (counter_ms >= 10000) {
            estado[1]=estado[0];
            estado[0]=ESTADO_OFF;
            Desactivar Bit 3 de salidas digitales
        }
    }
    delay(Tm_ms);
}
```



Control secuencial

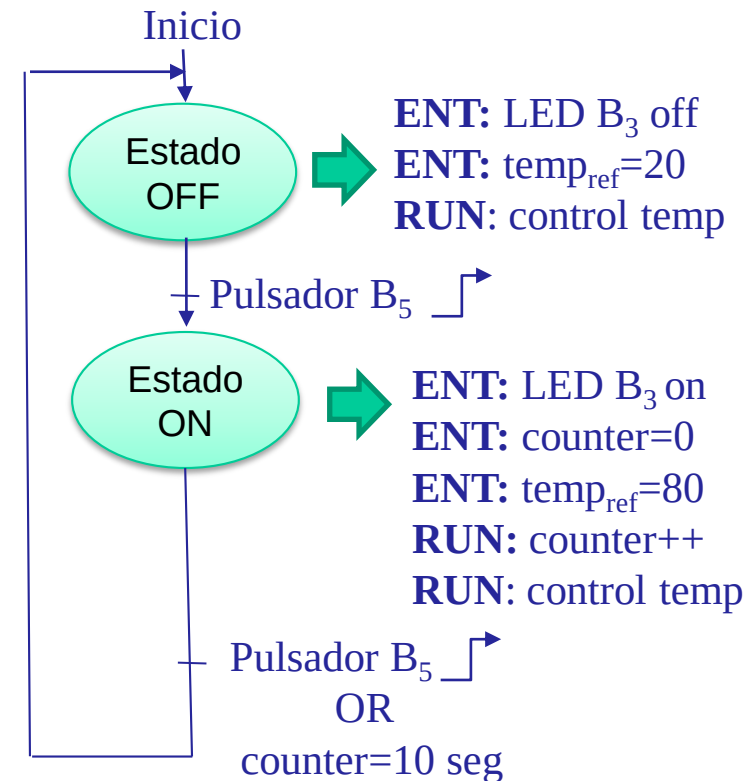
- Un ejemplo sencillo: pulsador que cambia temperatura consigna, mantiene activo 10 seg máx

```
enum estadosEnum { ESTADO_OFF, ESTADO_ON };

...
int main() {
    int counter_ms=0;
    float temp_consigna=20,temp_actual;
    ...
    while (1) {
        di[1]=di[0];
        ...
        case ESTADO_OFF:
            ...
            temp_consigna=20; // Ambiente
            break;
        } // Fin switch
    } // Fin if

    if (estado[0]==ESTADO_ON) {
        temp_consigna=80; // Calentar
        ...
    }

    Leer temp_actual → err=temp_consigna-temp_actual
    → actuar para calentar si es necesario
    delay(Tm_ms);
}
```

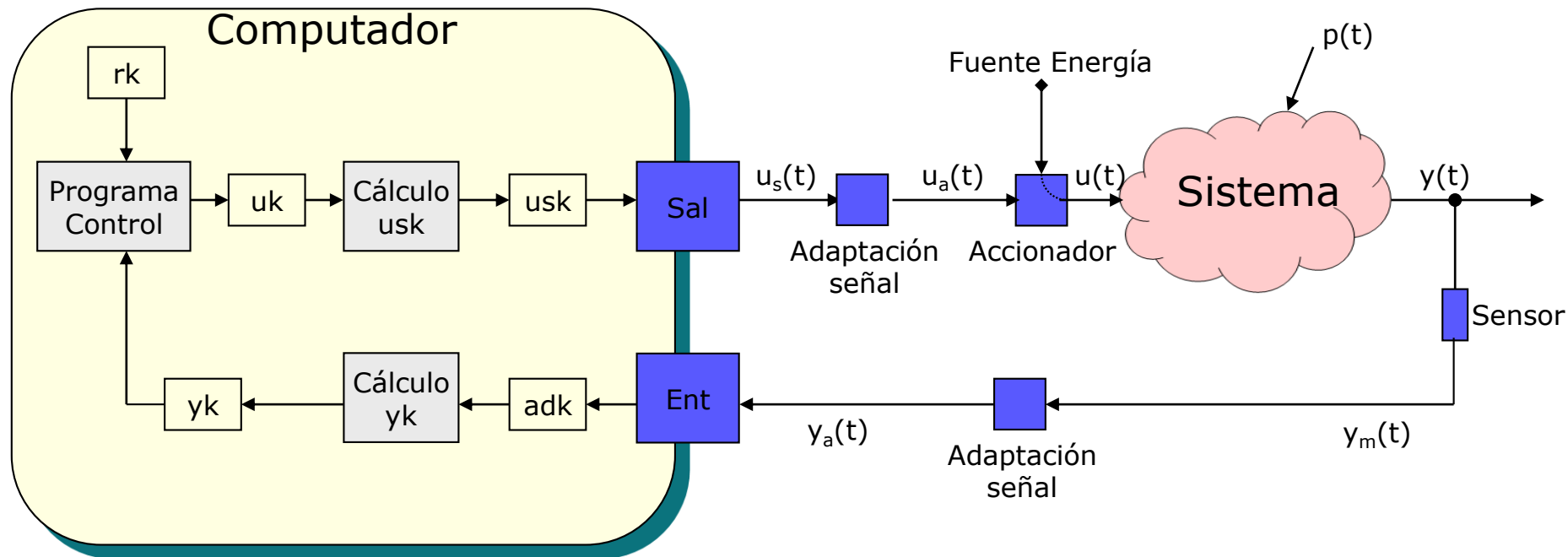


Indice

- ❑ Introducción control de procesos por computador
- ❑ Conceptos básicos para programación de control
- ❑ Interfaz del computador con el exterior
- ❑ El control secuencial
- ❑ **Las matemáticas del control**
- ❑ Programación del lazo de control
- ❑ Programación en lenguaje C
- ❑ Implantación del control en el computador

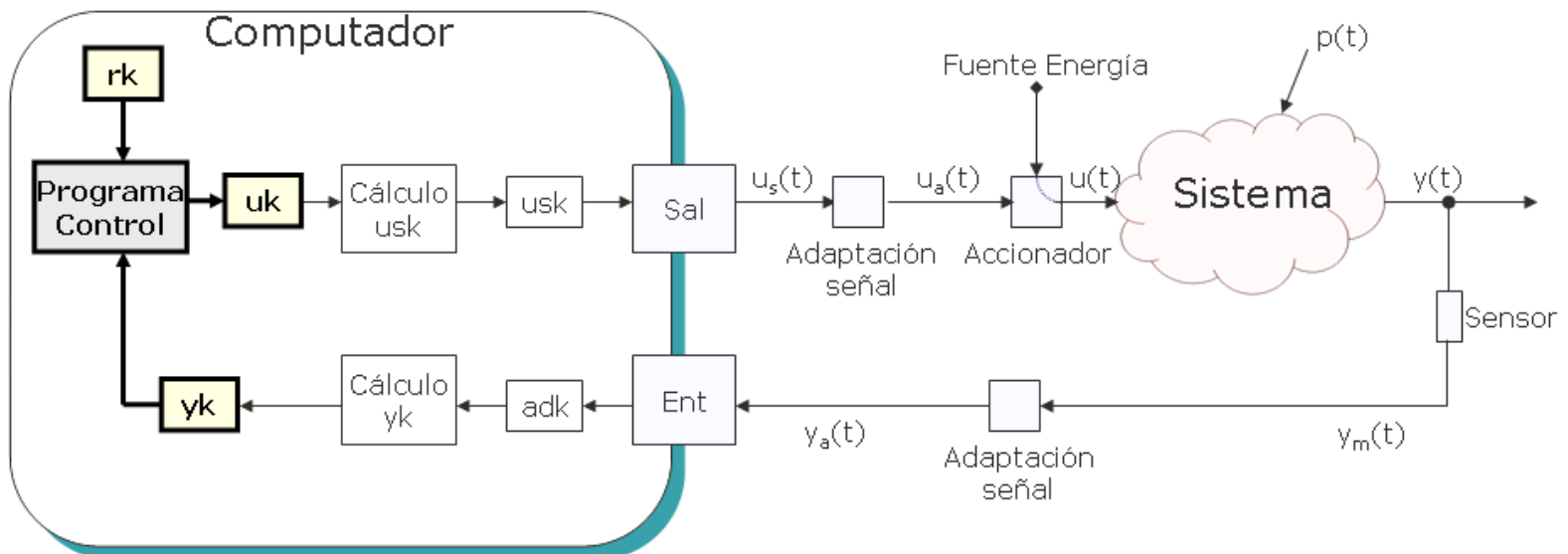
El lazo de control

- Esquema típico del lazo de control por computador (1 ent, 1 sal):



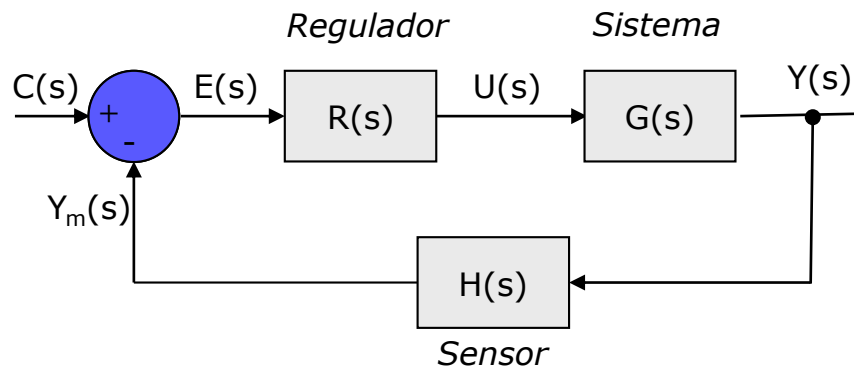
Las matemáticas del control

- El cálculo óptimo de $u_k = F_n(y_k, r_k)$ es crucial para satisfacer las necesidades del control:
 - **Precisión:** la salida debe seguir fielmente a la referencia.
 - **Velocidad:** la salida debe seguir rápidamente las variaciones de la referencia.
 - **Robustez:** las perturbaciones, ruidos, o variaciones del sistema no deben modificar el comportamiento.



Las matemáticas del control

- En las asignaturas de control se estudia la acción de control a aplicar de forma matemática mediante transformadas de Laplace :



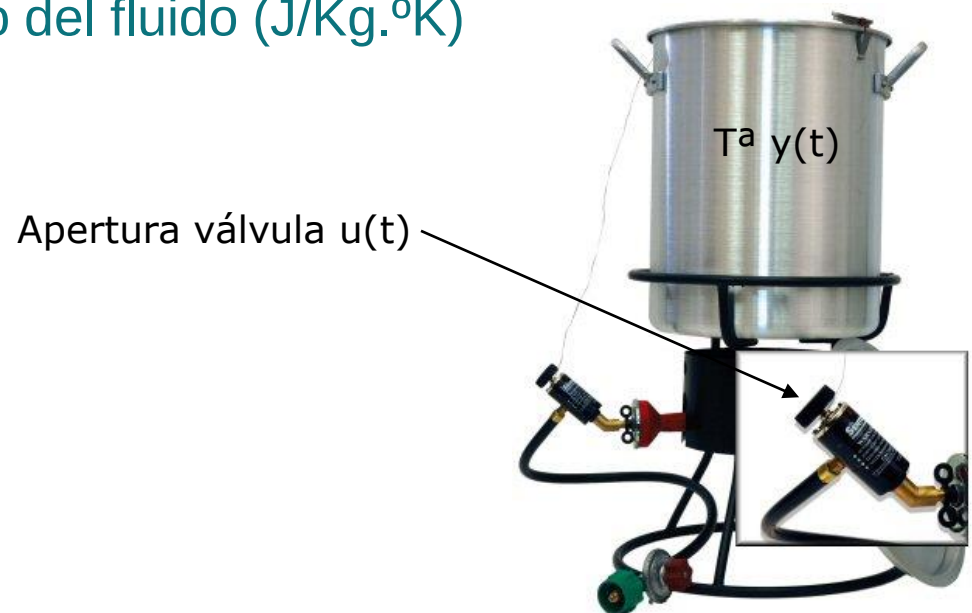
- Pero en el control por computador:
 - Sólo pueden realizar cálculos cada T_m (sistema discreto), por lo que no es aplicable la transformada de Laplace (requiere sistema continuo)
 - En el sistema real aparecen muchos más elementos que en el lazo de control “matemático”

El lazo de control

- ❑ Una aproximación intuitiva a los reguladores programados:
 - Mediante ejemplos se verán las diferentes acciones P-I-D y la forma de programarlas en el computador
 - Se llega a un formato general de cálculo para cualquier tipo de regulador
 - Por último, se relacionan los conocimientos previos de teoría de control analógica con el cálculo a realizar

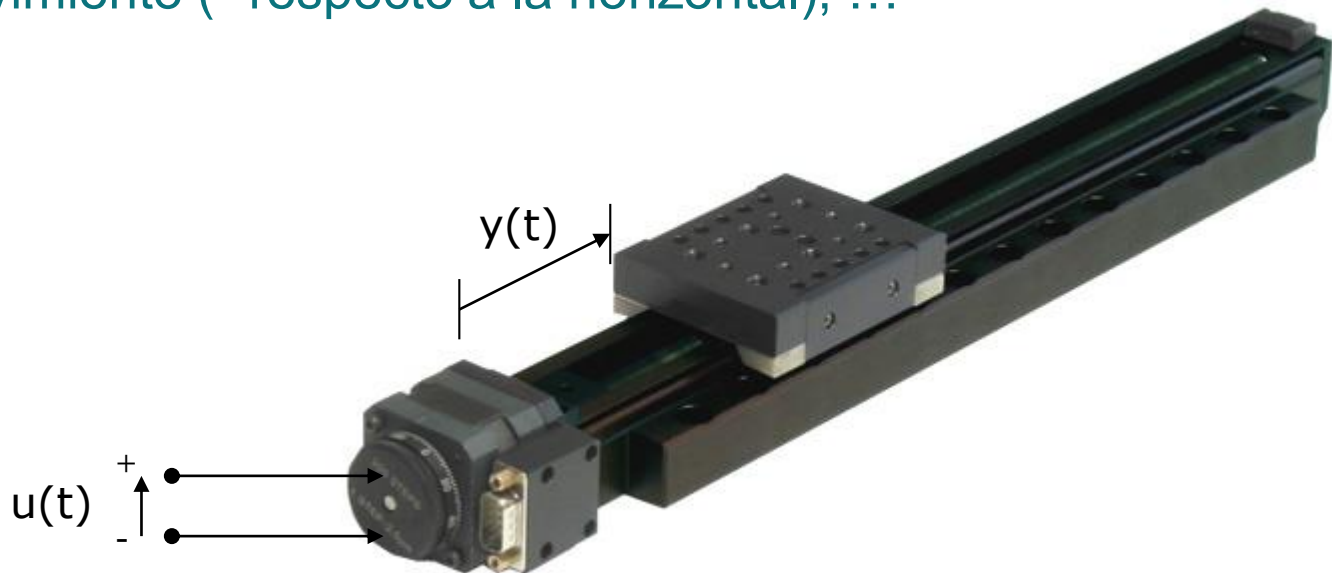
Las matemáticas del control

- Ejemplo 1: control de temperatura de un fluido mediante una llama de gas.
 - **Referencia:** temperatura deseada del fluido ($^{\circ}\text{C}$)
 - **Salida del sistema:** temperatura medida del fluido ($^{\circ}\text{C}$)
 - **Acción de control:** apertura de la válvula de gas (0%-100%)
 - **Perturbaciones:** temperatura exterior ($^{\circ}\text{C}$), cantidad de fluido (Kg), calor específico del fluido ($\text{J/Kg}^{\circ}\text{K}$)



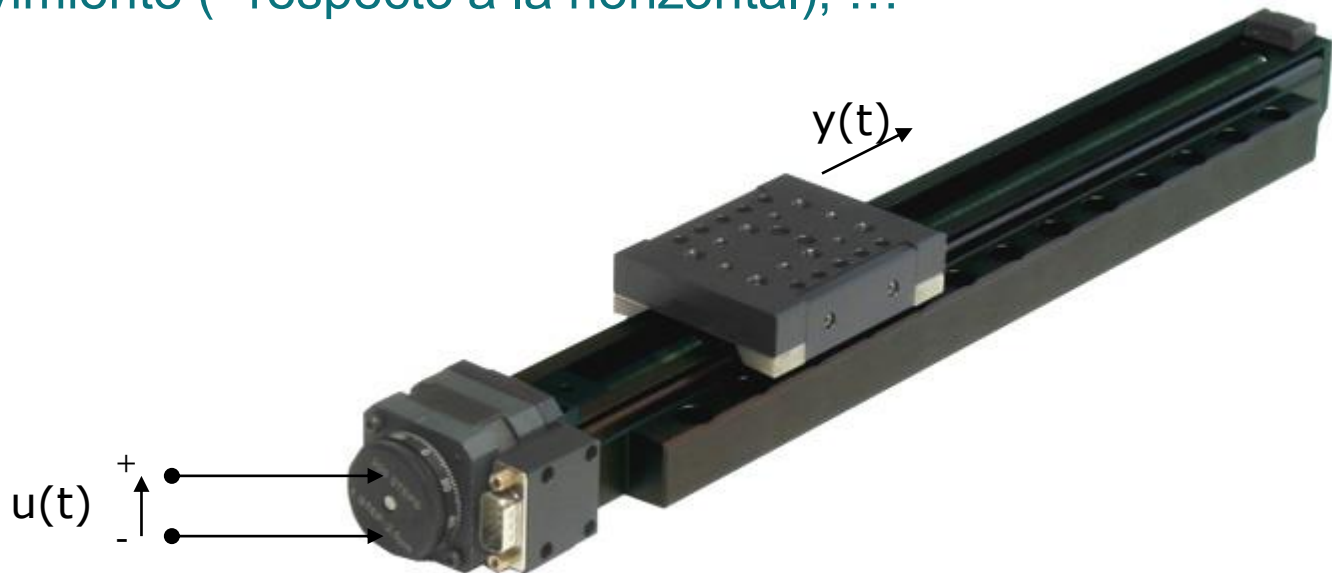
Las matemáticas del control

- Ejemplo 2: control de posición de un móvil mediante un motor DC.
 - **Referencia:** posición deseada del móvil (mm).
 - **Salida del sistema:** posición medida del móvil (mm).
 - **Acción de control:** tensión aplicada al motor (V).
 - **Perturbaciones:** peso del móvil (Kg), dirección del movimiento ($^{\circ}$ respecto a la horizontal), ...



Las matemáticas del control

- Ejemplo 3: control de velocidad de un móvil mediante un motor DC.
 - **Referencia:** velocidad deseada del móvil (mm/s).
 - **Salida del sistema:** velocidad medida del móvil (mm/s).
 - **Acción de control:** tensión aplicada al motor (V).
 - **Perturbaciones:** peso del móvil (Kg), dirección del movimiento ($^{\circ}$ respecto a la horizontal), ...



Las matemáticas del control

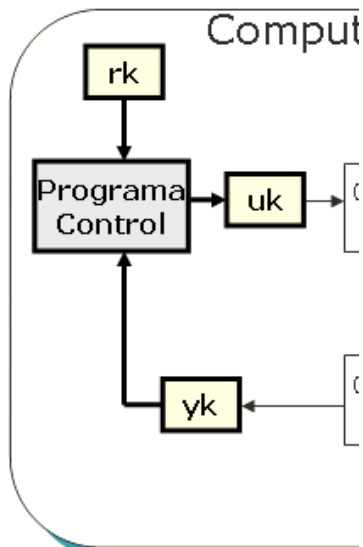
□ Un cálculo sencillo: control todo/nada

Si $y_k < r_k \rightarrow u_k = \text{cte1}$

Si $y_k > r_k \rightarrow u_k = \text{cte2}$

Si $y_k = r_k \rightarrow \text{no modificar } u_k$

- Se acciona el sistema con un valor constante, que depende del signo del error.
- Cálculo excesivamente simple: la acción de control no depende del valor absoluto del error.



Ejemplo 1:

- Se abre la válvula de gas (100%) si la T^a es menor que la referencia.
- Se cierra la válvula (0%) si la T^a es mayor o igual que la referencia.
- La apertura es la misma para $\text{error} = 0.1^\circ\text{C}$ ó $\text{error} = 100^\circ\text{C}$
- Excesivas conmutaciones cuando $y_k \approx r_k$ (se suele usar histéresis)

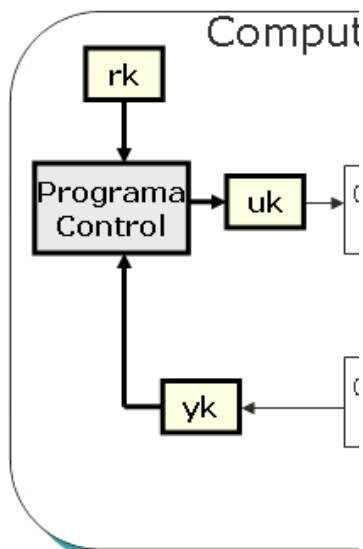
Las matemáticas del control

□ Un cálculo sencillo: control proporcional

$$e_k = r_k - y_k$$

$$u_k = cte \cdot e_k$$

- Cuando el error es 0, la acción de control es 0.
- La acción de control es mayor (en valor absoluto) cuanto más grande sea el error (en valor absoluto), y tiene el mismo signo que el error.
- El cálculo es muy simple

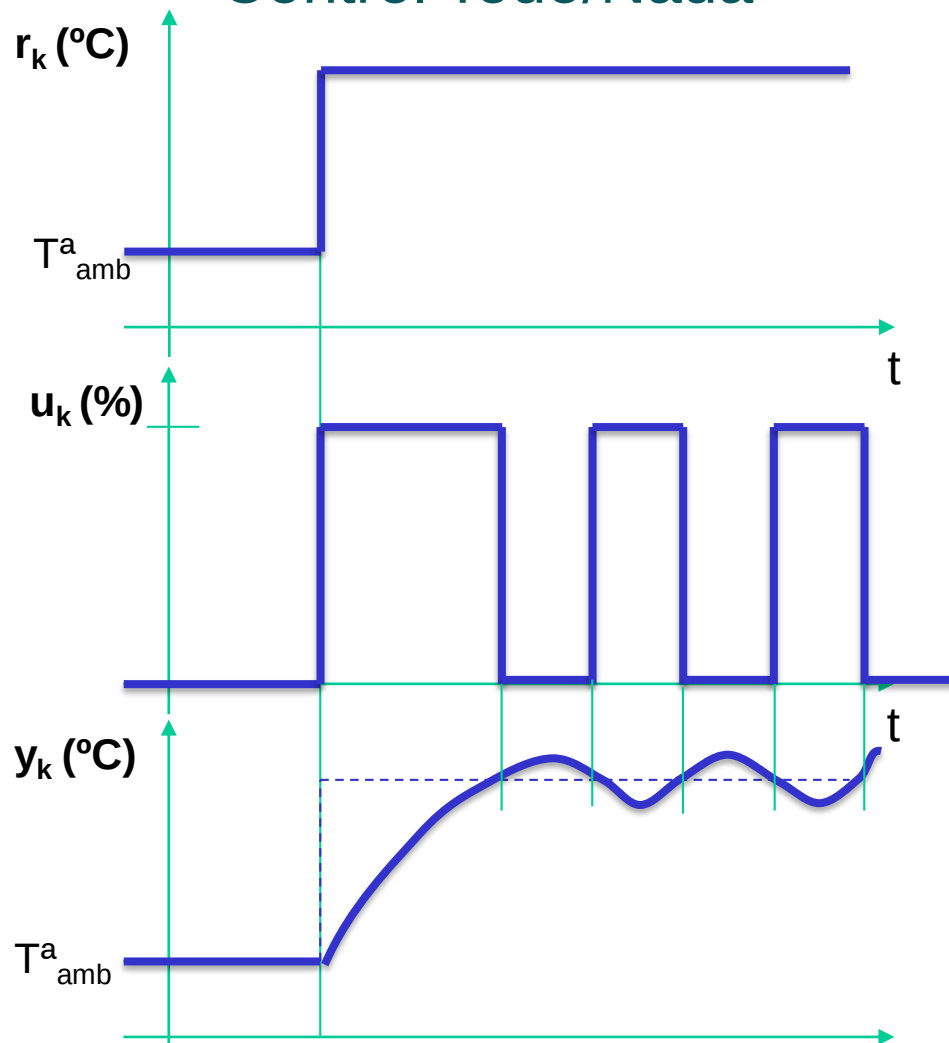


Ejemplo 1:

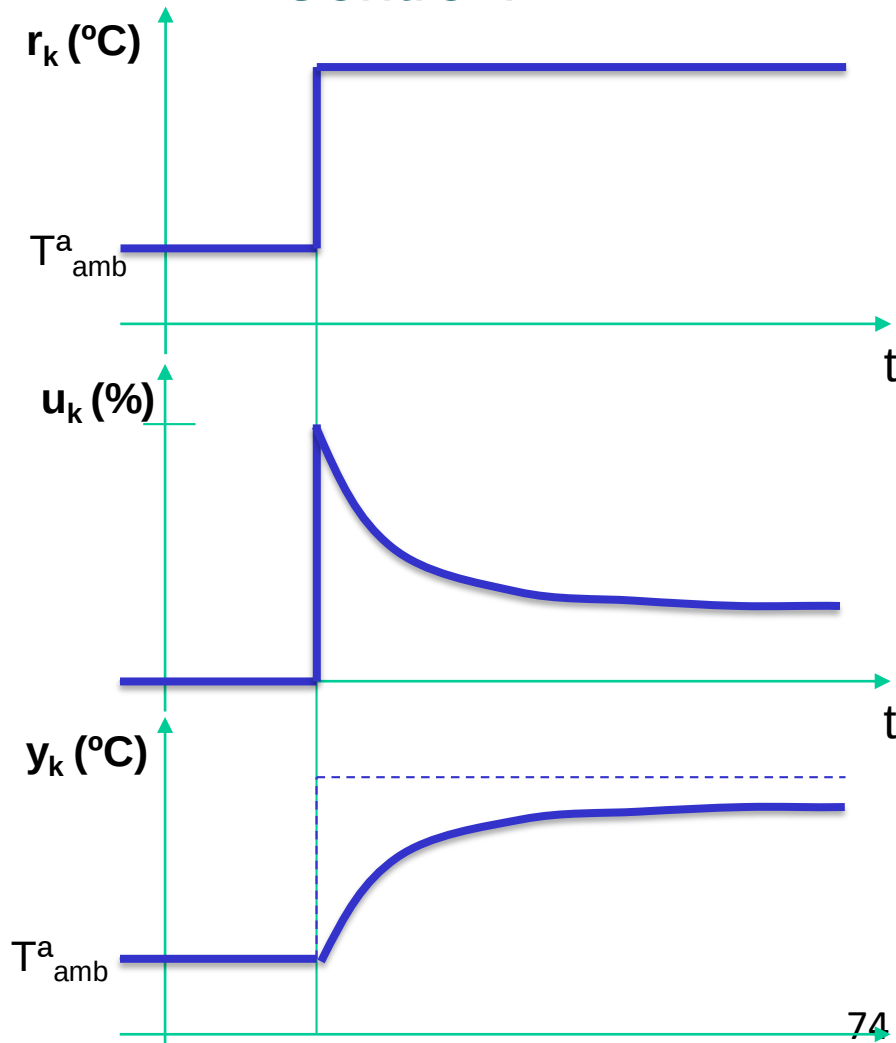
- Se abre más la válvula de gas cuanto mayor sea el error.
- Se cierra la válvula (0%) si la T^a iguala a la referencia.
- No se tiene en cuenta si la T^a estaba subiendo o bajando previamente.

Las matemáticas del control

Control Todo/Nada



Control P



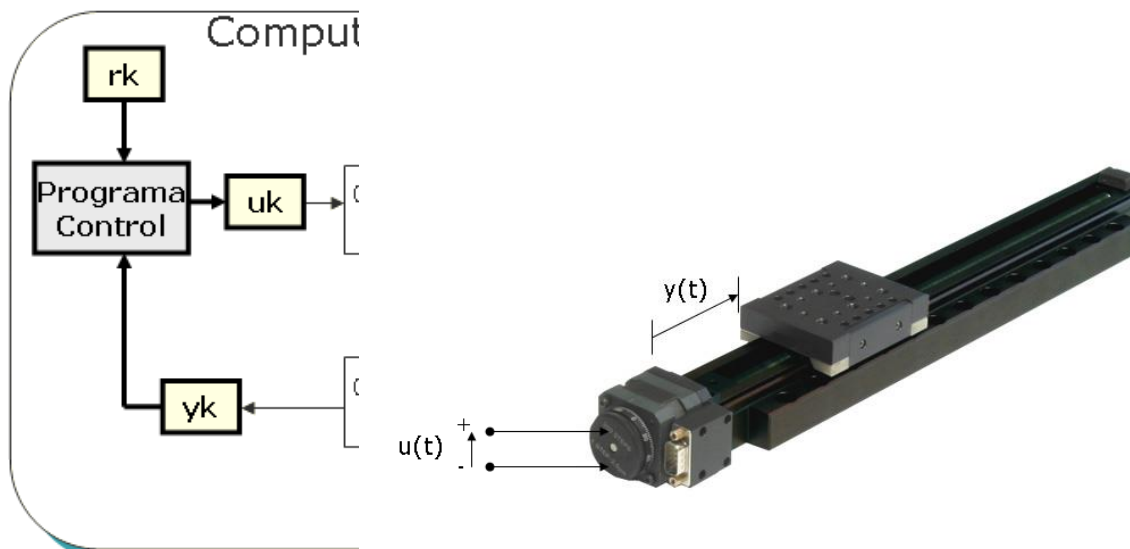
Las matemáticas del control

□ Un cálculo sencillo: control proporcional

$$e_k = r_k - y_k$$

$$u_k = cte \cdot e_k$$

- Cuando el error es 0, la salida es 0.
- La salida es mayor (en valor absoluto) cuanto más grande sea el error (en valor absoluto), y tiene el mismo signo que el error.
- El cálculo es muy simple: no tiene en cuenta la evolución que llevaba el sistema anteriormente.



Ejemplo 2:

- Se da más tensión al motor cuanto mayor sea la distancia al destino.
- Se para el motor (tensión 0) cuando alcanza el destino (error 0).
- No se tiene en cuenta si el móvil se está acercando o alejando del destino.

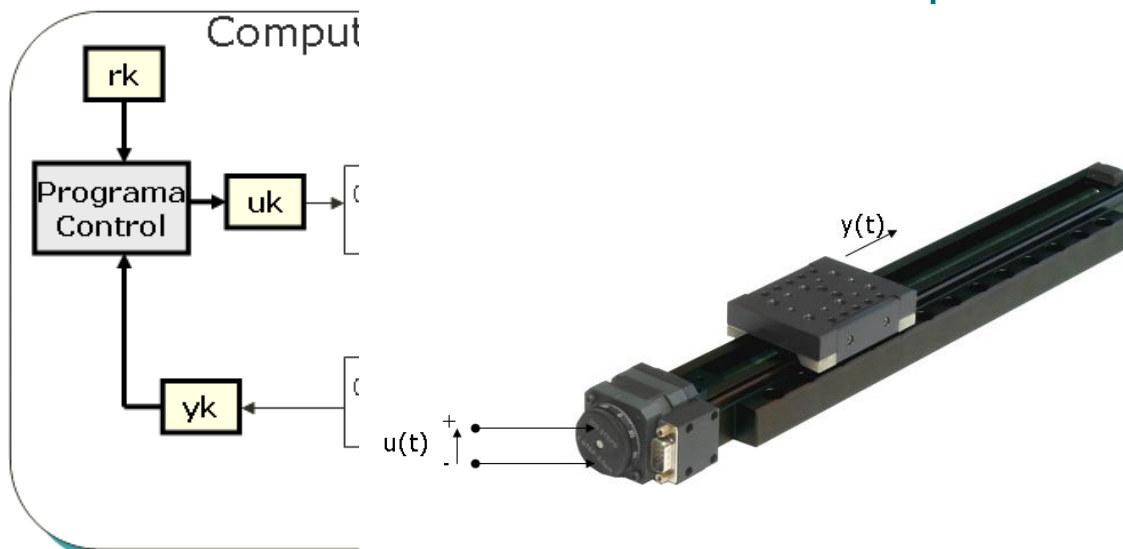
Las matemáticas del control

□ Un cálculo sencillo: control proporcional

$$e_k = r_k - y_k$$

$$u_k = cte \cdot e_k$$

- Cuando el error es 0, la acción de control es 0.
- La acción de control es mayor (en valor absoluto) cuanto más grande sea el error (en valor absoluto), y tiene el mismo signo que el error.
- En ciertos sistemas, no se puede llegar a error 0.



Ejemplo 3:

- Se aumenta la tensión del motor si la velocidad es menor que la deseada
- Si la velocidad fuese igual a la deseada, la tensión aplicada sería 0V, por tanto el motor tendería a pararse.
- Es necesario mantener un error para que el sistema se estabilice.

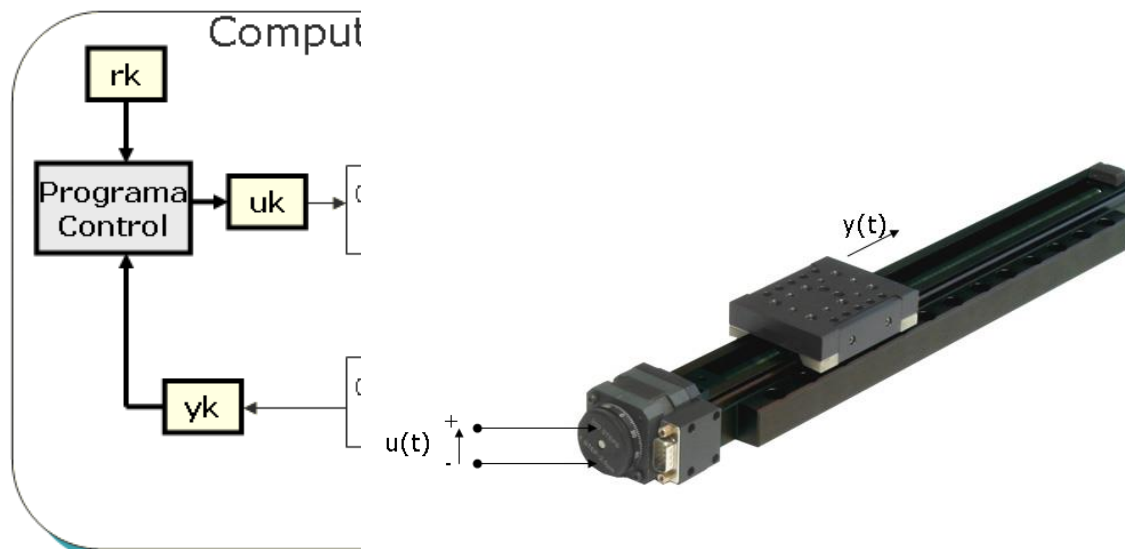
Las matemáticas del control

□ Un cálculo sencillo: control proporcional-integral

$$e_k = r_k - y_k$$

$$u_k = u_{k_anterior} + cte \cdot e_k$$

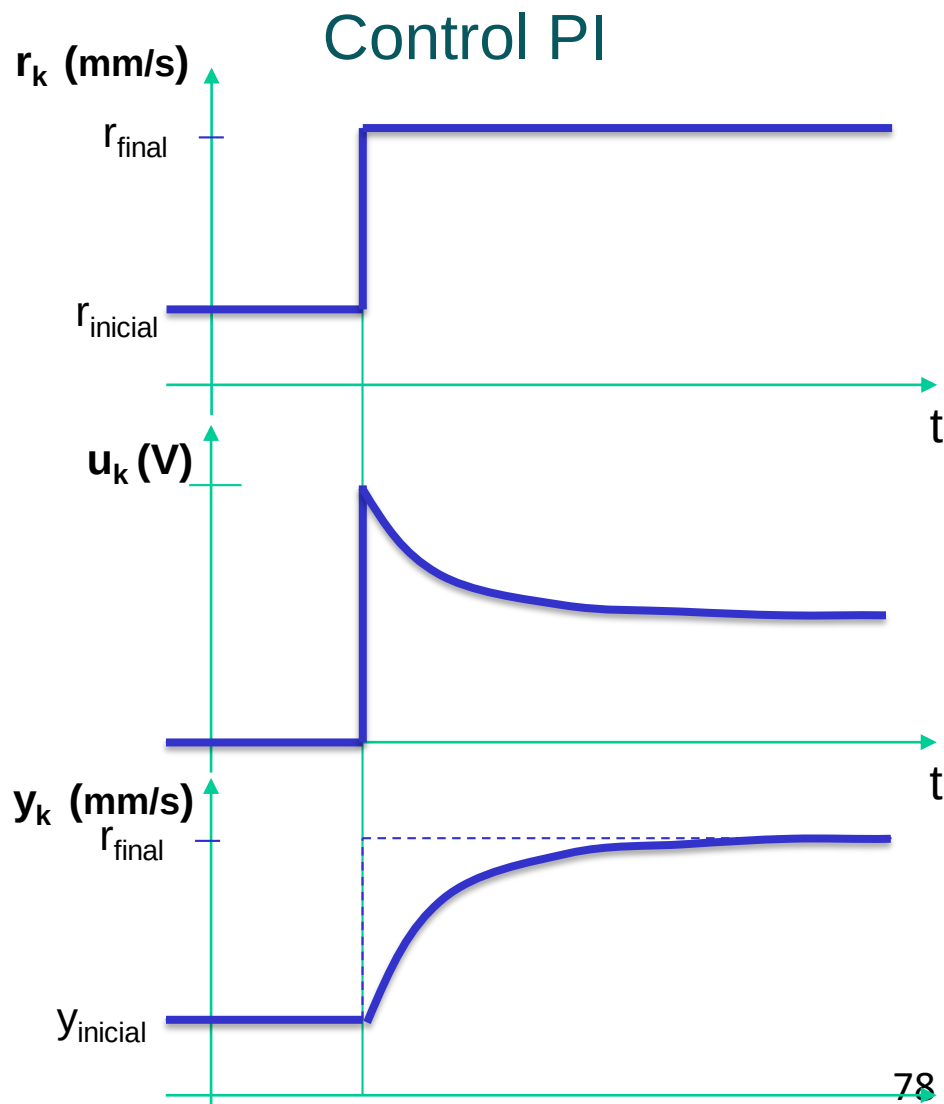
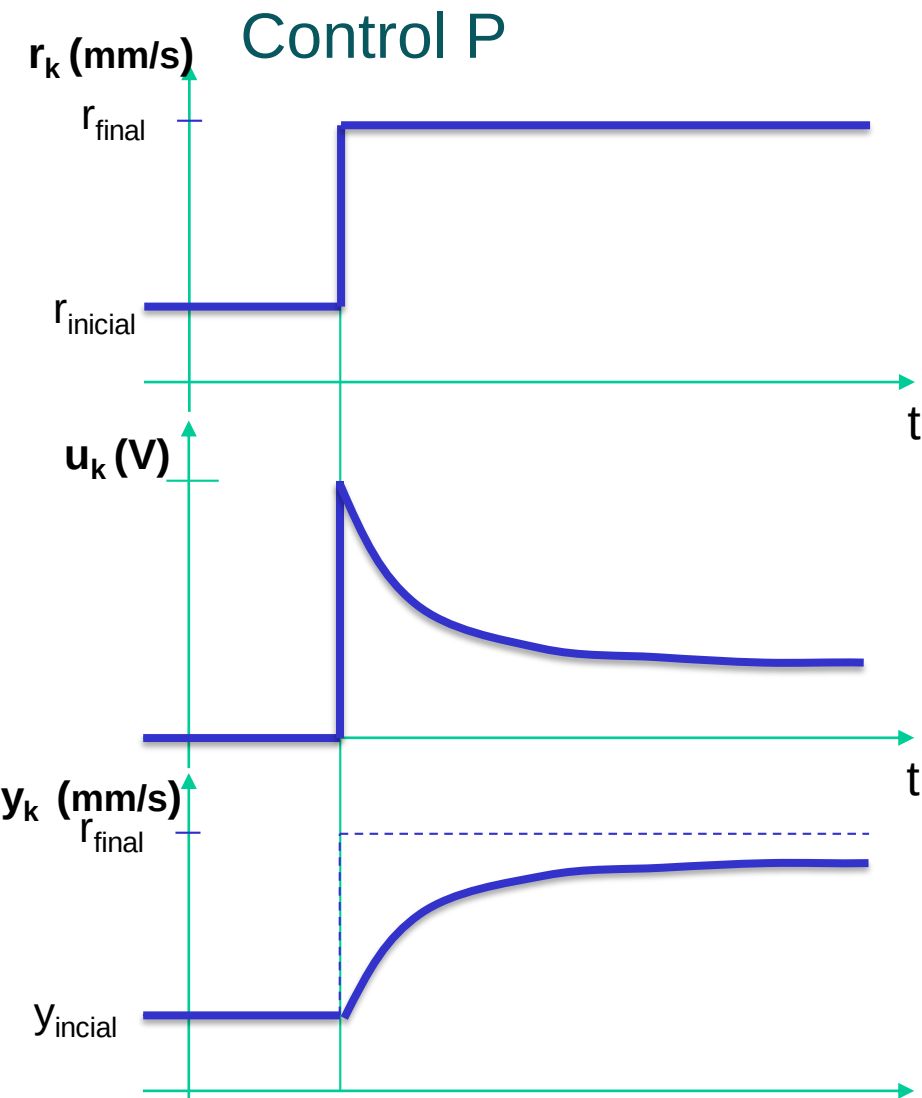
- Cuando el error es 0, la acción de control se mantiene.
- La acción de control se incrementa (en valor absoluto) tanto más cuanto más grande sea el error (en valor absoluto), y el incremento tiene el mismo signo que el error.
- El cálculo necesita recordar un valor anterior de u_k .



Ejemplo 3:

- Se aumenta la tensión del motor si la velocidad es menor que la deseada.
- Si la velocidad es igual a la deseada, se mantiene la tensión aplicada.
- El error llega a 0.

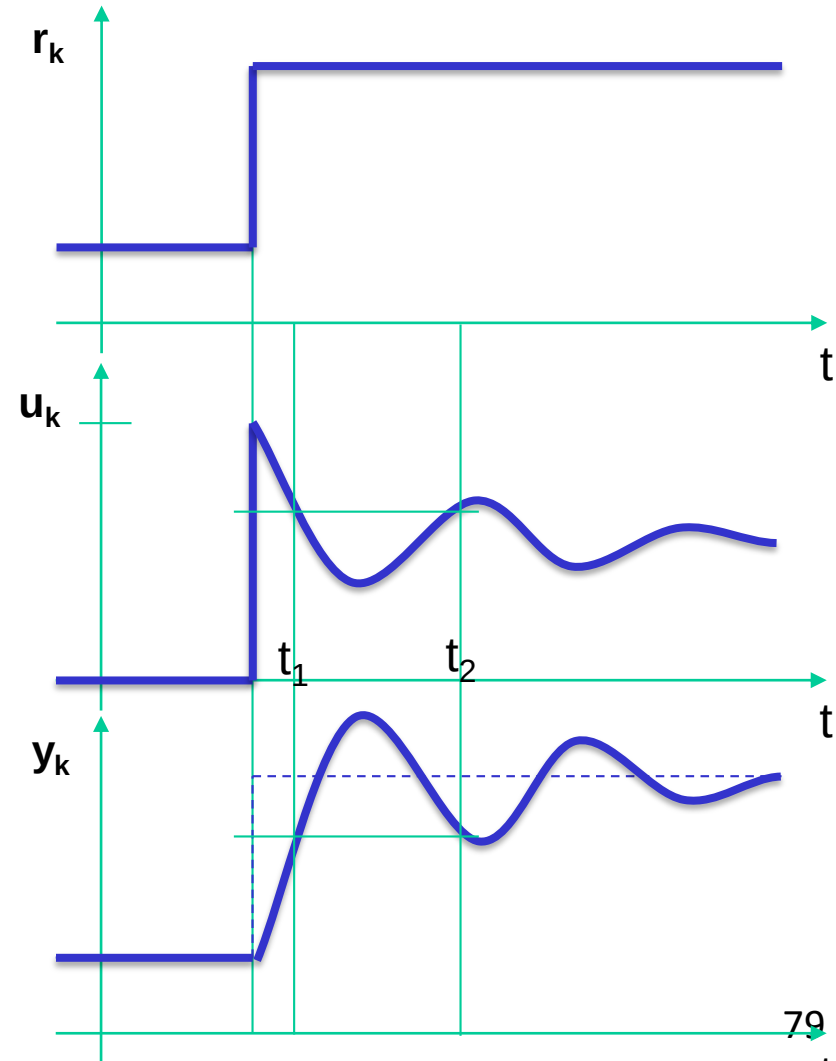
Las matemáticas del control



Las matemáticas del control

Control PI:

- ❑ En $t=t_1$ y $t=t_2$, la acción de control es la misma (mismo e_k), aunque las situaciones son distintas:
 - En $t=t_1$ el error está disminuyendo
 - En $t=t_2$ el error está aumentando
- ❑ Solución: Añadir acción dependiente de la derivada del error



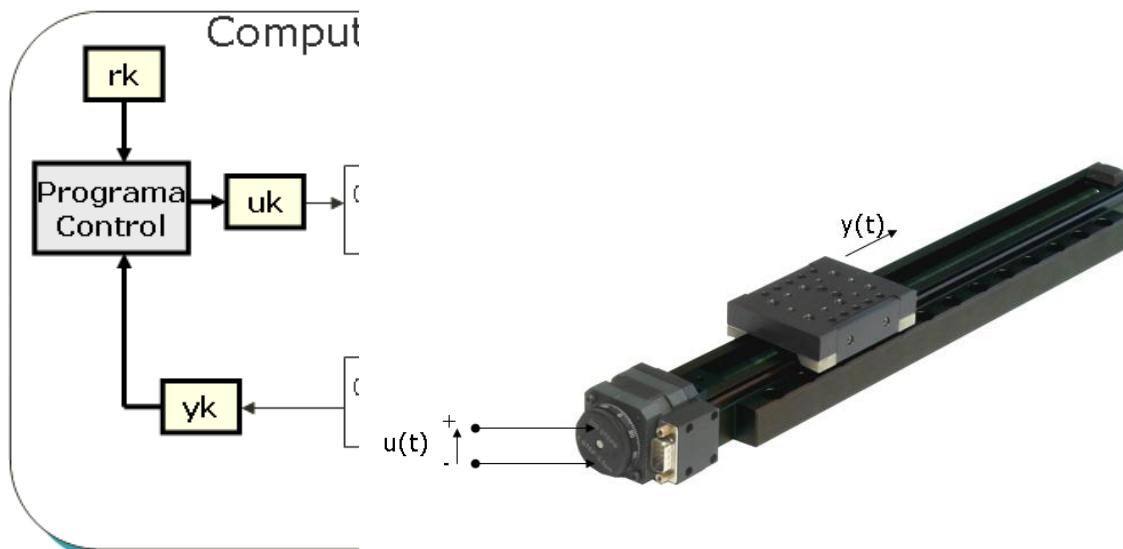
Las matemáticas del control

□ Un cálculo sencillo: control proporc.-integral-diferencial

$$e_k = r_k - y_k$$

$$u_k = u_{k_anterior} + cte1 \cdot e_k + cte2 \cdot (e_k - e_{k_anterior}) / T_m$$

- Cuando el error se mantiene en 0, la salida se mantiene.
- La acción de control se incrementa tanto más cuanto mayor sea el error, y cuanto el error vaya más en sentido contrario al deseado.
- El cálculo necesita recordar un valor anterior de u_k y e_k .

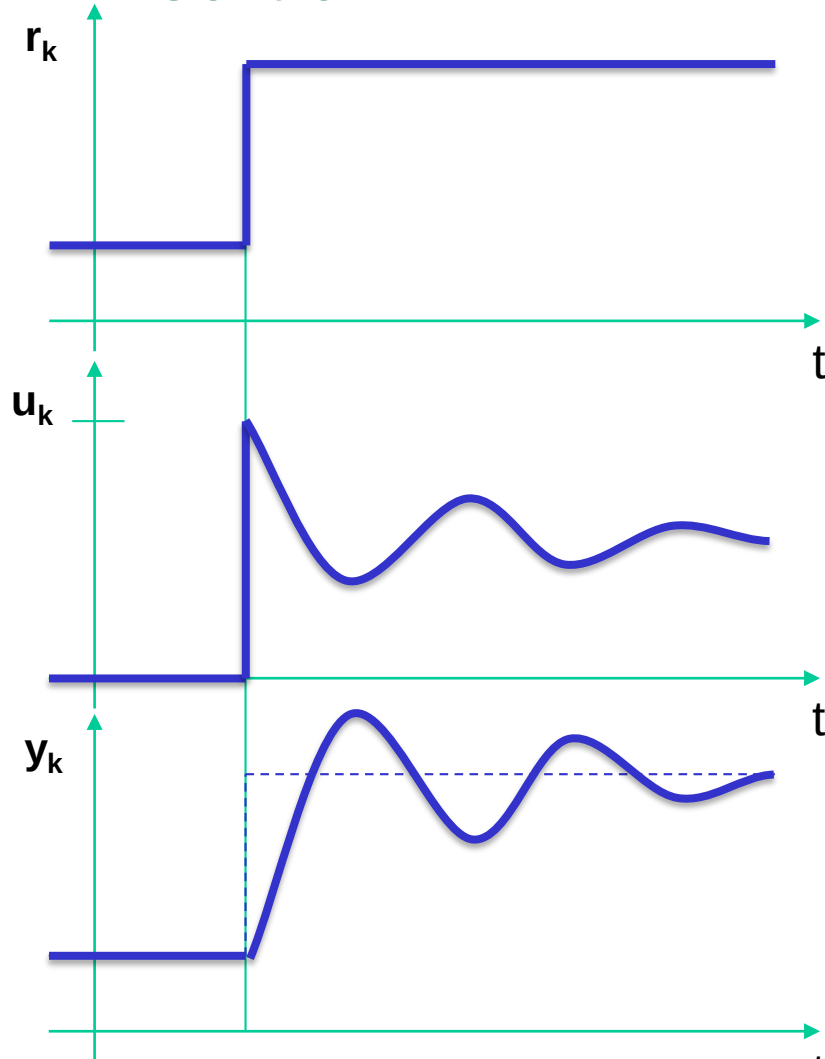


Ejemplo 3:

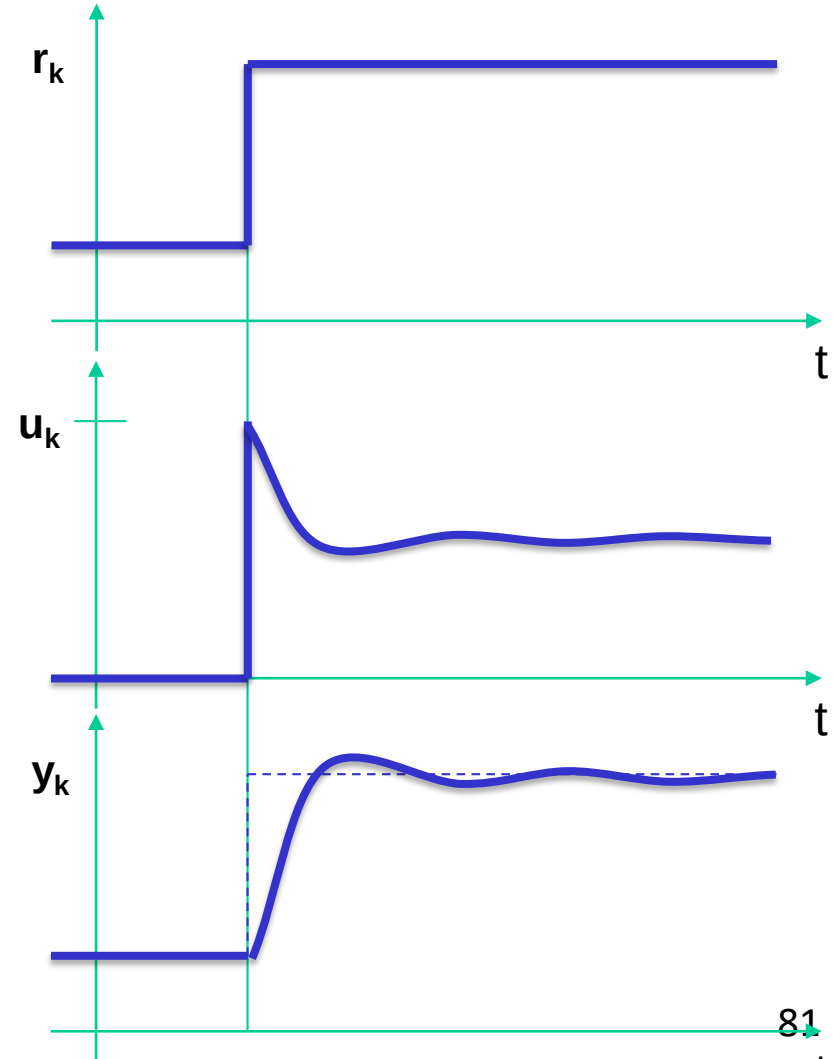
- Se aumenta la tensión del motor si la velocidad es menor que la deseada.
- Si la velocidad es igual a la deseada, se mantiene la tensión aplicada.
- El error llega a 0 más rápidamente, al tenerse en cuenta su derivada.

Las matemáticas del control

Control PI



Control PID



Las matemáticas del control

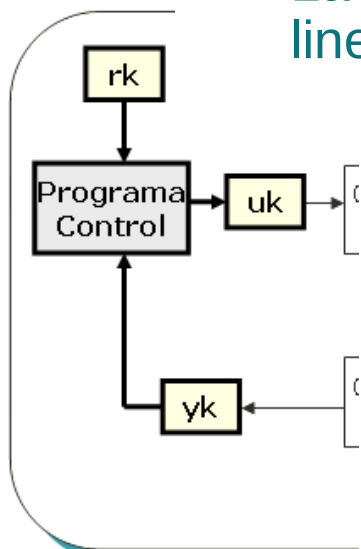
□ Cálculo general:

$$e_k = r_k - y_k$$

$$u_k = F_n(e_k, e_{k-1}, e_{k-2}, \dots, e_{k-m}, u_{k-1}, u_{k-2}, \dots, u_{k-n})$$

- La función realiza un cálculo con los valores actual y anteriores (hasta retraso m) del error, y los valores anteriores (hasta retraso n) de la acción de control calculada.
- Se pueden diseñar múltiples funciones, pero es adecuado disponer de una teoría unificada para calcularlas.
- La teoría unificada es más sencilla si se usan funciones lineales, ya que se puede aplicar el principio de superposición:

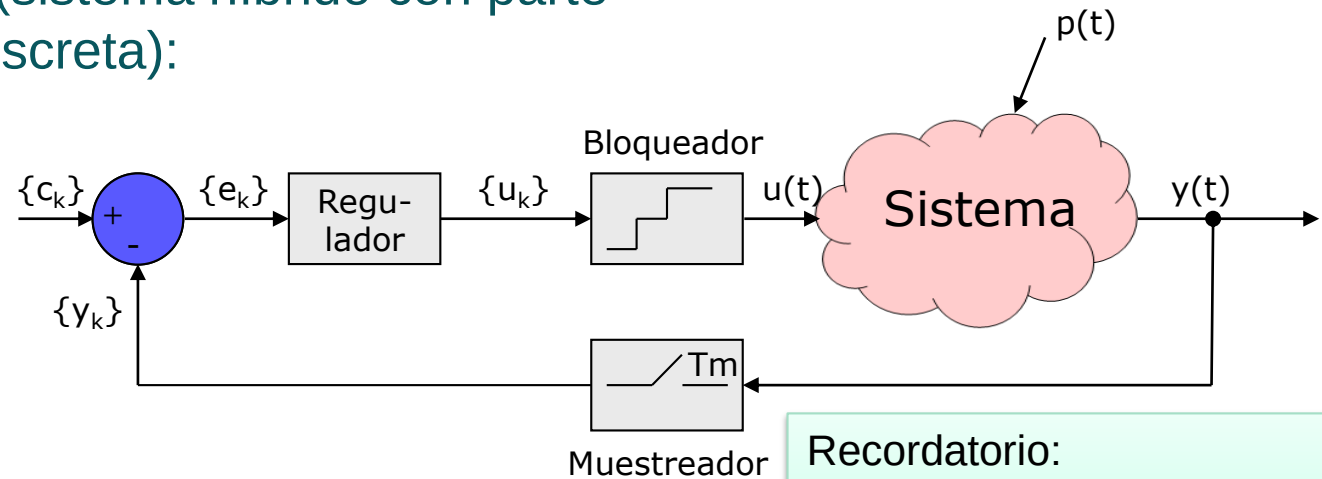
$$u_k = b_0 \cdot e_k + b_1 \cdot e_{k-1} + \dots + b_m \cdot e_{k-m} - a_1 \cdot u_{k-1} - a_2 \cdot u_{k-2} - \dots - a_n \cdot u_{k-n}$$



- Control proporcional:
» $m=0, n=0, b_0=cte$
- Control proporcional-integral:
» $m=0, n=1, b_0=cte, a_1=-1$
- Control proporcional-integral-diferencial:
» $m=1, n=1, b_0=cte1-cte2, b_1=cte2, a_1=-1$

Las matemáticas del control

- Simplificación del lazo de control para los cálculos (sistema híbrido con parte continua y discreta):



- La teoría más sencilla se obtiene (sistemas lineales):

- Calculando la Transformada de Laplace de las señales continuas
- Calculando la Transformada en Z de las señales discretas

Recordatorio:

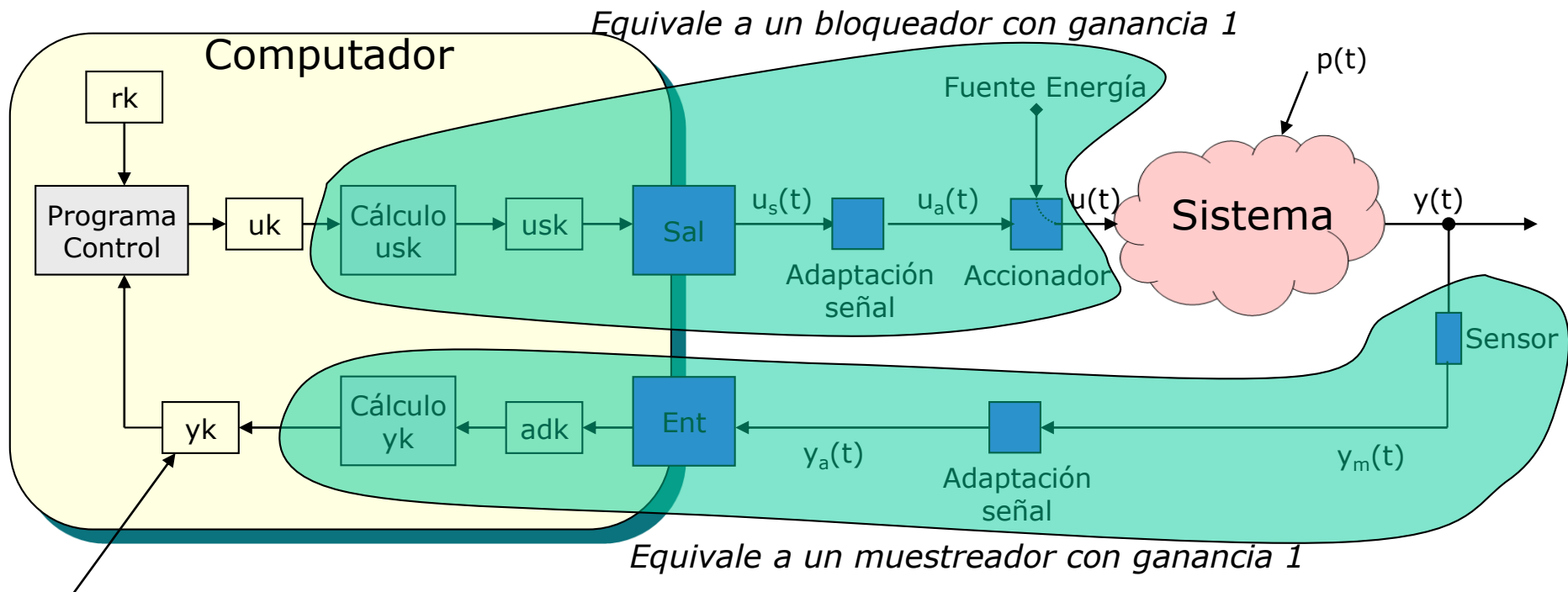
[El significado físico de los modelos matemáticos](#)

$$X(s) = \mathcal{L}[x(t)] = \int_{\tau=0}^t x(\tau) \cdot e^{-s\tau} \cdot d\tau$$

$$X(z) = \mathcal{Z}[\{x_k\}] = \sum_{i=0}^k x_{k-i} \cdot z^{-i}$$

Las matemáticas del control

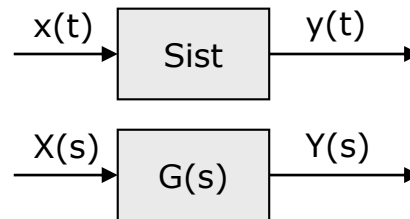
- ❑ Simplificación del lazo de control para los cálculos:



La representación matemática de los valores de las variables discretas a lo largo del tiempo es una secuencia: $\{y_k\}$

Las matemáticas del control

- Transformada de Laplace, relación entre transformada y señales temporales:



$$G(s) = \frac{Y(s)}{X(s)} = \frac{b_o.s^m + b_1.s^{m-1} + \dots + b_{m-1}.s + b_m}{s^n + a_1.s^{n-1} + \dots + a_{n-1}.s + a_n}, n \geq m$$

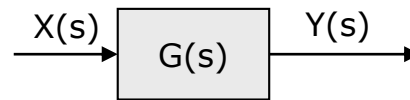


$$y^{(n)}(t) = b_o \cdot x^{(m)}(t) + b_1 \cdot x^{(m-1)}(t) + \dots + b_{m-1} \cdot \dot{x}(t) + b_m \cdot x(t) - \left(a_1 \cdot y^{(n-1)}(t) + \dots + a_{n-1} \cdot \dot{y}(t) + a_n \cdot y(t) \right)$$

- Ecuación diferencial del sistema

Las matemáticas del control

□ Transformada de Laplace, interpretación básica:



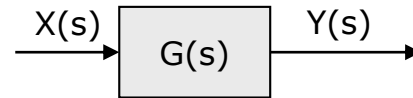
- $G(s) = Y(s)/X(s)$ para sistemas lineales dinámicos continuos.
- $G(s)$ es un cociente de polinomios, grado den $\geq$ grado num

$$G(s) = \frac{Y(s)}{X(s)} = \frac{b_0 \cdot s^m + b_1 \cdot s^{m-1} + \dots + b_{m-1} \cdot s + b_m}{s^n + a_1 \cdot s^{n-1} + \dots + a_{n-1} \cdot s + a_n}, \quad n \geq m$$

- Las raíces de los polinomios denominador (polos) y numerador (ceros) definen el comportamiento básico del sistema.
- Sistema estable: todos los polos con parte real negativa. En este caso:
 - Régimen permanente: $G(0)$ =ganancia estática (con unidades).
 - Respuesta dinámica (transitorio) dependiente de la posición de polos y ceros.

Las matemáticas del control

□ Transformada de Laplace, interpretación:

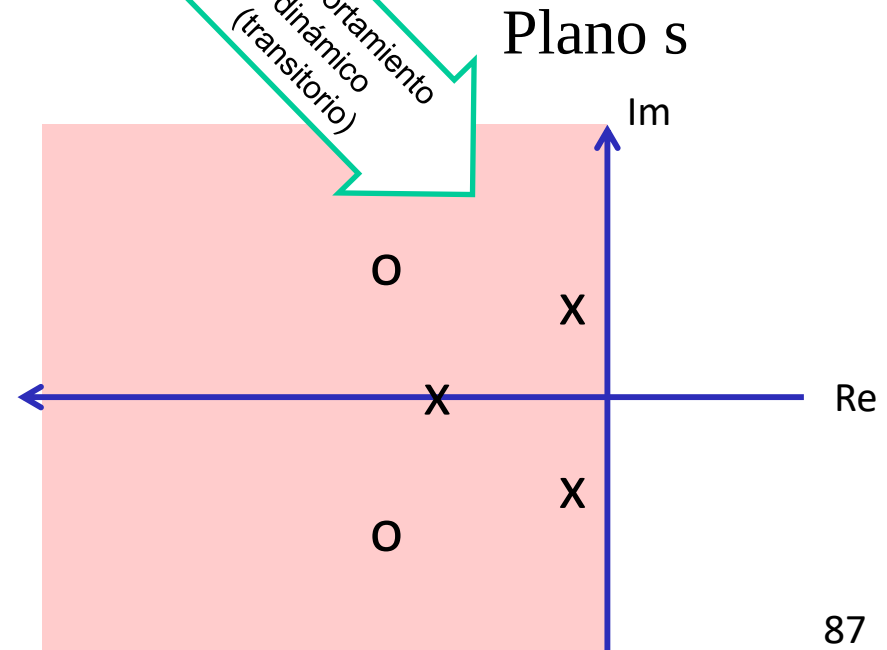


$$G(s) = \frac{Y(s)}{X(s)} = \frac{b_0 \cdot s^m + b_1 \cdot s^{m-1} + \dots + b_{m-1} \cdot s + b_m}{s^n + a_1 \cdot s^{n-1} + \dots + a_{n-1} \cdot s + a_n}, \quad n \geq m$$

Comportamiento
estático (régimen
permanente)

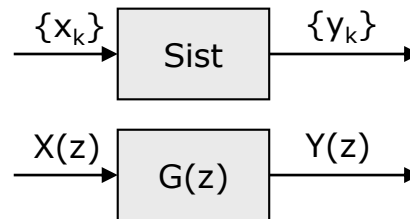
Comportamiento
dinámico
(transitorio)

$$K_p = \lim_{t \rightarrow \infty} \frac{y(t)}{x(t)} = \lim_{s \rightarrow 0} G(s)$$



Las matemáticas del control

- Transformada de en Z, relación entre transformada y señales temporales:



$$G(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 \cdot z^{-1} + \dots + b_{m-1} \cdot z^{-(m-1)} + b_m \cdot z^{-m}}{1 + a_1 \cdot z^{-1} + \dots + a_{n-1} \cdot z^{-(n-1)} + a_n \cdot z^{-n}}$$



$$y_k = b_0 \cdot x_k + b_1 \cdot x_{k-1} + \dots + b_{m-1} \cdot x_{k-(m-1)} + b_m \cdot x_{k-m} - (a_1 \cdot y_{k-1} + \dots + a_{n-1} \cdot y_{k-(n-1)} + a_n \cdot y_{k-n})$$

- Ecuación en diferencias del sistema

Las matemáticas del control

□ Transformada en Z, interpretación básica:



- $G(z) = Y(z)/X(z)$ para sistemas lineales dinámicos discretos.
- $G(z)$ es un cociente de polinomios, grado den $\geq$ grado num.

$$G(z) = \frac{Y(z)}{X(z)} = \frac{b_0 \cdot z^m + b_1 \cdot z^{m-1} + \dots + b_{m-1} \cdot z + b_m}{z^n + a_1 \cdot z^{n-1} + \dots + a_{n-1} \cdot z + a_n}, \quad n \geq m$$

- Es habitual presentar $G(z)$ dividiendo entre z^n :

$$G(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 \cdot z^{-1} + \dots + b_{m-1} \cdot z^{-(m-1)} + b_m \cdot z^{-m}}{1 + a_1 \cdot z^{-1} + \dots + a_{n-1} \cdot z^{-(n-1)} + a_n \cdot z^{-n}}$$

- Las raíces de los polinomios denominador (polos) y numerador (ceros) definen el comportamiento básico del sistema.
- Sistema estable: todos los polos con módulo < 1 . En este caso:
 - Régimen permanente: $G(1)$ =ganancia estática (con unidades).
 - Respuesta dinámica (transitorio) según posición de polos y ceros.

Las matemáticas del control

□ Transformada en Z, interpretación:

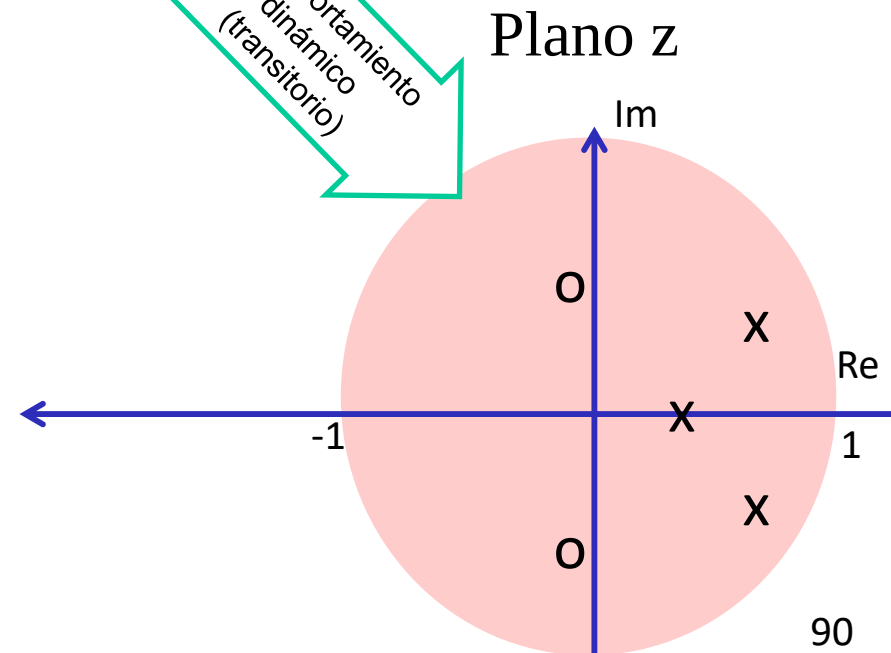


$$G(z) = \frac{Y(z)}{X(z)} = \frac{b_0 \cdot z^m + b_1 \cdot z^{m-1} + \dots + b_{m-1} \cdot z + b_m}{z^n + a_1 \cdot z^{n-1} + \dots + a_{n-1} \cdot z + a_n}, \quad n \geq m$$

Comportamiento
estático (régimen
permanente)

Comportamiento
dinámico
(transitorio)

$$K_p = \lim_{t \rightarrow \infty} \frac{\{y_k\}}{\{x_k\}} = \lim_{z \rightarrow 1} G(z)$$

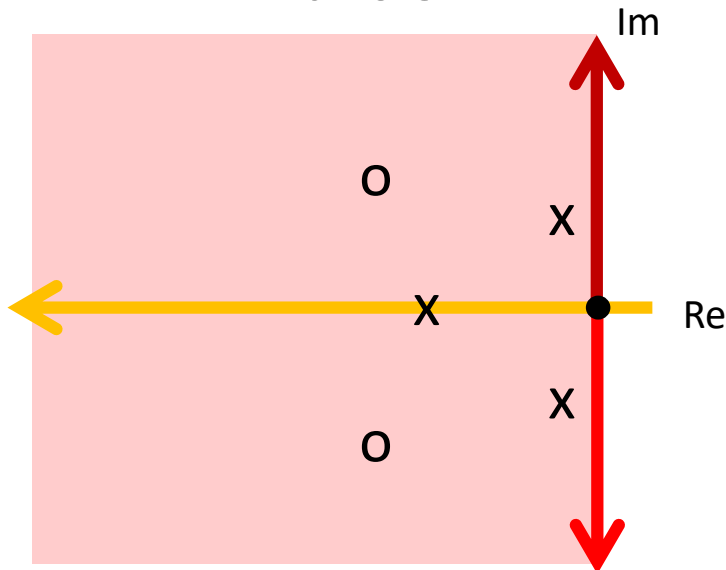


Las matemáticas del control

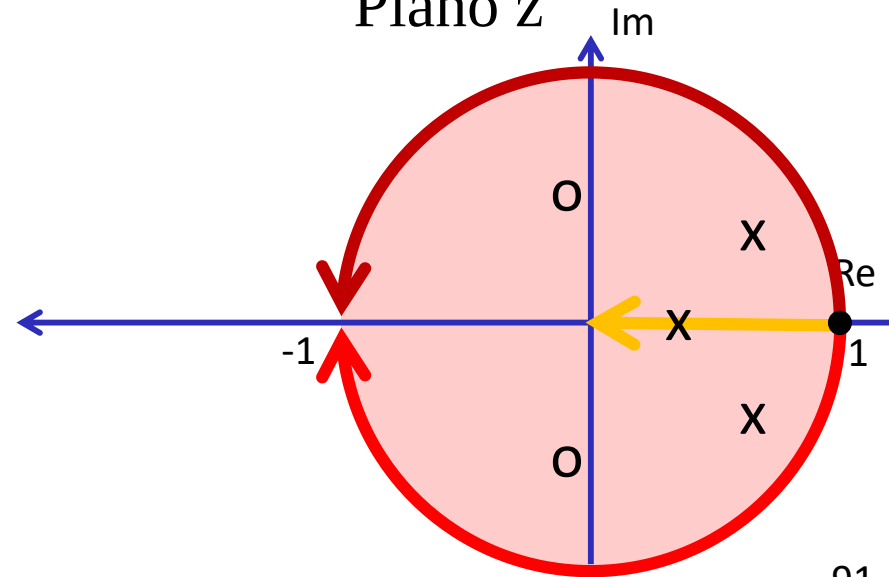
□ Sistema continuo vs sistema discreto:

- El semiplano de estabilidad $\text{Re}(s) < 0$ se convierte en el interior de la circunferencia de radio unidad $|z| < 1$
- $s=0 \equiv z=1$
- $s=\infty \equiv z=0$

Plano s

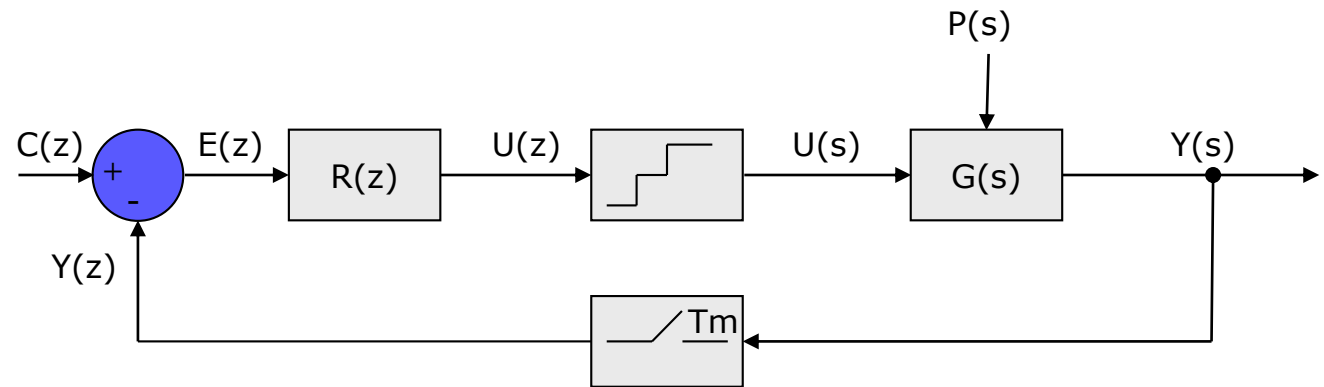


Plano z



Las matemáticas del control

□ El lazo de control híbrido en forma matemática:



□ En el lado continuo:

- $G(s) = Y(s)/U(s)$ $G(s)$: función de transferencia del sistema continuo

□ En la parte discreta:

- $E(z) = C(z) - Y(z)$
- $R(z) = U(z)/E(z)$ $R(z)$: función de transferencia del sistema discreto

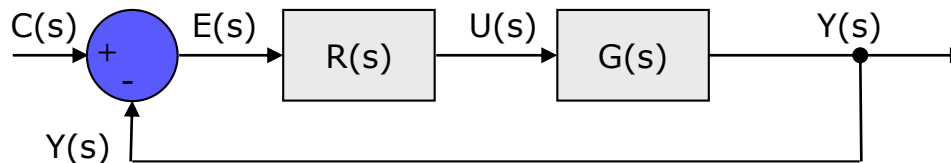
□ Elección de T_m :

- Suficientemente pequeño para que en el sistema realimentado se puedan 'seguir' las variaciones: $< 0.1 * T_{\text{sistema_realimentado}}$

Las matemáticas del control

□ Dos aproximaciones:

- **Suponer sistema completo continuo, calcular regulador continuo $R(s)$ y discretizar:**



$R(s)$ = resultado aplicación teoría Regulación Automática

$$R(z) = R(s) \frac{2}{T_m} \frac{z-1}{z+1} \quad (\text{Aproximación de Tustin})$$

(En Matlab)

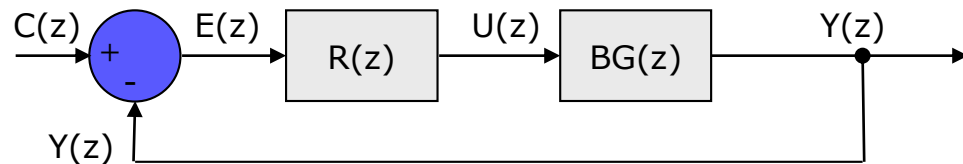
```
>> rs=tf([coefs numerador rs], [coefs denominador rs])
>> rz=c2d(rs,Tm,'tustin')
```

- **Suponer sistema completo discreto y calcular directamente regulador discreto**

Las matemáticas del control

□ Dos aproximaciones:

- Suponer sistema completo continuo, calcular regulador continuo $R(s)$ y discretizar.
- **Suponer sistema completo discreto y calcular directamente regulador discreto**



$BG(z)$ = discretización del conjunto bloqueador + sistema

(En Matlab)

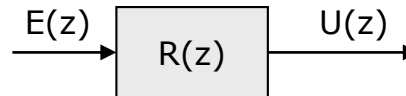
```
>> gs=tf([coefs numerador gs], [coefs denominador gs])  
>> bgz=c2d(gs,Tm,'zoh')
```

$R(z)$ = cálculo utilizando teoría de sistemas discretos

Las matemáticas del control

□ En cualquiera de los casos:

- $R(z)$ define la relación dinámica entre el error obtenido (entrada) y la salida de control a generar (salida)



- $R(z)$ es un cociente de polinomios en z . Poner en forma de potencias negativas, con $a_0=1$:

$$R(z) = \frac{U(z)}{E(z)} = \frac{b_0 + b_1 \cdot z^{-1} + \dots + b_{m-1} \cdot z^{-(m-1)} + b_m \cdot z^{-m}}{1 + a_1 \cdot z^{-1} + \dots + a_{n-1} \cdot z^{-(n-1)} + a_n \cdot z^{-n}}$$

- Convertir a ecuación en diferencias para el programa:

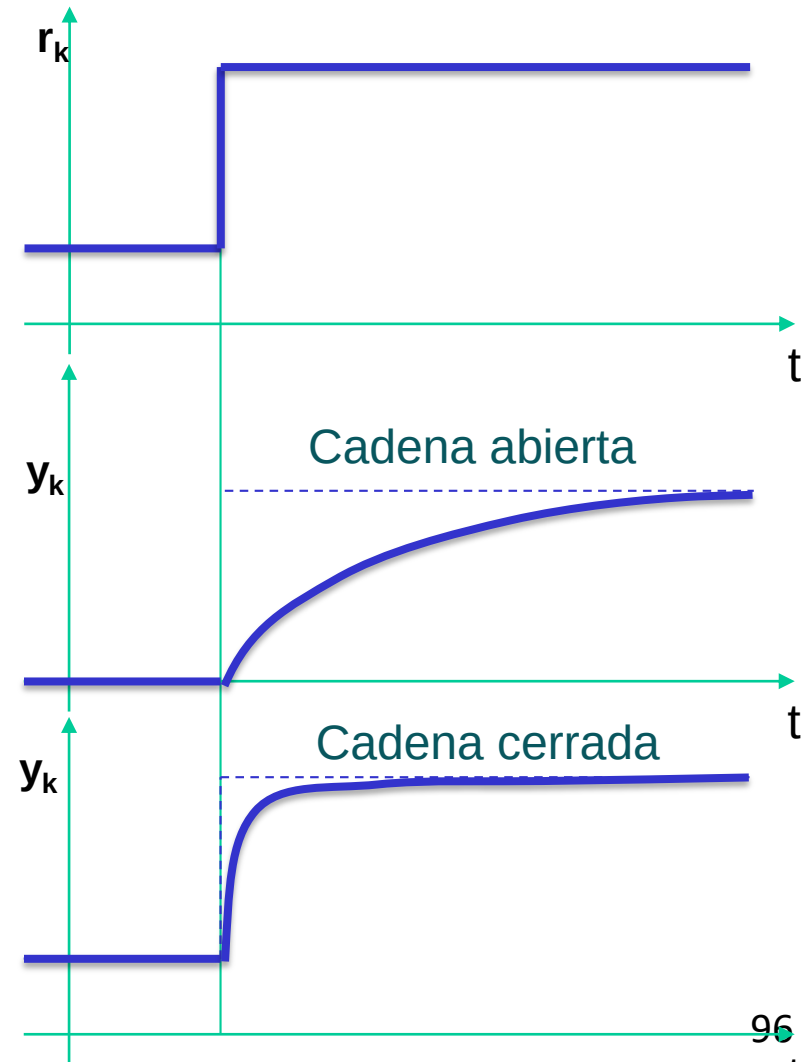
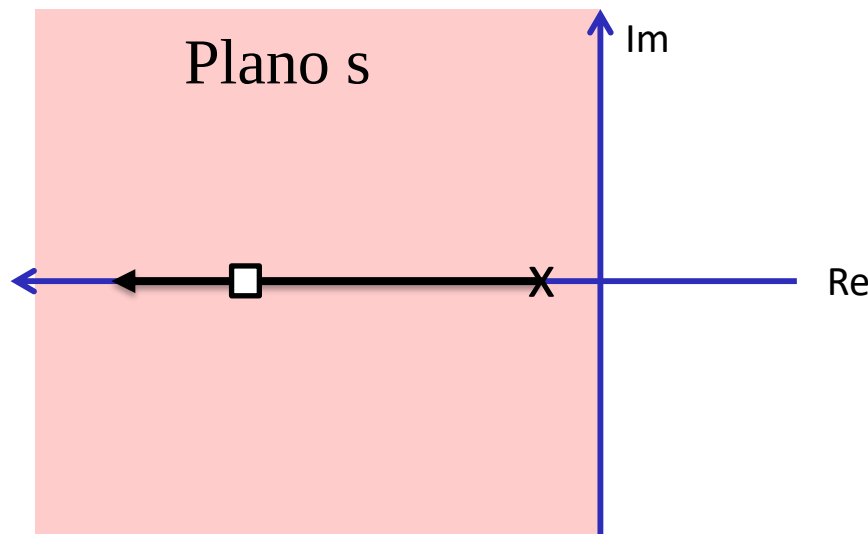
$$u_k = b_0 \cdot e_k + b_1 \cdot e_{k-1} + \dots + b_{m-1} \cdot e_{k-(m-1)} + b_m \cdot e_{k-m} - (a_1 \cdot u_{k-1} + \dots + a_{n-1} \cdot u_{k-(n-1)} + a_n \cdot u_{k-n})$$

- En forma de variables, según la interpretación desde diapo 24:

$$uk[0] = b[0].ek[0] + b[1].ek[1] + \dots + b[m].ek[m] - (a[1].uk[1] + \dots + a[n].uk[n])$$

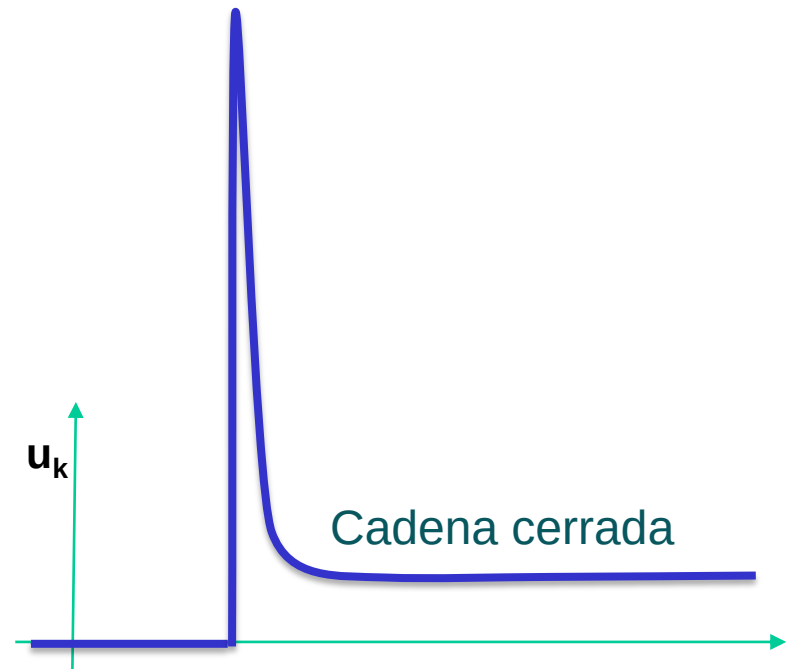
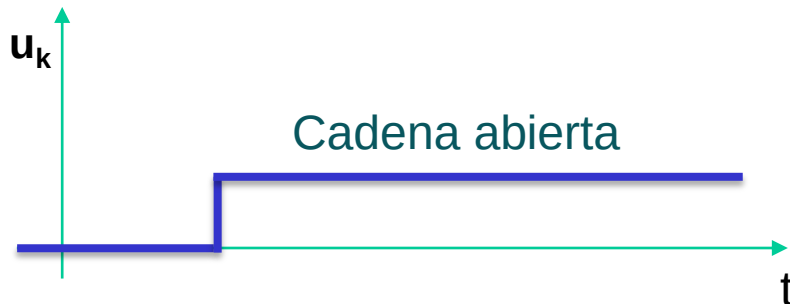
Las matemáticas del control

- ❑ ¡¡¡¡ ATENCION !!!! Las matemáticas “admiten todo”, la realidad no. Ejemplo :
 - En un sistema con un solo polo, se puede realimentar con K_p grande y conseguir una respuesta mucho más rápida que en cadena abierta.



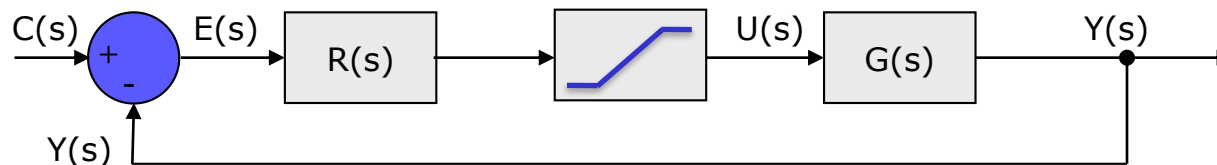
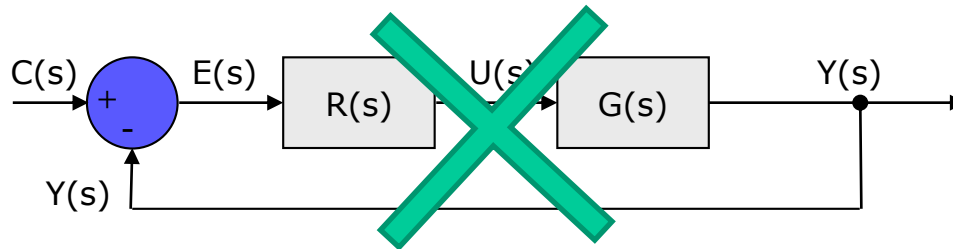
Las matemáticas del control

- ¡¡¡¡ ATENCION !!!! Las matemáticas “admiten todo”, la realidad no:
 - Para conseguir mayor velocidad, el controlador calcula acciones de control muy grandes al inicio, que son imposibles de generar en el sistema físico real



Las matemáticas del control

- Si se desea simular el comportamiento real, hay que añadir una saturación en el modelo que refleje los límites del accionador (comportamiento no-lineal)

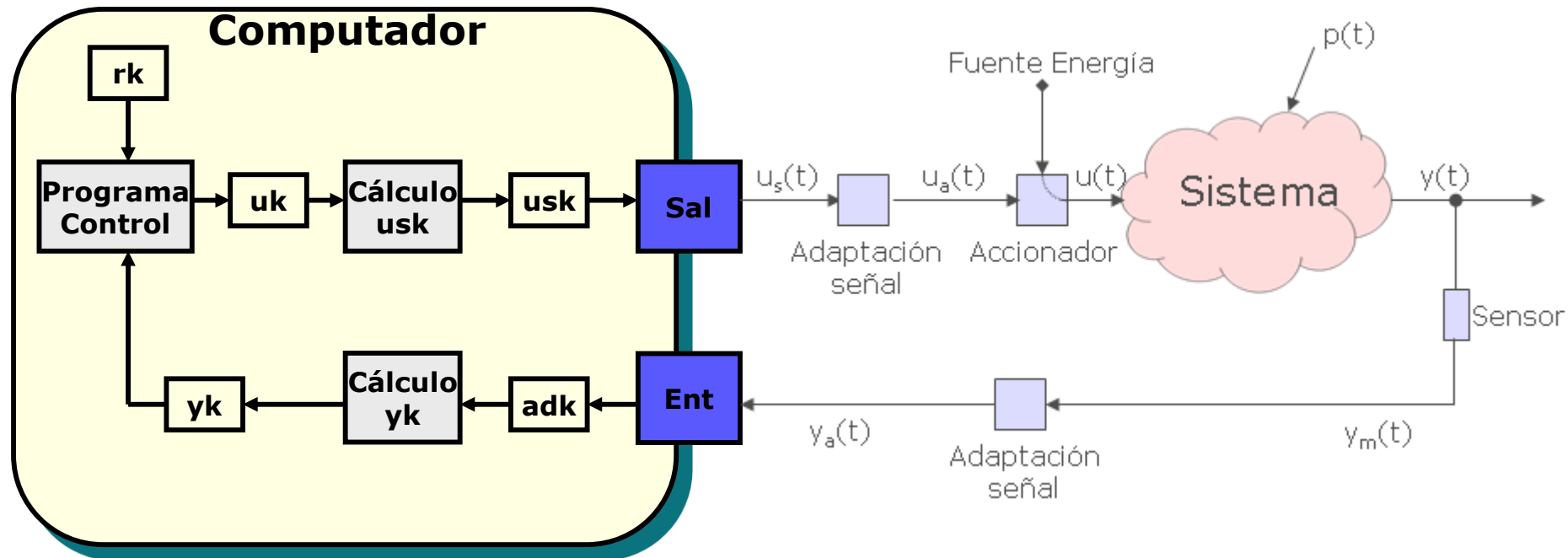


Indice

- ❑ Introducción control de procesos por computador
- ❑ Conceptos básicos para programación de control
- ❑ Interfaz del computador con el exterior
- ❑ El control secuencial
- ❑ Las matemáticas del control
- ❑ **Programación del lazo de control**
- ❑ Programación en lenguaje C
- ❑ Implantación del control en el computador

Programación del lazo de control

□ Programación del lazo de control:



Programación del lazo de control

- ❑ Programación del lazo de control:
 - Cada T_m (periodo de muestreo) se debe repetir el mismo algoritmo.
 - El algoritmo debe poder ejecutarse hasta un tiempo indefinido.
- ❑ Ejecución de un paso del lazo de control:
 - Actualización de tablas temporales
 - Adquisición de datos
 - Cálculo de valores adquiridos en sus unidades
 - Cálculo de la acción de control (regulador)
 - Cálculo de los valores de salida
 - Escritura de los dispositivos de salida
 - Espera al siguiente T_m

Programación del lazo de control

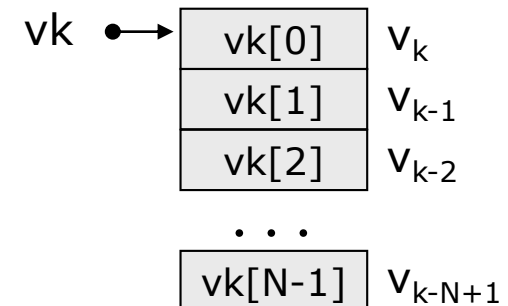
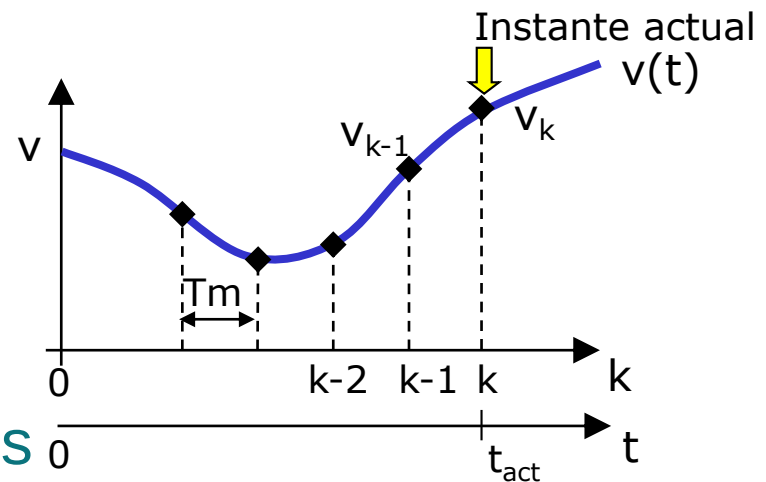
□ Variables para almacenar secuencias temporales:

- No es posible almacenar todos los valores desde el inicio del tiempo.
- Son necesarios el valor actual y algunos retrasados (n° finito) para los cálculos.
- Solución. Tabla de los N valores más recientes:

```
float vk[N];
```

```
// En cada instante:
```

```
// vk[i]  $\equiv$   $v_{k-i}$ 
```



Programación del lazo de control

□ Aspecto general de un programa de control

```
...

main()
{
    float c_k,y_k,u_k[N_MAX+1],e_k[M_MAX+1];
    otras variables necesarias (ej: m,n, tablas b[i],a[i])

    inicializaciones: m,n,b[i],a[i],e_k[i]=0,u_k[i]=0, etc.

    while (1)
    {
        Desplazar Tablas de variables temporales (al menos u_k, e_k)
        c_k=Nuevo valor consigna
        y_k=Valor leído de sensor
        e_k[0]=c_k-y_k;
        u_k[0]=CalculoAccionRegulador(e_k,u_k,...);
        AplicarAccionDeControl(u_k[0]);
        Otras operaciones auxiliares (comu operador, bb.dd., etc.)
        Sleep(Tm);
    }
}
```

Programación del lazo de control

□ Aspecto general de un programa de control

```

...
main()
{
    float c_k,y_k,u_k[N_MAX+1],e_k[M_MAX+1];
    otras variables necesarias (ej: m,n, tablas b[i],a[i])

    inicializaciones: m,n,b[i],a[i],e_k[i]=0,u_k[i]=0, etc.

    while (1)
    {
        Desplazar Tablas de variables temporales (al menos u_k, e_k)
        c_k=Nuevo valor consigna
        y_k=Valor leído de sensor
        e_k[0]=c_k-y_k;
        u_k[0]=CalculoAccionRegulador(e_k,u_k,...);
        AplicarAccionDeControl(u_k[0]);
        Otras operaciones auxiliares (comu operador, bb.dd., etc.)
        Sleep(Tm);
    }
}
    
```

Variables temporales (secuencias):

- Variable 'normal' si sólo es necesario valor actual
- Array si son necesarios valores anteriores

Variables atemporales

Programación del lazo de control

□ Aspecto general de un programa de control

```
...  
  
main()  
{  
    float c_k,y_k,u_k[N_MAX+1],e_k[M_MAX+1];  
    otras variables necesarias (ej: m,n, tablas b[i],a[i])  
  
    inicializaciones: m,n,b[i],a[i],e_k[i]=0,u_k[i]=0, etc.  
  
    while (1)  
    {  
        Desplazar Tablas de variables temporales (al menos u_k, e_k)  
        c_k=Nuevo valor consigna  
        y_k=Valor leído de sensor  
        e_k[0]=c_k-y_k;  
        u_k[0]=CalculoAccionRegulador(e_k,u_k,...);  
        AplicarAccionDeControl(u_k[0]);  
        Otras operaciones auxiliares (comu operador, bb.dd., etc.)  
        Sleep(Tm);  
    }  
}
```

Dar valores iniciales a todas las variables antes de iniciar el bucle (para secuencias, típicamente 0)

Programación del lazo de control

□ Aspecto general de un programa de control

```
...  
main()  
{  
    float c_k,y_k,u_k[N_MAX+1],e_k[M_MAX+1];  
    otras variables necesarias (ej: m,n, tablas b[i],a[i])  
  
    inicializaciones: m,n,b[i],a[i],e_k[i]=0,u_k[i]=0, etc.  
  
    while (1)  
    {  
        Desplazar Tablas de variables temporales (al menos u_k, e_k)  
        c_k=Nuevo valor consigna  
        y_k=Valor leído de sensor  
        e_k[0]=c_k-y_k;  
        u_k[0]=CalculoAccionRegulador(e_k,u_k,...);  
        AplicarAccionDeControl(u_k[0]);  
        Otras operaciones auxiliares (comu operador, bb.dd., etc.)  
        Sleep(Tm) ;  
    }  
}
```

Se ejecuta una pasada del bucle
cada Tm: cada ejecución del
bucle corresponde a un nuevo instante

Programación del lazo de control

□ Aspecto general de un programa de control

```
...  
main()  
{  
    float c_k, y_k, u_k[N_MAX+1], e_k[M_MAX+1];  
    otras variables necesarias (ej: m, n, tablas b[i], a[i])  
  
    inicializaciones: m, n, b[i], a[i], e_k[i]=0, u_k[i]=0, etc.  
  
    while (1)  
    {  
        Desplazar Tablas de variables temporales (al menos u_k, e_k)  
        c_k=Nuevo valor consigna  
        y_k=Valor leído de sensor  
        e_k[0]=c_k-y_k;  
        u_k[0]=CalculoAccionRegulador(e_k, u_k, ...);  
        AplicarAccionDeControl(u_k[0]);  
        Otras operaciones auxiliares (comu operador, bb.dd., etc.)  
        Sleep(Tm);  
    }  
}
```

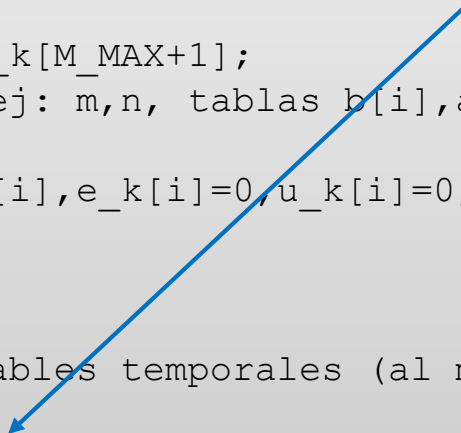
Si cada ejecución del bucle corresponde a un nuevo instante, los valores de las señales temporales están ahora retrasados 1 T_m : actualizar

Programación del lazo de control

□ Aspecto general de un programa de control

```
...  
  
main()  
{  
    float c_k,y_k,u_k[N_MAX+1],e_k[M_MAX+1];  
    otras variables necesarias (ej: m,n, tablas b[i],a[i])  
  
    inicializaciones: m,n,b[i],a[i],e_k[i]=0,u_k[i]=0, etc.  
  
    while (1)  
    {  
        Desplazar Tablas de variables temporales (al menos u_k, e_k)  
        c_k=Nuevo valor consigna  
        y_k=Valor leído de sensor  
        e_k[0]=c_k-y_k;  
        u_k[0]=CalculoAccionRegulador(e_k,u_k,...);  
        AplicarAccionDeControl(u_k[0]);  
        Otras operaciones auxiliares (comu operador, bb.dd., etc.)  
        Sleep(Tm);  
    }  
}
```

Obtener los valores actuales
(del nuevo instante) de las
diferentes señales

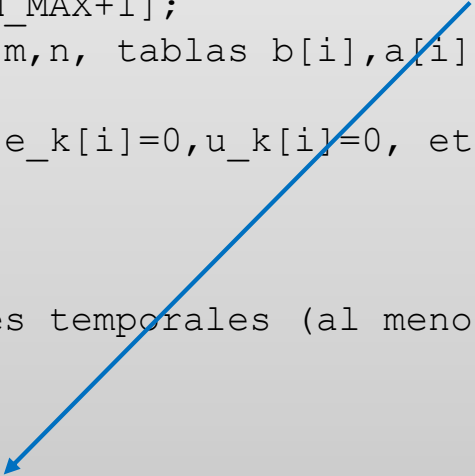


Programación del lazo de control

□ Aspecto general de un programa de control

```
...  
main()  
{  
    float c_k,y_k,u_k[N_MAX+1],e_k[M_MAX+1];  
    otras variables necesarias (ej: m,n, tablas b[i],a[i])  
  
    inicializaciones: m,n,b[i],a[i],e_k[i]=0,u_k[i]=0, etc.  
  
    while (1)  
    {  
        Desplazar Tablas de variables temporales (al menos u_k, e_k)  
        c_k=Nuevo valor consigna  
        y_k=Valor leído de sensor  
        e_k[0]=c_k-y_k;  
        u_k[0]=CalculoAccionRegulador(e_k,u_k,...);  
        AplicarAccionDeControl(u_k[0]);  
        Otras operaciones auxiliares (comu operador, bb.dd., etc.)  
        Sleep(Tm);  
    }  
}
```

Con los valores actualizados,
calcular la nueva acción de
control, y aplicarla al exterior



Programación del lazo de control

□ Aspecto general de un programa de control

```
...  
main()  
{  
    float c_k,y_k,u_k[N_MAX+1],e_k[M_MAX+1];  
    otras variables necesarias (ej: m,n, tablas b[i],a[i])  
  
    inicializaciones: m,n,b[i],a[i],e_k[i]=0,u_k[i]=0, etc.  
  
    while (1)  
    {  
        Desplazar Tablas de variables temporales (al menos u_k, e_k)  
        c_k=Nuevo valor consigna  
        y_k=Valor leído de sensor  
        e_k[0]=c_k-y_k;  
        u_k[0]=CalculoAccionRegulador(e_k,u_k,...);  
        AplicarAccionDeControl(u_k[0]);  
        Otras operaciones auxiliares (comu operador, bb.dd., etc.)  
        Sleep(Tm);  
    }  
}
```

Ahora puede haber tiempo para 'ocuparse' de tareas 'menos importantes'



Indice

- ❑ Introducción control de procesos por computador
- ❑ Conceptos básicos para programación de control
- ❑ Interfaz del computador con el exterior
- ❑ El control secuencial
- ❑ Las matemáticas del control
- ❑ Programación del lazo de control
- ❑ **Programación en lenguaje C**
- ❑ Implantación del control en el computador

Programación en lenguaje C

- Ver presentación específica de lenguaje C:

http://isa.uniovi.es/~ialvarez/Curso/descargas/lenguaje_c.pdf?version=1

Indice

- ❑ Introducción control de procesos por computador
- ❑ Conceptos básicos para programación de control
- ❑ Interfaz del computador con el exterior
- ❑ El control secuencial
- ❑ Las matemáticas del control
- ❑ Programación del lazo de control
- ❑ Programación en lenguaje C
- ❑ **Implantación del control en el computador**

Computadores para control

- ❑ Microcontrolador



- ❑ Procesador Digital de Señal (DSP)



- ❑ Dispositivos Electrónicos Programables (FPGA, PLD)

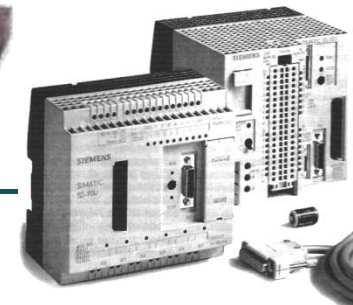
- ❑ Computador embebido



- ❑ Ordenador Industrial



- ❑ Autómata Programable (PLC)



Computadores para control

- ❑ **De propósito general (programables en C y otros lenguajes a alto nivel):**
 - **Microcontroladores:** muy bajo coste, prestaciones reducidas, un solo programa en ejecución, periferia integrada en chip (memoria, reloj, AD, DA, DI, DO, PWM, etc.)
 - **DSP:** como los μ C pero con más potencia de cálculo
 - **Basados en microprocesador:** prestaciones más elevadas, usan programa especial Sistema Operativo, periferia habitualmente fuera del chip
 - Embebidos: más cercanos a los μ C/DSP
 - De propósito general: PCs y similares
- ❑ **Para aplicaciones específicas (lenguajes de programación propios):**
 - **Autómatas programables (PLCs):** orientados a automatización industrial, E/S 24VDC y/o 220VAC, salidas de relé, lenguajes de programación LD, SFC, STL, etc.
 - **Dispositivos electrónicos programables (FPGA, PLD, etc.):** orientados a aplicaciones electrónicas de muy alta velocidad, procesamiento hardware en paralelo, lenguajes de programación VHDL, etc.

Implantación del control

- ❑ **Variables:** almacenan valores actuales y anteriores de entradas, salidas, estados y parámetros. ¡ Ojo unidades !
- ❑ **Funciones:** realizan cálculos habituales (media, rz, gráfico, ...).
- ❑ **Funcionamiento temporal:** los valores evolucionan en el tiempo.
- ❑ **Acceso a E/S:** es necesario intercambiar datos con el mundo real.
- ❑ **Tiempo-real:** importan el valor de la respuesta y su plazo.
- ❑ **Multi-tarea:** varias cosas que hacer 'a la vez'.
- ❑ **Programación orientada a eventos:**
la secuencia de acontecimientos es desconocida a priori
- ❑ **Ejemplos de eventos:**
 - Vencimiento temporización
 - Cambio de estado de entrada(s)
 - Entrada de datos por teclado
 - ...
- ❑ **Prioridades:** algunos eventos son más importantes que otros

Interrupciones

Interrupciones :

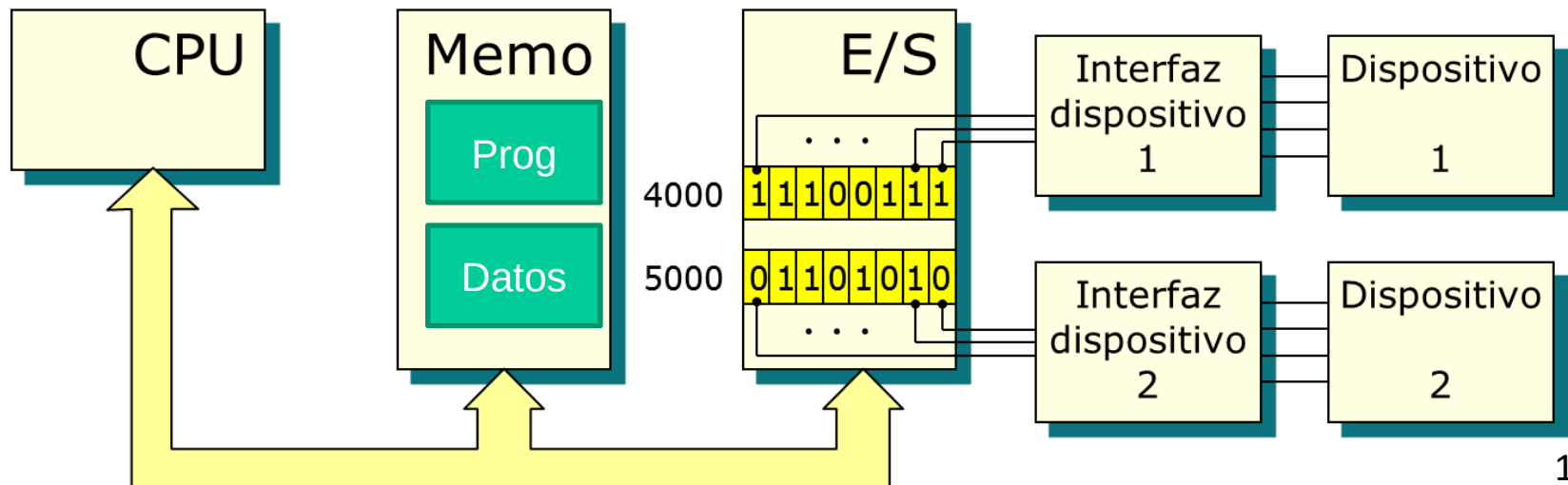
Señales binarias externas al computador que, cuando se activan, provocan que la CPU deje temporalmente de ejecutar el programa en curso, para ejecutar el código de tratamiento de la interrupción (ISR).

Una vez terminado el código de la ISR, la CPU sigue ejecutando el programa.

Implantación del control

□ Programación de E/S:

- El intercambio de datos con el mundo exterior se realiza escribiendo y leyendo puertos de E/S: direcciones “especiales” cuyo contenido provoca cambios o es modificado por los dispositivos de E/S.
- Los puertos están organizados en direcciones.
- Es necesario conocer las direcciones de E/S de cada dispositivo, y el significado de cada bit de los puertos.

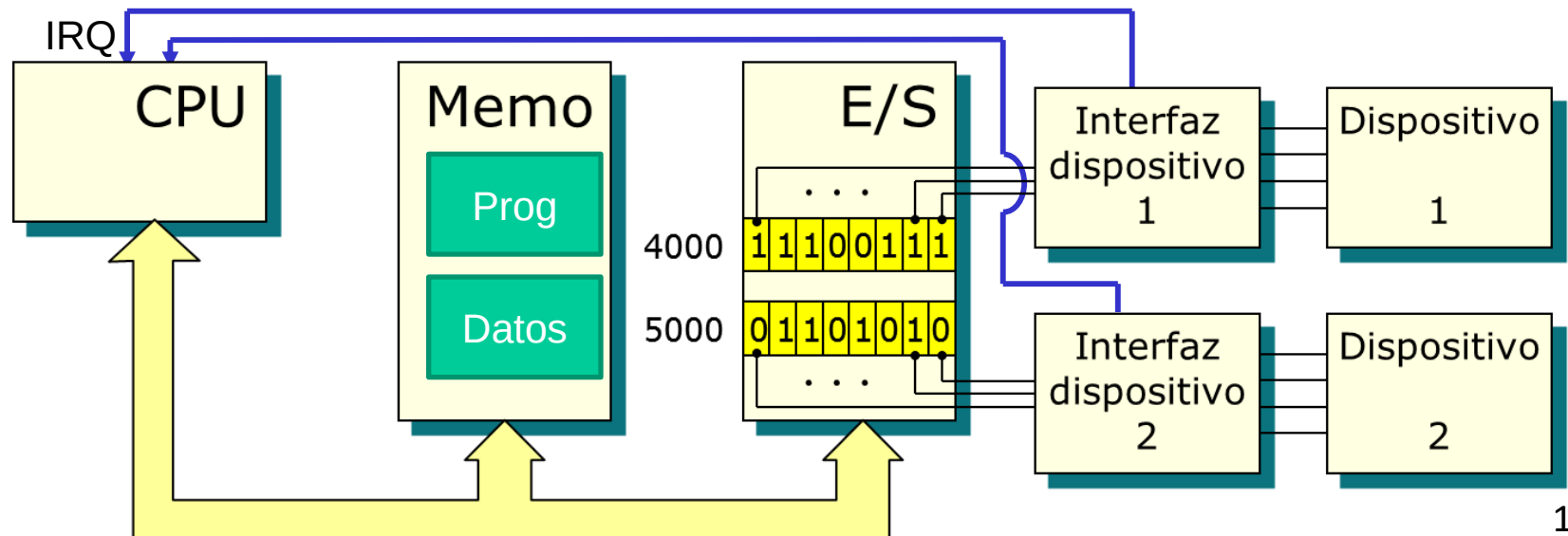


Programación de E/S

❑ Servicio de eventos:

- Los dispositivos de E/S envían una interrupción a la CPU cuando necesitan ser atendidos.
- La interrupción provoca que la CPU abandone temporalmente el programa que estaba ejecutando, pase a ejecutar el código de la ISR, retornando al programa cuando ésta termina.
- Ejemplo animado disponible en:

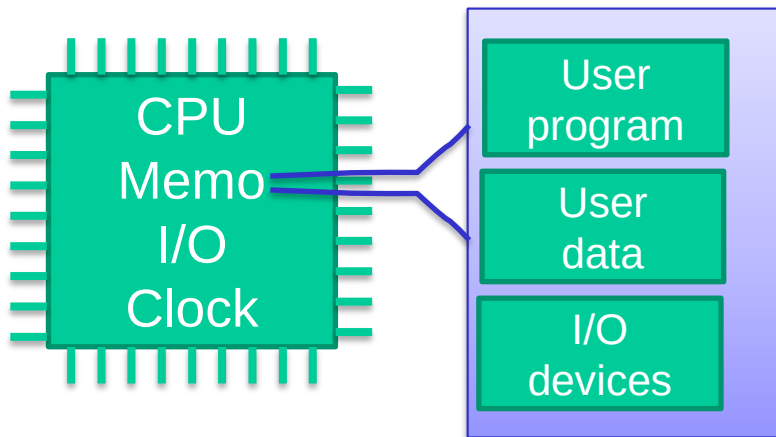
<http://isa.uniovi.es/~ialvarez/Curso/descargas/Funcionamiento%20Computador.pps>



Implantación del control

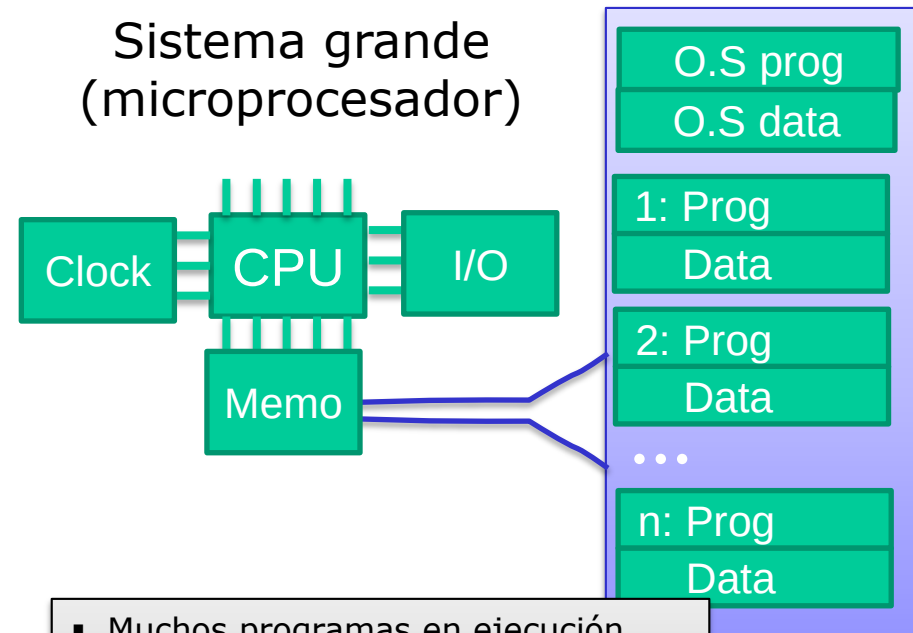
❑ Sistemas 'pequeños' vs 'grandes':

Sistema pequeño (microcontrolador)



- 1 solo programa en ejecución
- Recursos escasos
- El programador debe conocer:
 - Los dispositivos existentes
 - Las direcciones de E/S
 - El significado de los bits de cada dirección de E/S
 - La gestión de interrupciones

Sistema grande (microprocesador)



- Muchos programas en ejecución
- Recursos amplios
- Sistema Operativo gestiona todo: (memoria, CPU, I/O, etc.)
- El programador usa funcionalidades del S.O. para E/S
- CPUs multi-núcleo

Implantación del control

- ❑ Sistemas ‘pequeños’:
 - Sin soporte de Sistema Operativo
 - El programador accede directamente al hardware
 - Programación de puertos de E/S
 - Funciones para servicio de interrupciones
 - El programador debe realizar la planificación de la ejecución temporal de las diferentes tareas:
 - Round-robin: chequeo ordenado de las tareas a realizar en cada momento, sin uso de interrupciones.
 - Round-robin con interrupción: las tareas más prioritarias o puntuales son lanzadas por interrupciones hardware y servidas en Rutinas de Servicio de Interrupción (ISR).
 - Function-queue-scheduling: las ISR encolan las tareas a realizar con sus prioridades. Un planificador round-robin comprueba la cola y va sirviendo por orden de prioridad

Implantación del control

❑ Sistemas ‘grandes’:

- Una tarea especial Sistema Operativo (S.O.) se encarga de dar servicio a:
 - Acceso al hardware (modo síncrono y asíncrono):
 - Nivel de S.O.
 - Nivel de driver
 - Planificación de tareas:
 - Cada vez que sucede un ‘evento’, el S.O. toma el control, actualiza estado de tareas, y pasa a ejecutar la más prioritaria.
 - Gestión de memoria
 - El programador debe realizar llamadas a funciones del S.O. para todos los servicios
 - El S.O. puede llamar a funciones del programador (callback) para rutinas de servicio asíncrono

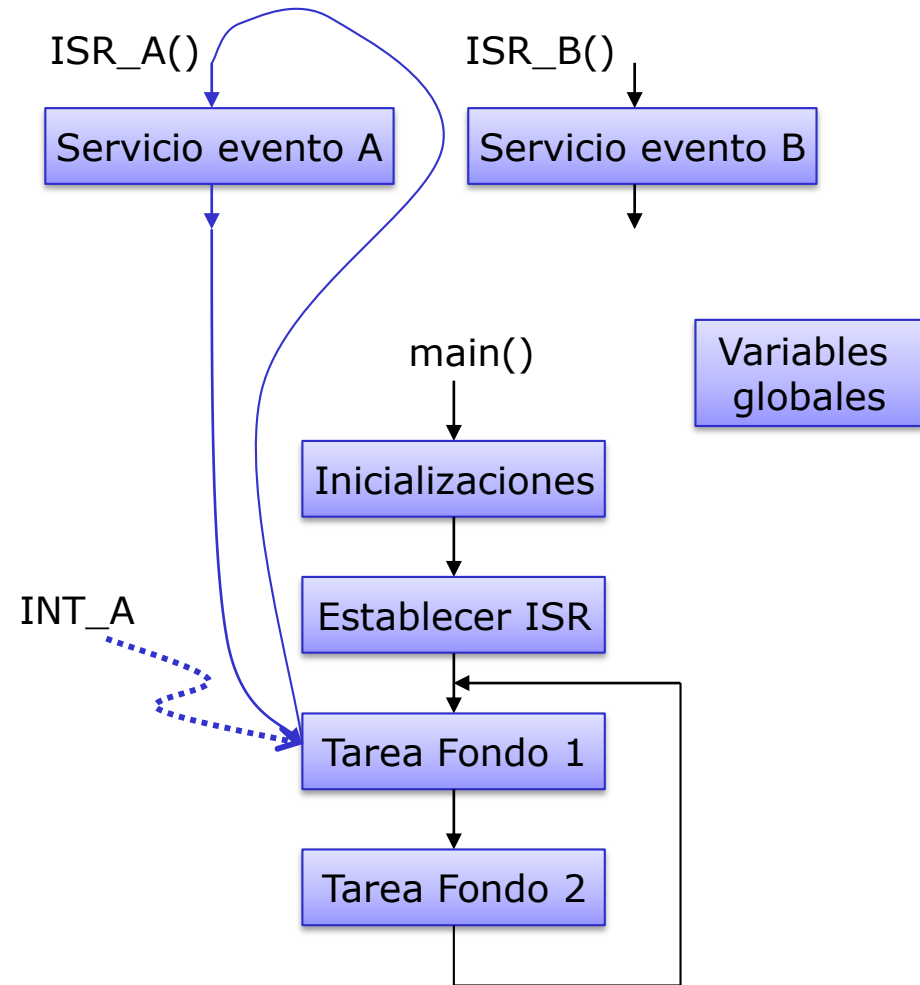
Implantación del control

- ❑ Sistemas “pequeños” vs sistemas “grandes”:
 - La tarea S.O. requiere recursos (memoria y tiempo de ejecución), que pueden no estar disponibles en sistemas pequeños.
 - La tarea S.O. requiere ejecutarse en modo privilegiado: la CPU debe disponer de 2 modos de funcionamiento (usuario y privilegiado).
 - La tarea S.O. facilita operaciones que pueden ser requeridas en sistemas grandes:
 - La programación de E/S (no hay que conocer todos los puertos e interrupciones)
 - La gestión de tareas (prioridades, sincronización, memoria)
 - Las protecciones de seguridad basadas en privilegios y claves.
 - Las comunicaciones

Sistemas “pequeños”

□ Organización del programa:

- Un bucle principal con la(s) tarea(s) de fondo.
- Funciones de Servicio de Interrupción (ISR) para el servicio de eventos.
- Las ISR detienen a la tarea de fondo o a otras ISR: ejecución rápida y sin esperas.
- Gestión de prioridades de las ISR.
- Las variables compartidas por el programa y las ISR deben ser globales.



Sistemas “pequeños”

❑ Variables globales:

- Se declaran al principio del programa, fuera de todas las funciones.
- Existen durante todo el programa, y son accesibles por todas las funciones (incluidas las ISR).
- Por ello, permiten el intercambio de datos entre funciones que no se llaman directamente (código principal e ISR).
- **!!! ATENCION !!!**
 - Es recomendable usar un nombre adecuado para distinguir las variables globales: comenzando por `_` (ej: `int _v1;`)
 - No abusar de las variables globales: utilizarlas sólo cuando sea estrictamente necesario.
 - El acceso “concurrente” a variables globales puede dar lugar a problemas de sincronización.

Sistemas “pequeños”

- ❑ Ejemplo con interrupciones:
 - Realización de un control todo/nada temporizado, con entrada de consigna por teclado.

```
#define TM_MS          100
...
float _ck;    // La consigna es global para que
              // sea accesible por main() y la ISR

void ISR_Control()
// Se ejecuta por interrupción cada TM_ms:
// (1) Debe ejecutarse rápido y sin esperas
// (2) Sus variables locales se crean y
//     destruyen en cada llamada
// (3) Se comunica con main() mediante
//     la variable global _ck
{
    int adk;
    float yk;

    adk=LeerCanalAD(...);
    yk=ConvertirValor(adk,...);
    if (_ck - yk > 0)
        ActivarSalidaControl(...);
    else
        DesactivarSalidaControl(...);
}
```

```
main()
{
    ...
    // Programa el dispositivo de E/S
    // temporizador para generar una interrupción
    // cada TM_ms, y establece que la
    // función de tratamiento será ISR_Control
    EstablecerISRTemporiz(TM_ms,ISR_Control);

    while (1)
    {
        // Este código será interrumpido cada
        // TM_ms para hacer un paso del control,
        // retornando después al mismo código.
        printf("Introduzca consigna:");
        scanf("%d",&_ck);
    }
}
```

Sistemas “pequeños”

- Ejemplo con interrupciones:
 - Realización de un control $R(z)$ con entrada de consigna por teclado.

```
#define TM_MS      100
#define M          2
#define N          3
...
float _ck;
float _a[N+1], _b[M+1], _uk[N+1], _ek[M+1];

void ISR_Control()
{
    int adk, dak;
    float yk;

    DesplazaTabla(_uk, N+1);
    DesplazaTabla(_ek, M+1);
    adk=LeerCanalAD(...);
    yk=ConvertirValor(adk,...);
    _ek[0]=_ck - yk;
    _uk[0]= ProdEsc(_b, _ek, M+1)-
            ProdEsc(_a+1, _uk+1, N);
    dak=ConvertirValor(_uk[0],...);
    ActivarSalidaAnalogica(dak,...);
}
```

```
main()
{
    ...
    InicializacionVariables(...);
    EstablecerISRTemporiz(TM_ms, ISR_Control);

    while (1)
    {
        printf("Introduzca consigna:");
        scanf("%d", &_ck);
    }
}
```

Son globales porque necesitan mantener su valor entre una llamada y otra de la función (además, normalmente será main quien les dé los valores iniciales)

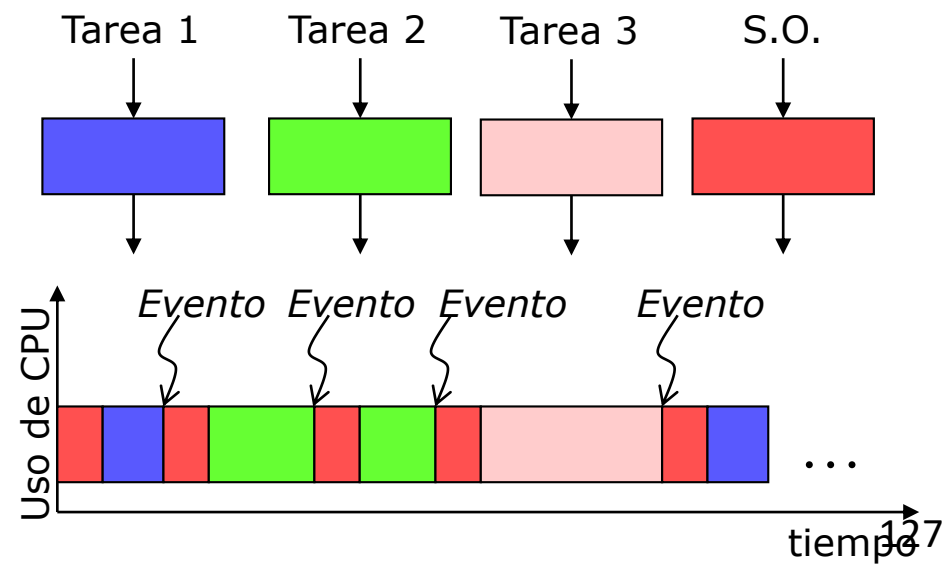
Son locales porque NO necesitan mantener su valor entre una llamada y otra de la función

Sistemas “grandes”

- ❑ El programador organiza su código en tareas, que se ejecutan “en paralelo” o, más exactamente, “en concurrencia”.
- ❑ Las tareas no siempre quieren ejecutar su código, están en muchas ocasiones en estado de espera (por teclado, por temporización, por aviso de otra tarea, ...)
- ❑ Una tarea especial, el Sistema Operativo, se encarga de:
 - Gestión de puertos E/S mediante pequeños programas (drivers)
 - Gestión de eventos y temporizaciones
 - Planificación de tareas
 - Gestión de memoria

Ante cualquier evento:

- El S.O. toma el control (la CPU pasa a ejecutar su código).
- El evento puede provocar el cambio de estado de alguna tarea.
- El S.O. revisa las tareas pendientes y cede el control a la más prioritaria que necesite ejecutarse, hasta el siguiente evento.



Sistemas “grandes”

- ❑ Variedad de Sistemas Operativos:
 - Windows, Unix, Linux, Mac OS, Android, VX-Works, ...
- ❑ Tipos de Sistemas Operativos:
 - De tiempo compartido:
 - El retardo en ejecutar una tarea no es importante.
 - El S.O. distribuye tiempos de ejecución entre todas las tareas.
 - De tiempo real:
 - El retardo en ejecutar una tarea puede ser muy importante.
 - El S.O. ejecuta siempre la tarea más prioritaria.
- ❑ POSIX: Portable Operating System Interface:
 - Basado en Unix
 - Mismo interfaz con las tareas (system calls) para diferentes Sistemas Operativos.